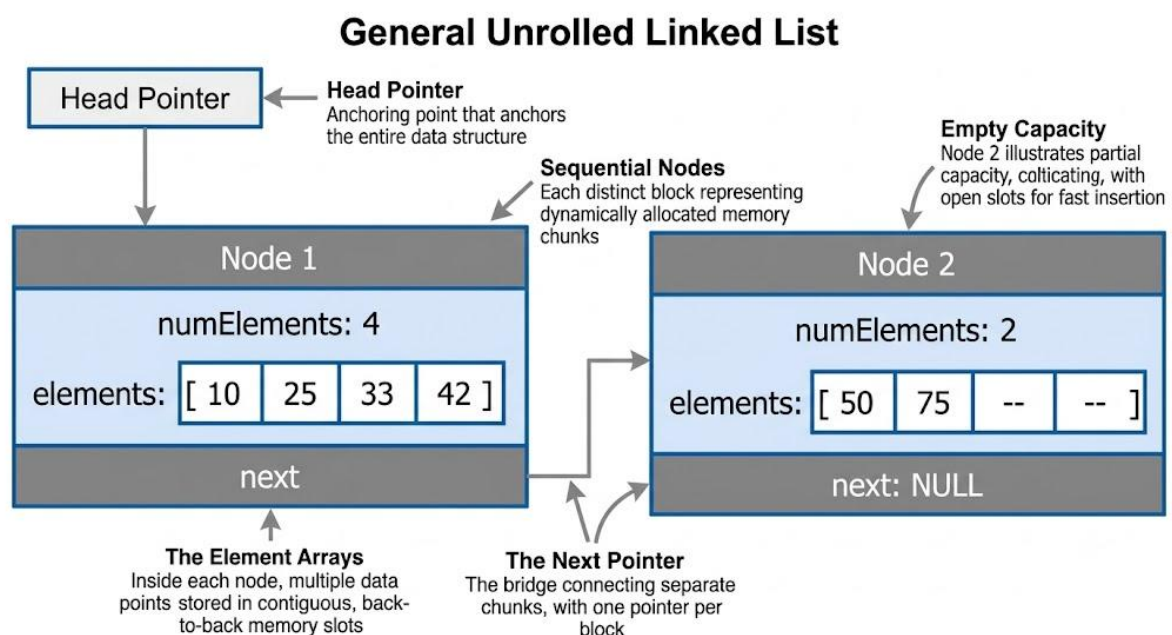


Unrolled List

–Introduction

An unrolled linked list is a hybrid data structure that bridges the gap between traditional linked lists and arrays. By combining the dynamic growth capabilities of a list with the fast, contiguous memory layout of an array, it creates a highly optimized system for managing large amounts of data.

Instead of storing just one piece of data per node, an unrolled linked list stores a fixed – size array of elements (often called a chunk or block) inside each node, alongside a single pointer directing to the next node in the chain.



Core Mechanics

- **Capacity – Based Insertion:** *When new data is added, the algorithm places it into the current node's array. The list only asks the computer for new memory to create a subsequent node when the current array reaches its absolute maximum capacity.*
- **Compacting Deletion:** *When an item is removed, the remaining elements within that specific node's array are shifted to fill the gap. If a node becomes entirely empty, it is surgically removed from the chain and its memory is deallocated.*
- **Balance Algorithms:** *In advanced implementations, if a node becomes less than half – full after deletions, the list may merge it with a neighboring node to prevent having a chain of nearly empty arrays.*

Primary Advantages

- **Cache Locality:** *Modern CPUs load data from main memory in blocks (cache lines). Because an unrolled list stores data sequentially in arrays, the CPU can load multiple elements into its high – speed cache at once. This makes searching and traversing dramatically faster than jumping to random memory*

addresses for every single element in a standard linked list.

- **Reduced Pointer Overhead:** *In a traditional linked list, a 64 – bit system wastes 8 bytes on a pointer for every single integer stored. Unrolled lists slash this overhead by sharing a single pointer across multiple elements (e. g., 1 pointer per 4, 64, or 256 elements).*
- **Dynamic Growth:** *Unlike standard arrays that suffer from expensive, massive memory reallocations when they run out of space, unrolled lists simply attach a new lightweight node to the end of the chain.*

Trade – offs and Limitations

- **Shift Penalties:** *While finding data is fast, inserting or deleting an item in the middle of an array chunk requires shifting the adjacent elements left or right.*
- **Internal Wasted Space:** *If a system heavily deletes items and nodes are left only partially full, the unused array slots within those nodes consume memory that isn't actively storing data (internal fragmentation).*

1. Unrolled List
–Insert Element
–Program Analysis

Program:

```
#define MAX_ELEMENTS 4

// Structure for the Unrolled Linked List Node
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

// Global head pointer
struct Node* head = NULL;

void insertElement(int val) {
    if (head == NULL) {
        head = (struct Node*)malloc(sizeof(struct Node));
        head->numElements = 1;
        head->elements[0] = val;
        head->next = NULL;
        cout << "Inserted " << val << " into a new head node.\n";
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }
}
```

```

else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
}
...
    case 1:
        cout << "Enter integer value to insert: ";
        cin >> value;
        insertElement(value);
        break;

```

Program Analysis

Head globally declared NULL ,through:

```

struct Node* head = NULL;

```

1. When the program starts:

*Every time we double – click the executable or run it from the terminal,
the operating system gives the program a completely fresh block of memory.*

*The C + + runtime looks at struct Node * head = NULL;*

and sets that memory address to NULL (0).

Because of this, every completely new run of the program starts with a perfectly clean, empty list.

2. head == NULL means the list is empty:

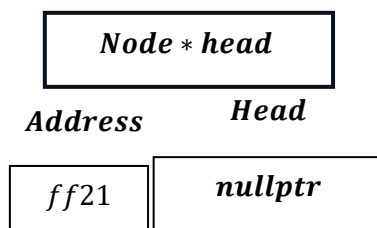
*In linked list design, the **head** pointer represents the doorway to our data.*

*If that doorway leads to nowhere (**NULL**), it is the mathematical guarantee that zero nodes exist in memory. That is why functions like **insertElement** and **deleteElement** always check if (**head == NULL**) first.*

***Head** is only automatically NULL the very first millisecond the program launches.*

After that, it only becomes NULL again if we explicitly make it NULL in our code .

*Therefore , during the start of our program , **Head** is **NULL**.*



```
cout << "Enter integer value to insert: ";
cin >> value;
insertElement(value);
break;
```

``Enter integer value to insert: ``

say, 20

*Function Call: **insertElement**(**value**: 20)*

```

if (head == NULL) {
    head = (struct Node*)malloc(sizeof(struct Node));
    head->numElements = 1;
    head->elements[0] = val;
    head->next = NULL;
    cout << "Inserted " << val << " into a new head node.\n";
    return;
}

```

∴ ***if(Head = NULL) is TRUE :***

head = (struct Node *)malloc(sizeof(struct Node));

And we have:

```

struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

```

On 32 – bit architecture:

- ***int numElements;*** = 4 bytes
- ***int elements[4];*** = 16 bytes (4 integers × 4 bytes)
- ***struct Node * next;*** = 4 bytes (on 32 – bit)

Total memory allocated: 24 bytes

On 64 – bit architecture:

- ***int numElements;*** = 4 bytes
- ***int elements[4];*** = 16 bytes (4 integers × 4 bytes)
- ***Padding*** : 4 bytes to align with 8 byte pointer variable ***next***.
- ***struct Node * next;*** = 8 bytes (on 64 – bit)

Total memory allocated: 32 bytes

Most modern system architecture follows 64 – bit , hence :

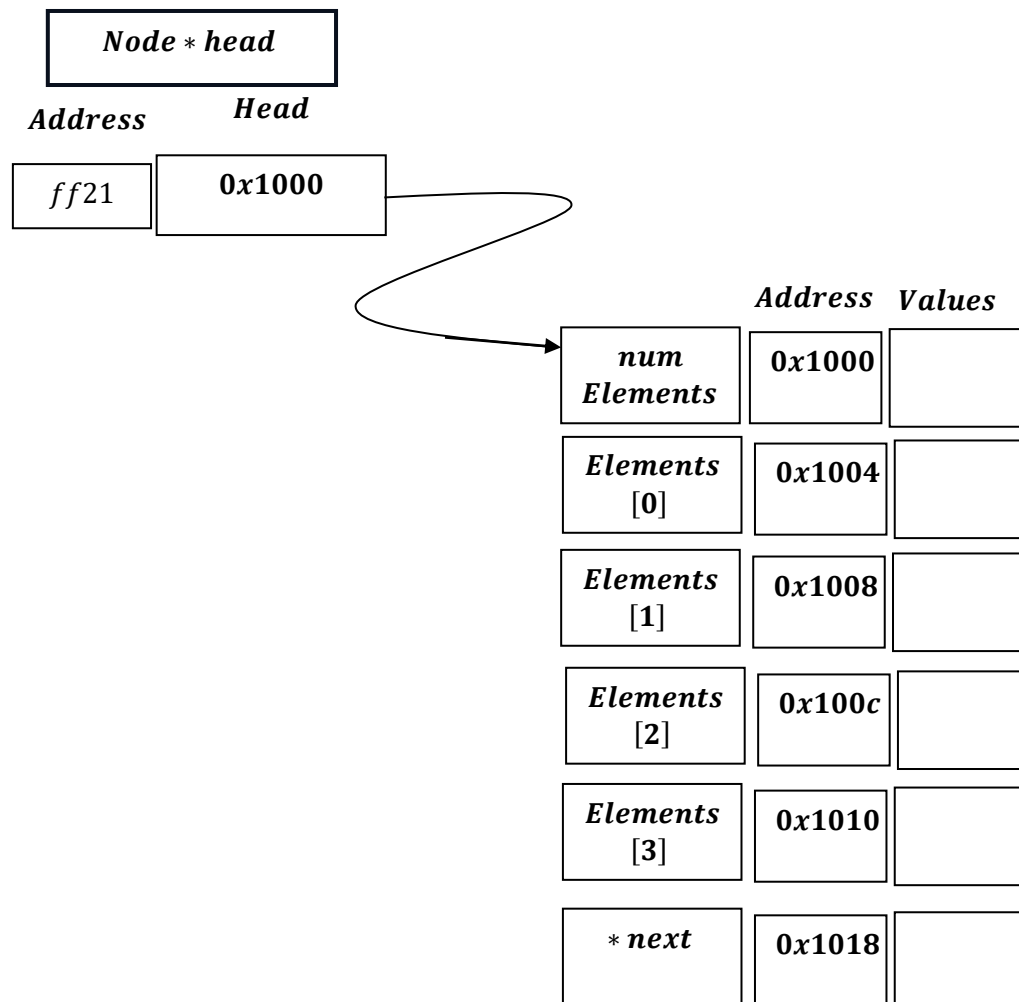
<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1000	<i>E.g.: numElements = 1, 2 ... etc.</i>	<i>numElements[0x1000 – 0x1003] 4 bytes for int</i>
0x1004	<i>E.g.: elements = {15, 42, ... etc. }</i>	<i>elements[0x1004 – 0x1013] 16 bytes for int array</i>
		<i>padding → [0x1014 – 0x1017] 4 bytes for padding</i>
0x1018	<i>E.g.: Address: 0x2000</i>	<i>* next → [0x1018 – 0x101F] 8 bytes for pointer variable</i>

Where as ,we have elements table like:

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1004	<i>elements[0]</i>	<i>elements[0] → [0x1004 – 0x1007]</i> <i>4 bytes for int</i>
0x1008	<i>elements[1]</i>	<i>elements[1] → [0x1008 – 0x100b]</i> <i>4 bytes for int</i>
0x100c	<i>elements[2]</i>	<i>elements[2] → [0x100c – 0x100f]</i> <i>4 bytes for int</i>
0x1010	<i>elements[3]</i>	<i>elements[3] → [0x1010 – 0x1013]</i> <i>4 bytes for int</i>

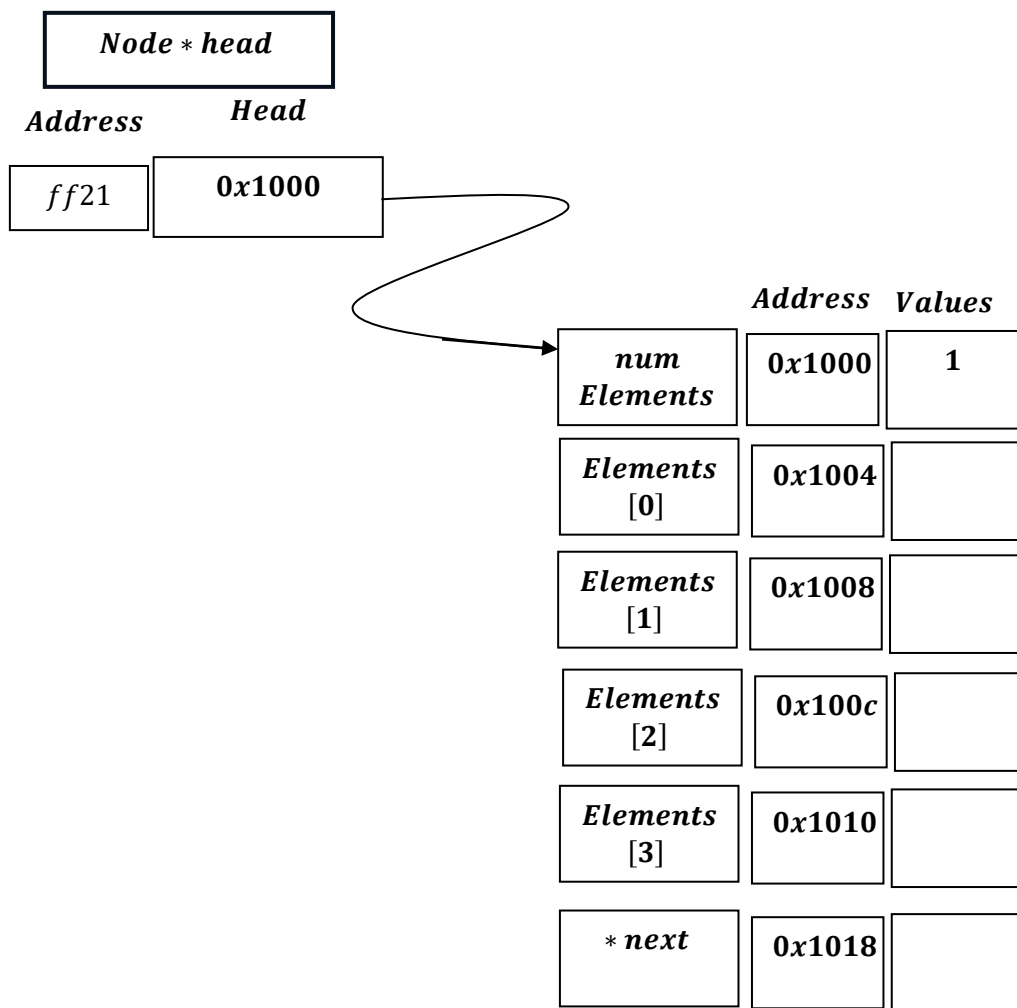
Now, head will start to point to the first element i. e. NumElements

–an integer type of the struct .



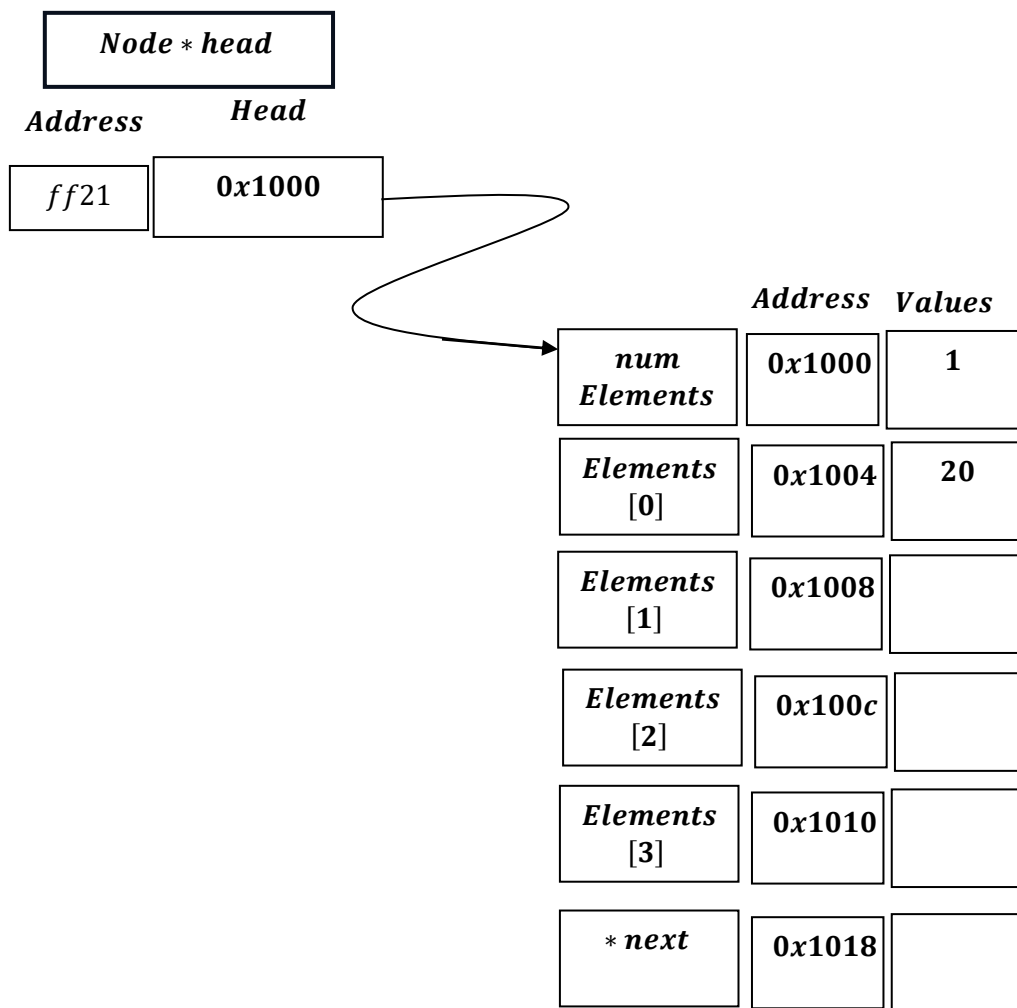
```
head->numElements = 1;
```

We get:



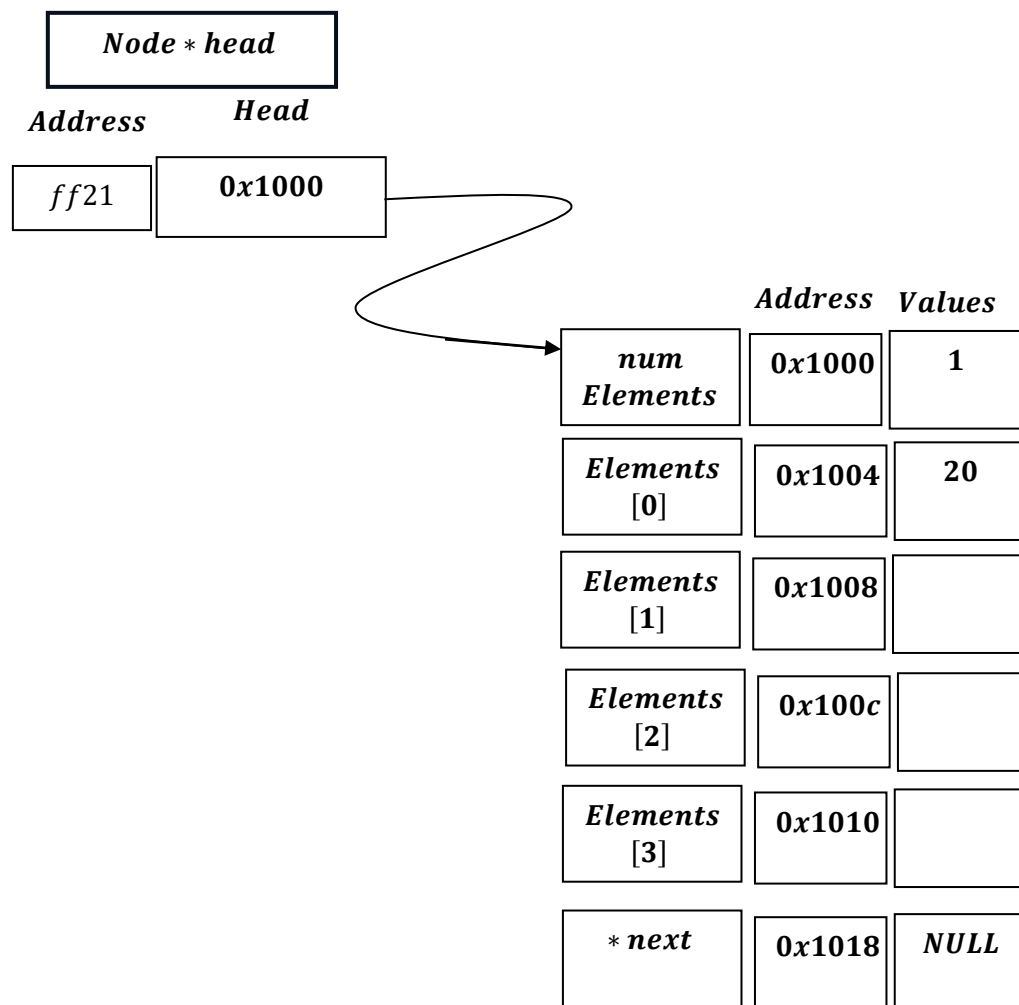
```
head->elements[0] = val;
```

Now, our val is : **20** (during function call)



```
head->next = NULL;
```

head → *next* = *NULL*;



Again say,

```
cout << "Enter integer value to insert: ";
cin >> value;
insertElement(value);
break;
```

``Enter integer value to insert: ``

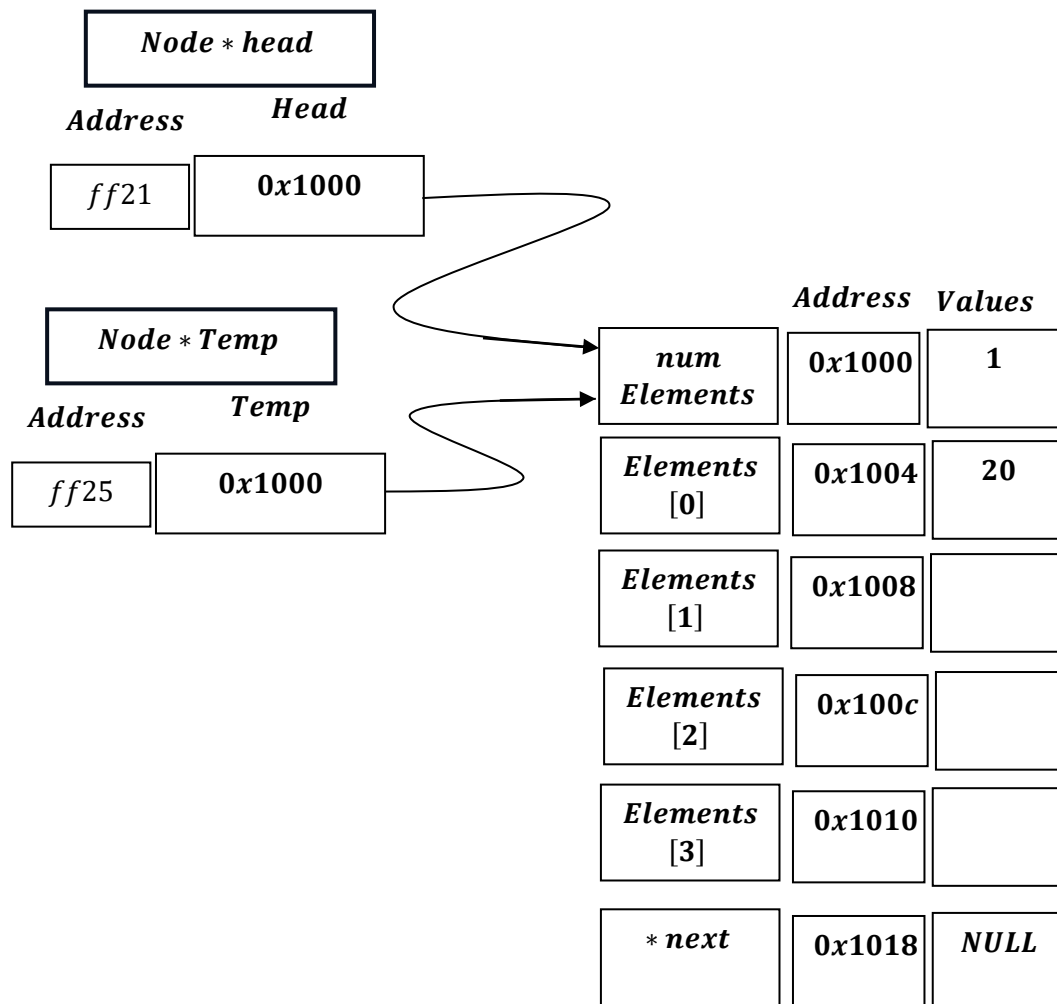
say, 30

Function Call: insertElement(**value**: 30)

Now ,Head is not NULL hence :

```
struct Node* temp = head;
```

*struct Node * temp = head = 0x1000;*



```
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

TEMP → NEXT = NULL ,hence condition of the while loop is false .

Therefore ,loop exists and goes to next line of code.

```

if (temp->numElements < MAX_ELEMENTS) {
    temp->elements[temp->numElements] = val;
    temp->numElements++;
    cout << "Inserted " << val << " into the existing last
node.\n";
}

```

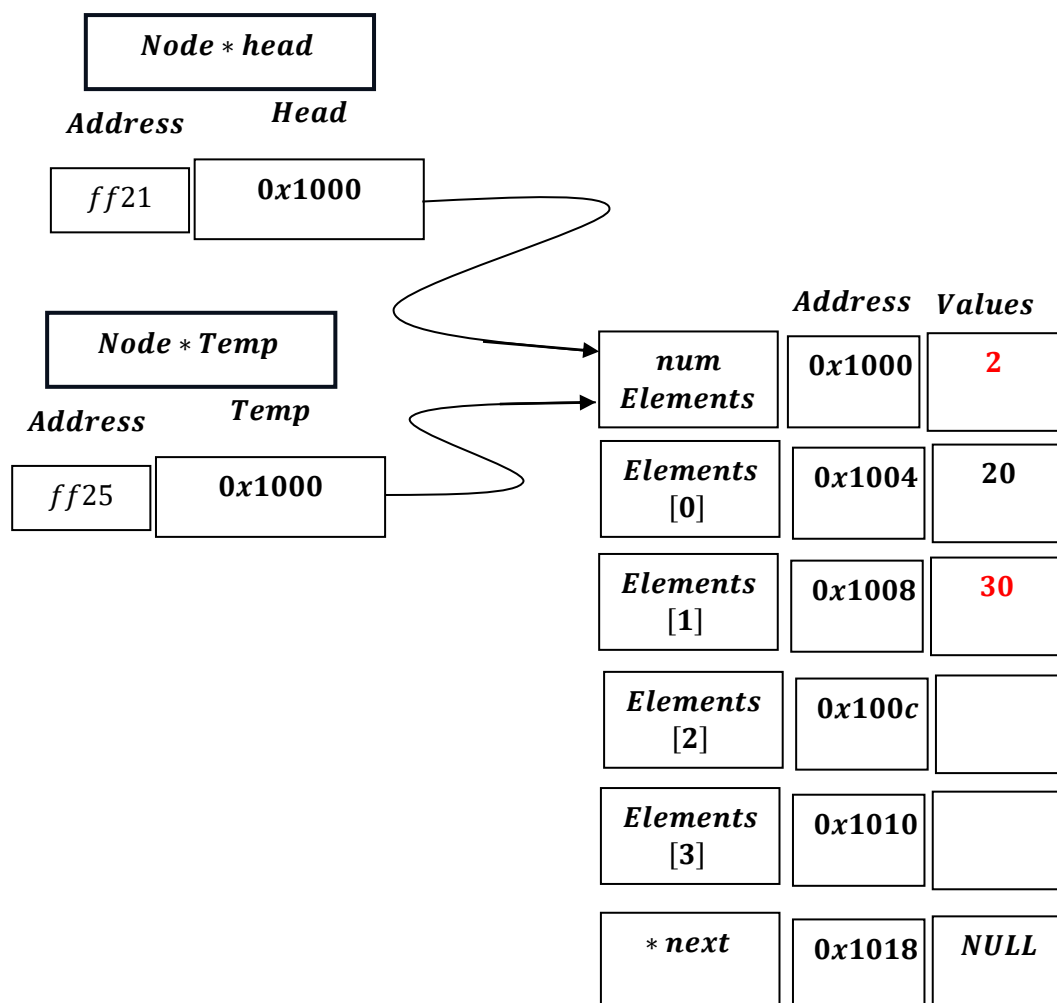
$if (temp \rightarrow numElements = 1 < MAX_ELEMENTS = 4) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$temp \rightarrow elements[temp \rightarrow numElements = 1] = val := 30;$

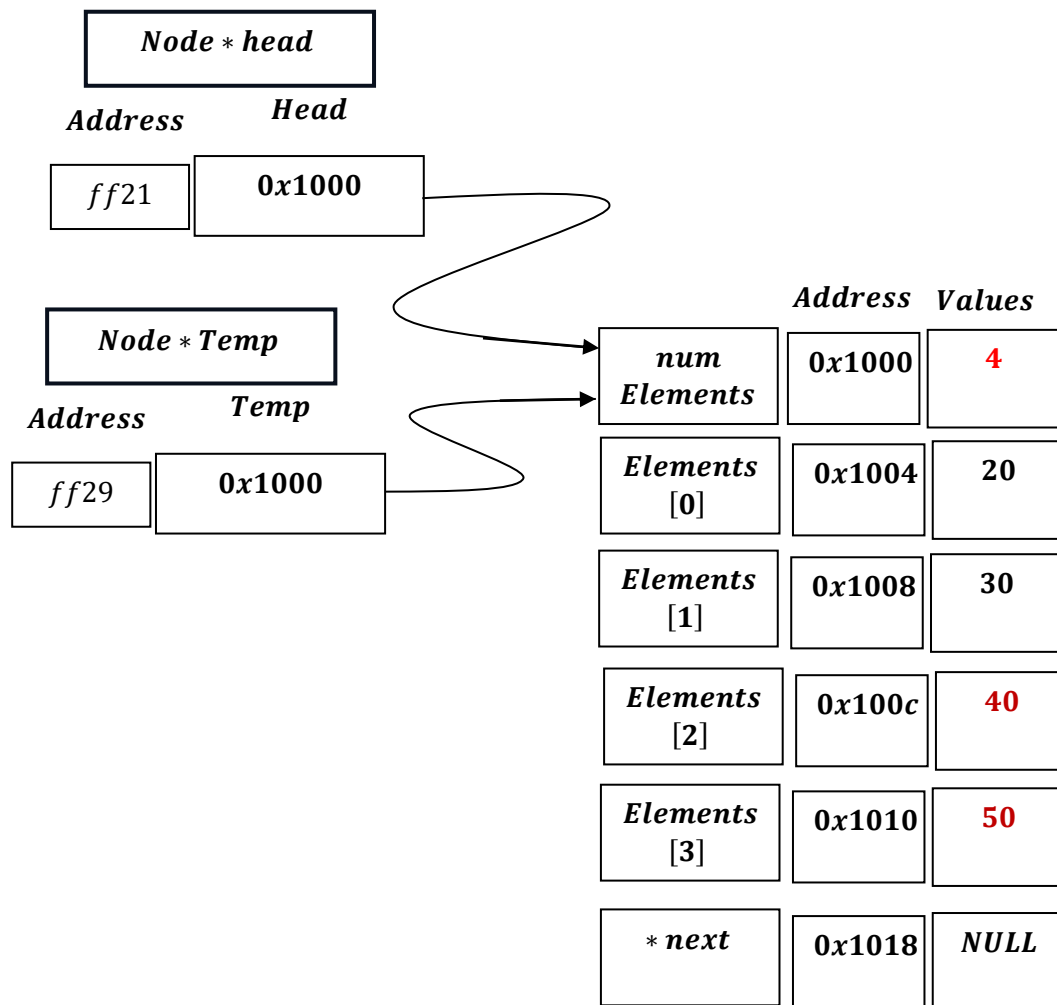
$temp \rightarrow numElements = temp \rightarrow numElements + 1 = 1 + 1 = 2;$

Print: "Inserted " val: 30 " into the existing last node."

[Exit if structure]



Next if we run two(2) times *Insert Element* functions we get:



On next function call:

```
cout << "Enter integer value to insert: ";
cin >> value;
insertElement(value);
break;
```

``Enter integer value to insert:``

say, 60

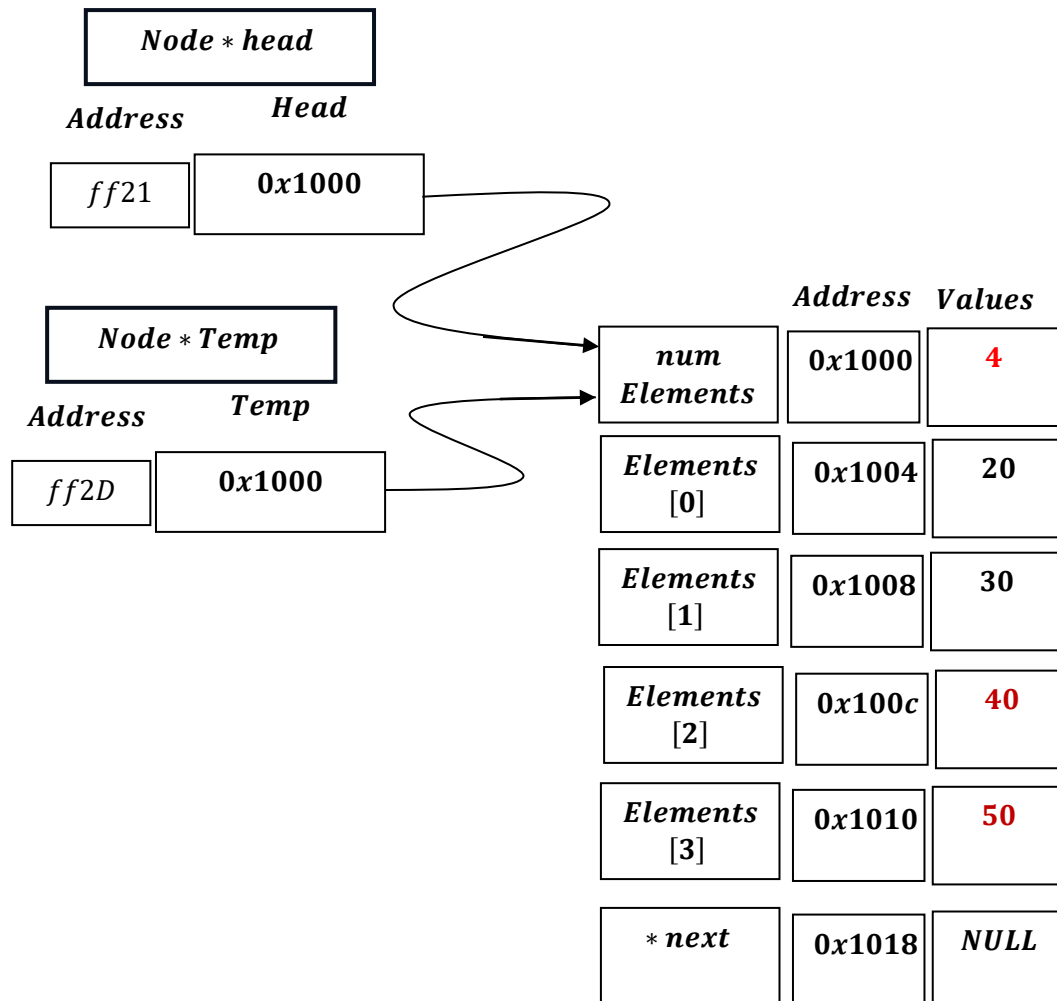
Function Call: *insertElement*(**value**: 60)

```
if (head == NULL)
```

Head $\neq$ NULL [Condition is False]

```
struct Node* temp = head;
```

Temp will be again point to Head



```
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

while (temp: 0x1000 $\rightarrow$ next: 0x1018) $\neq$ NULL is False , hence condition is false $\therefore$ [Loop Exits].

```
if (temp->numElements < MAX_ELEMENTS)
```

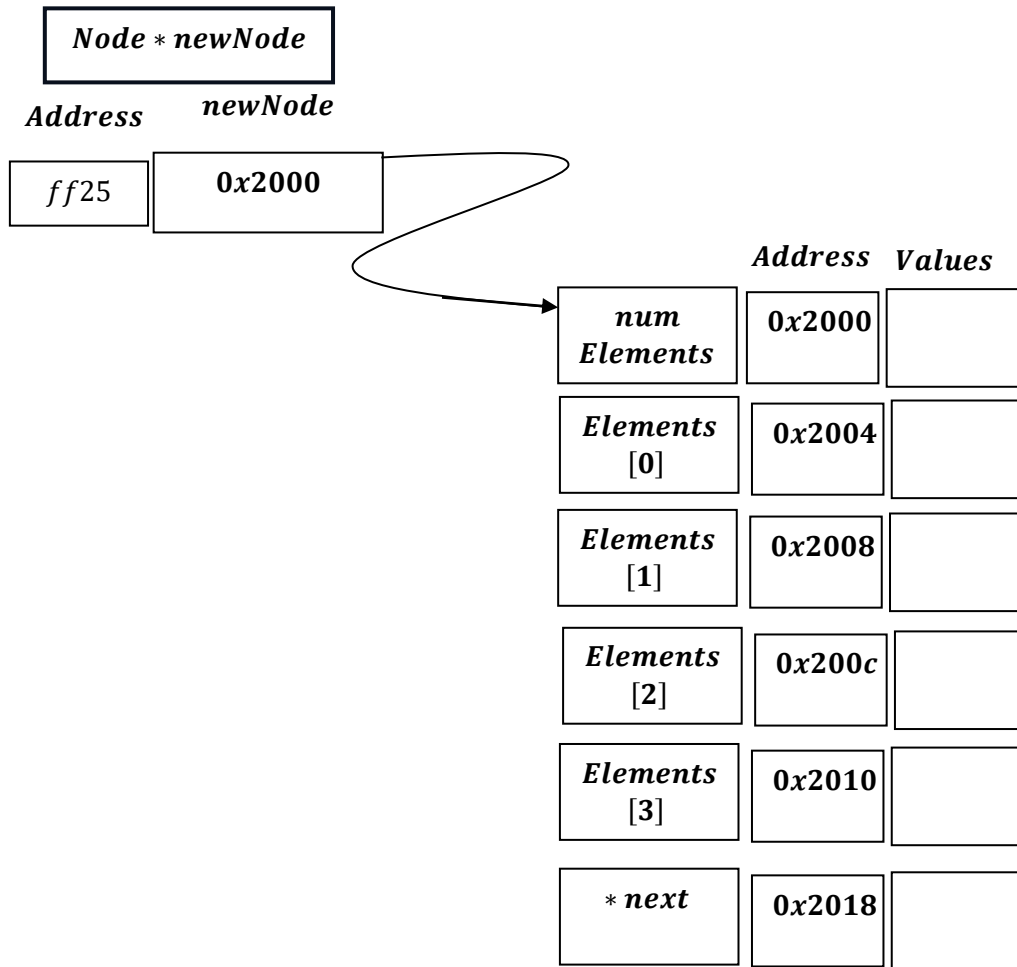
temp → *numElements*: 4 < *MAX_ELEMENTS* : 4 $\left[\begin{array}{l} \text{if condition false,} \\ \text{hence goes to} \\ \text{Else part.} \end{array} \right]$

```
else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

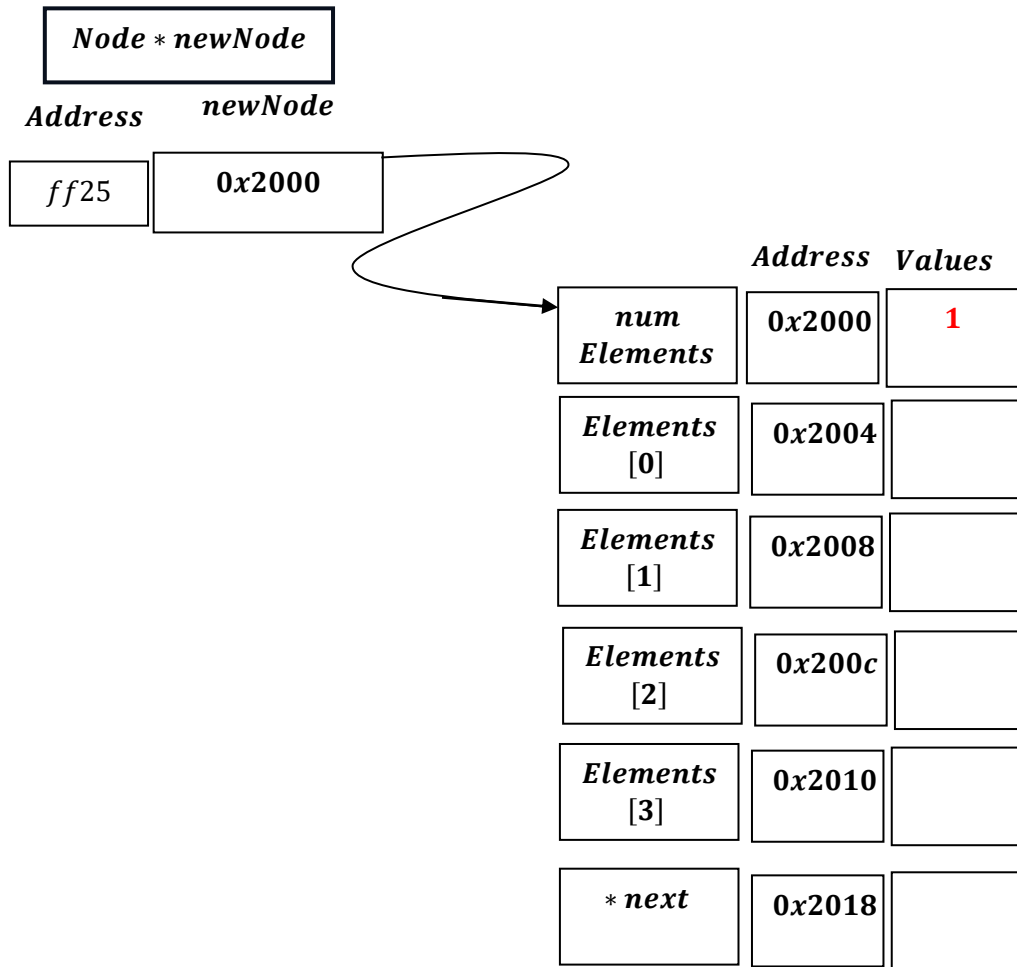
    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
```

i.e.

```
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
```



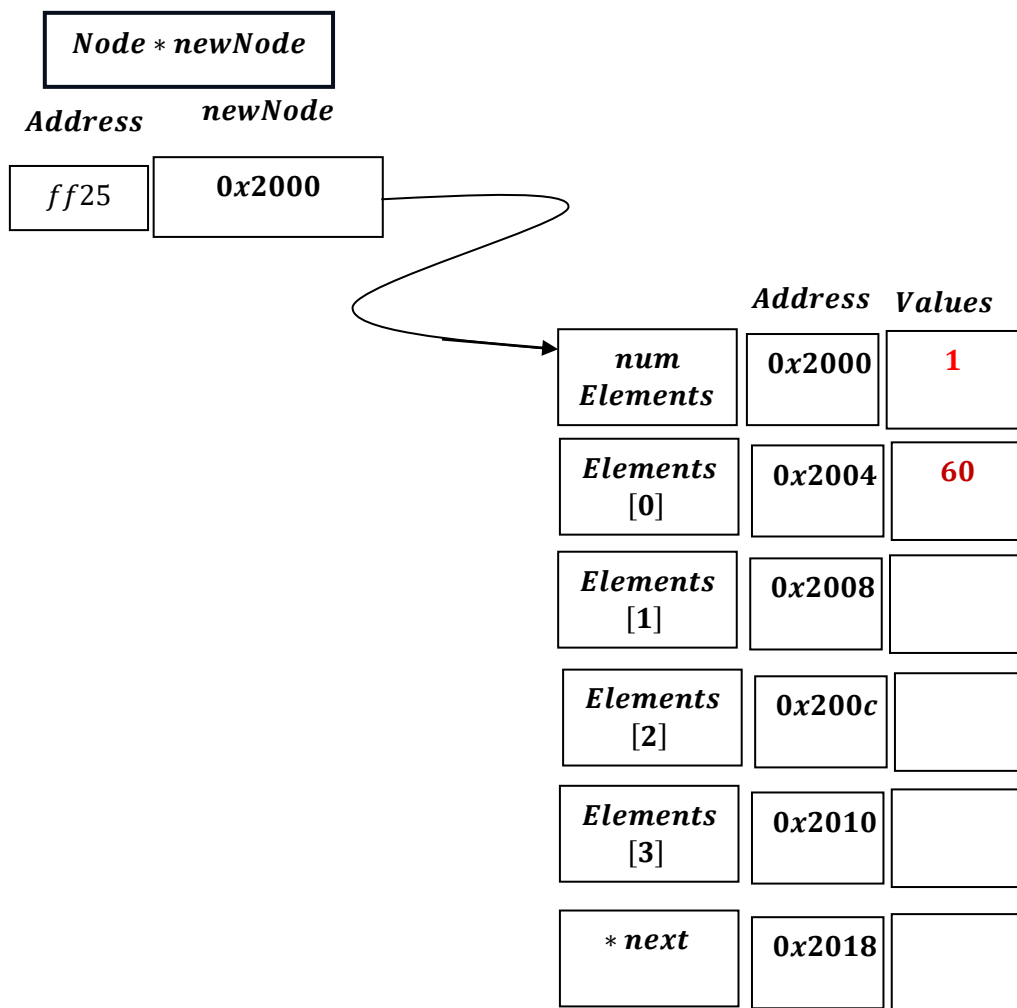
```
newNode->numElements = 1;  
newNode → numElements = 1.
```



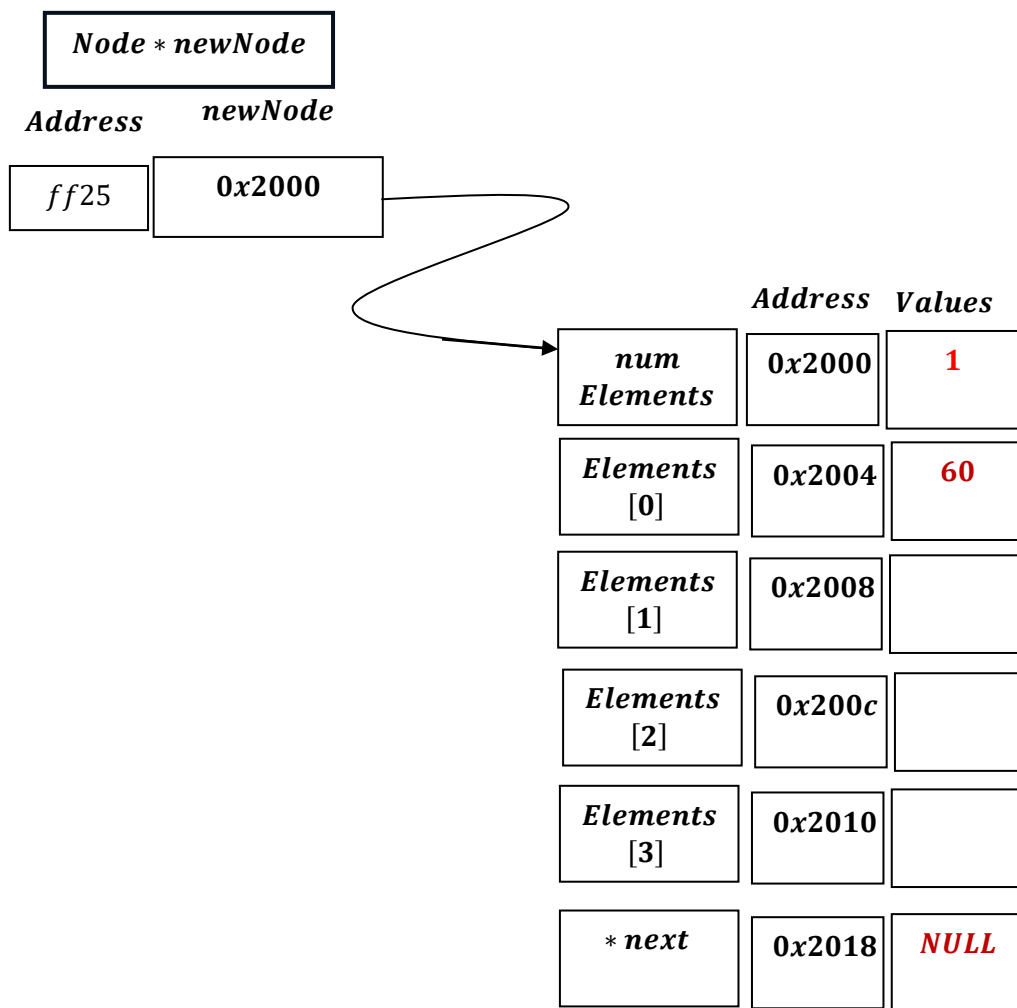
```
newNode->elements[0] = val;
```

Function Call: insertElement(*value*: 60)

newNode → *elements*[0] = *val* = 60.

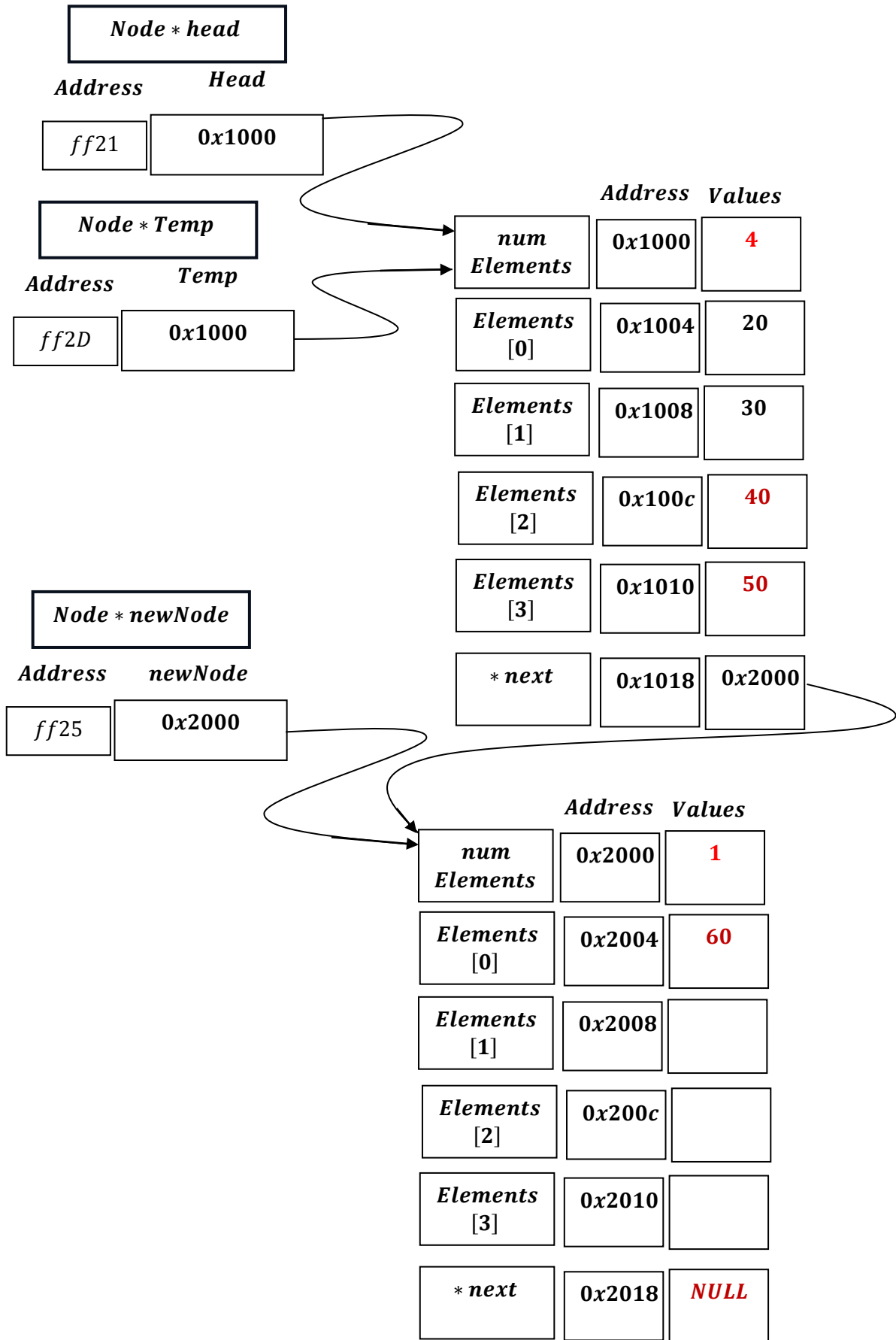


```
newNode->next = NULL;
```



```
temp->next = newNode;
cout << "Node full. Inserted " << val << " into a newly
created node.\n";
```

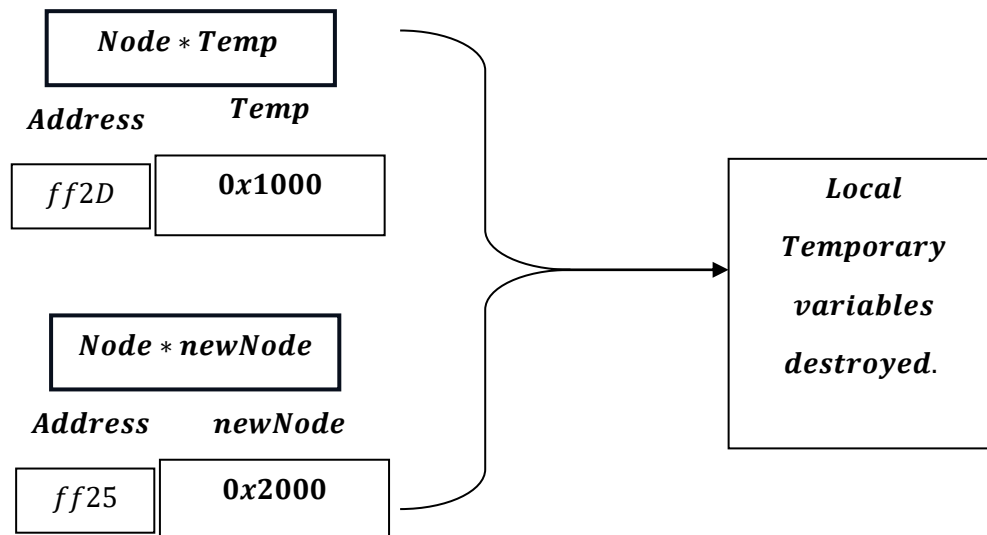
Temp → next = newNode = 0x2000.



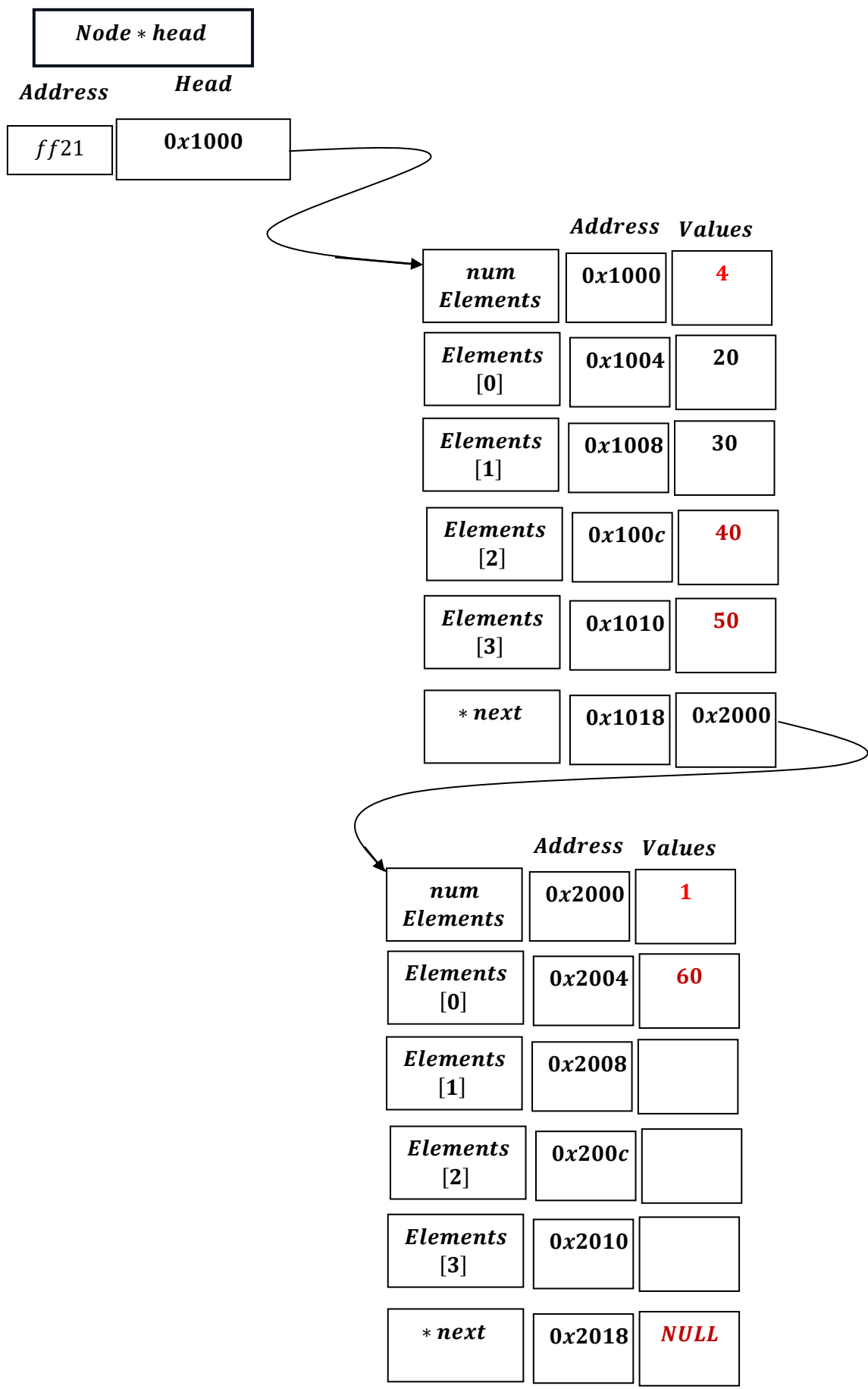
```
cout << "Node full. Inserted " << val << " into a newly  
created node.\n";
```

Print : `` Node full.Inserted `` val: 60 ``into a newly created node.``

As function ends, temporary local variables ends.



And we are left with:



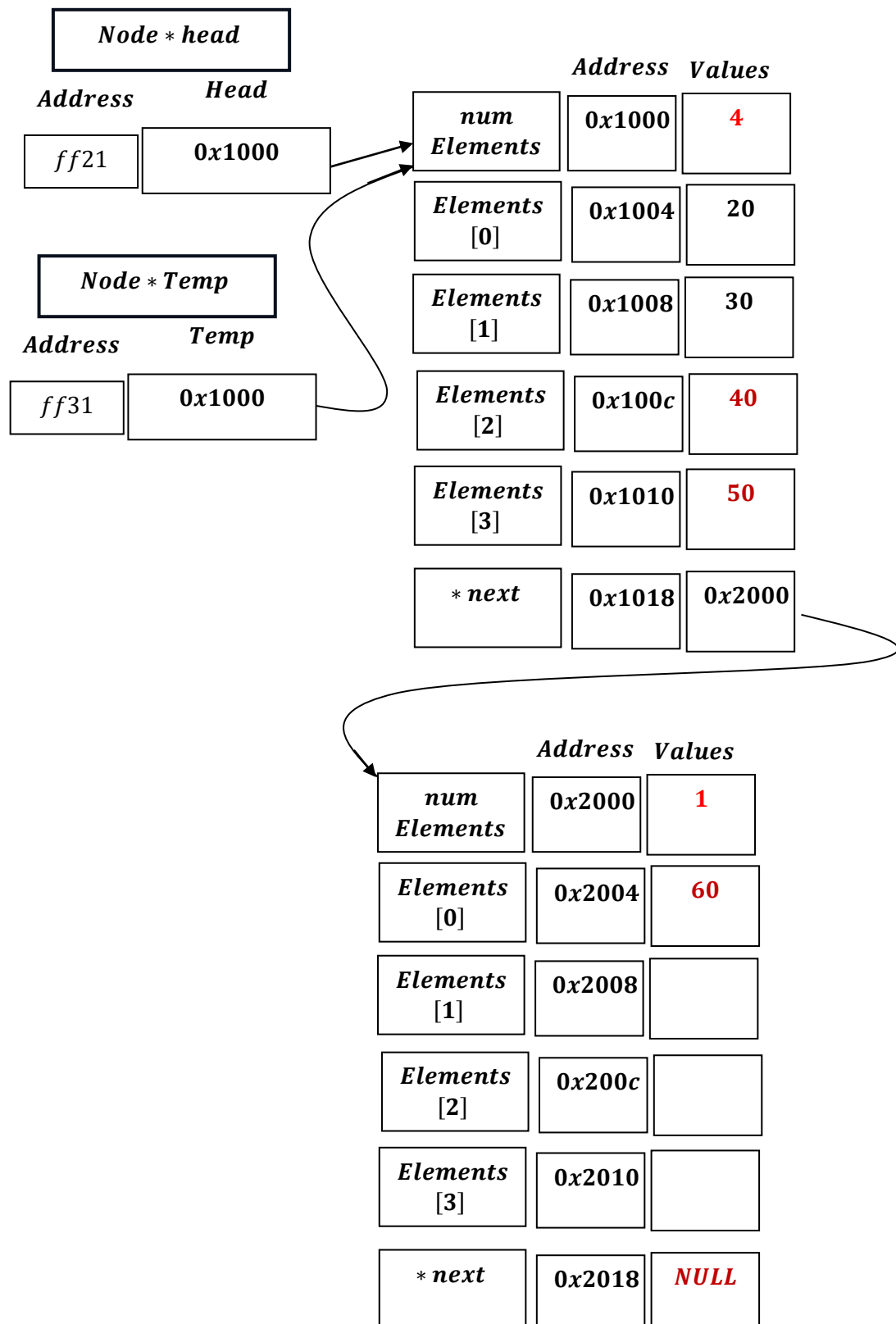
Next if we try to enter another value: **70**.

```
if (head == NULL)
```

Head $\neq$ *NULL* [Condition is False]

```
struct Node* temp = head;
```

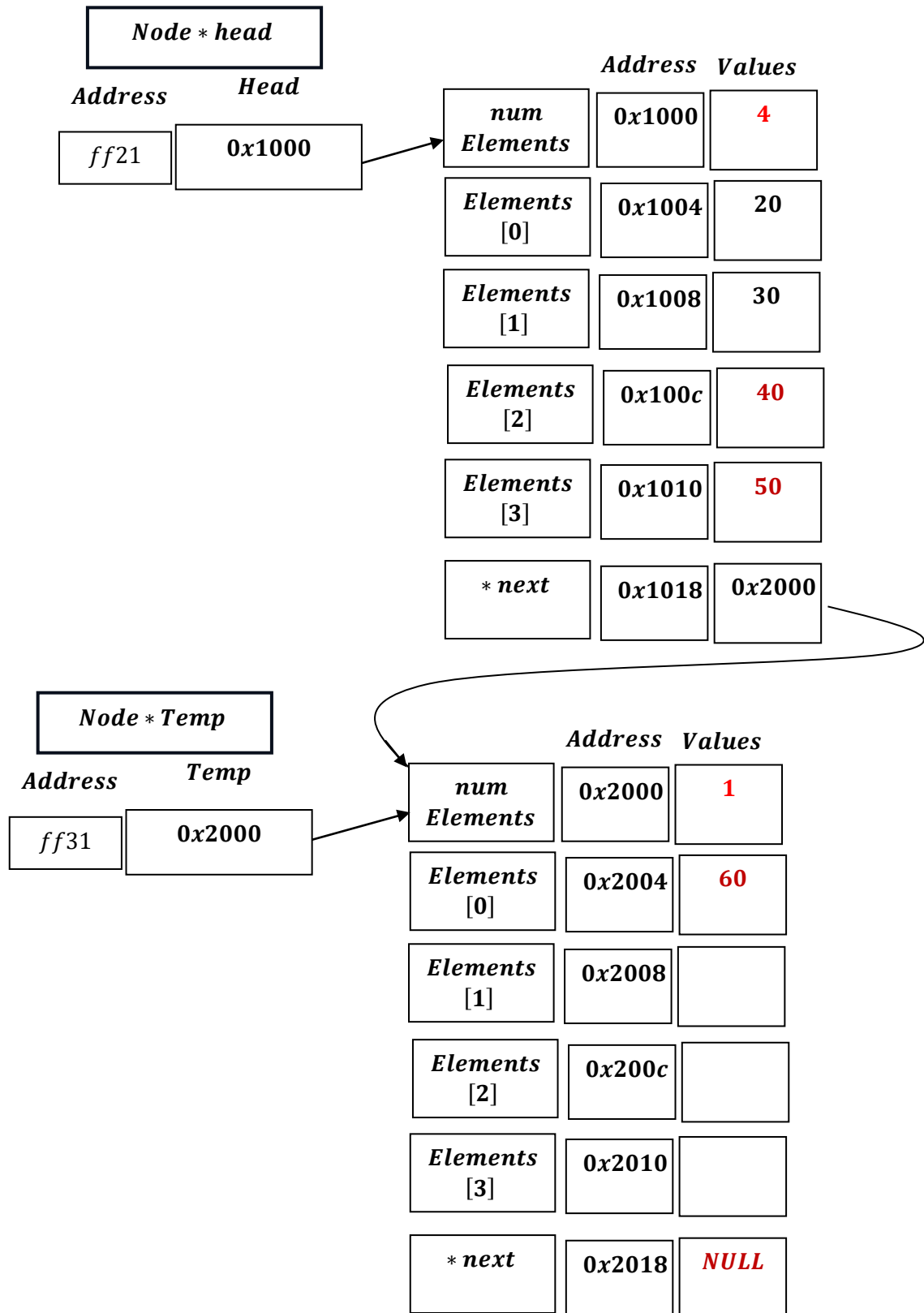
Temp will be again point to *Head*



```
while (temp->next != NULL) {  
    temp = temp->next;  
}
```

While(Temp: 0x1000 → Next : 0x2000 ≠ NULL) $\left[\begin{array}{c} \text{Condition is} \\ \text{True} \end{array} \right] :$
TEMP = TEMP → NEXT : 0x2000

While(Temp: 0x2000 → Next : NULL ≠ NULL) $\left[\begin{array}{c} \text{Condition is} \\ \text{False} \end{array} \right] :$



```

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }

```

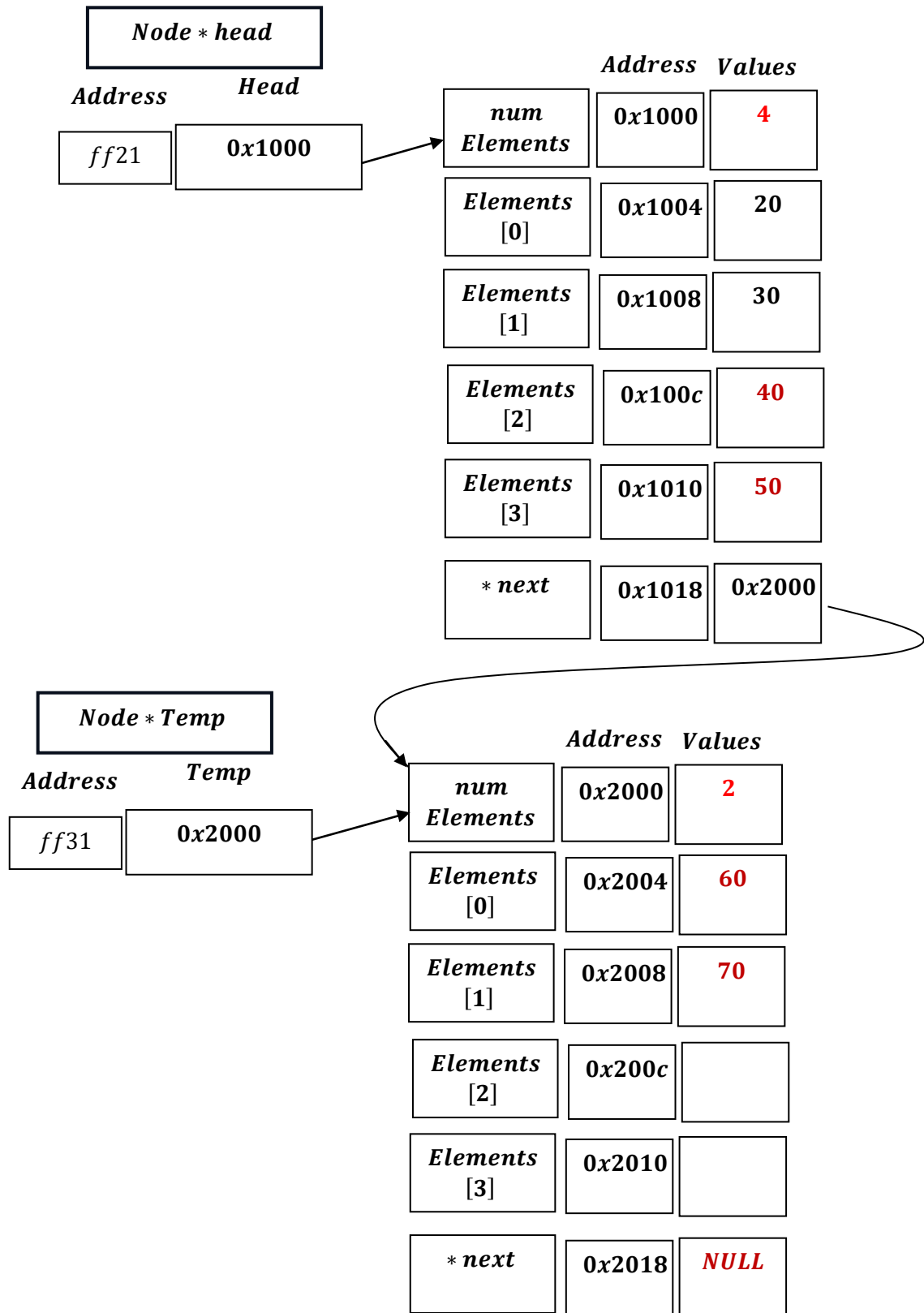
if (*temp*: 0x2000 → *numElements* = 1 < *MAX_ELEMENTS* = 4) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

temp → *elements*[*temp* → *numElements* = 1] = *val* := 70;

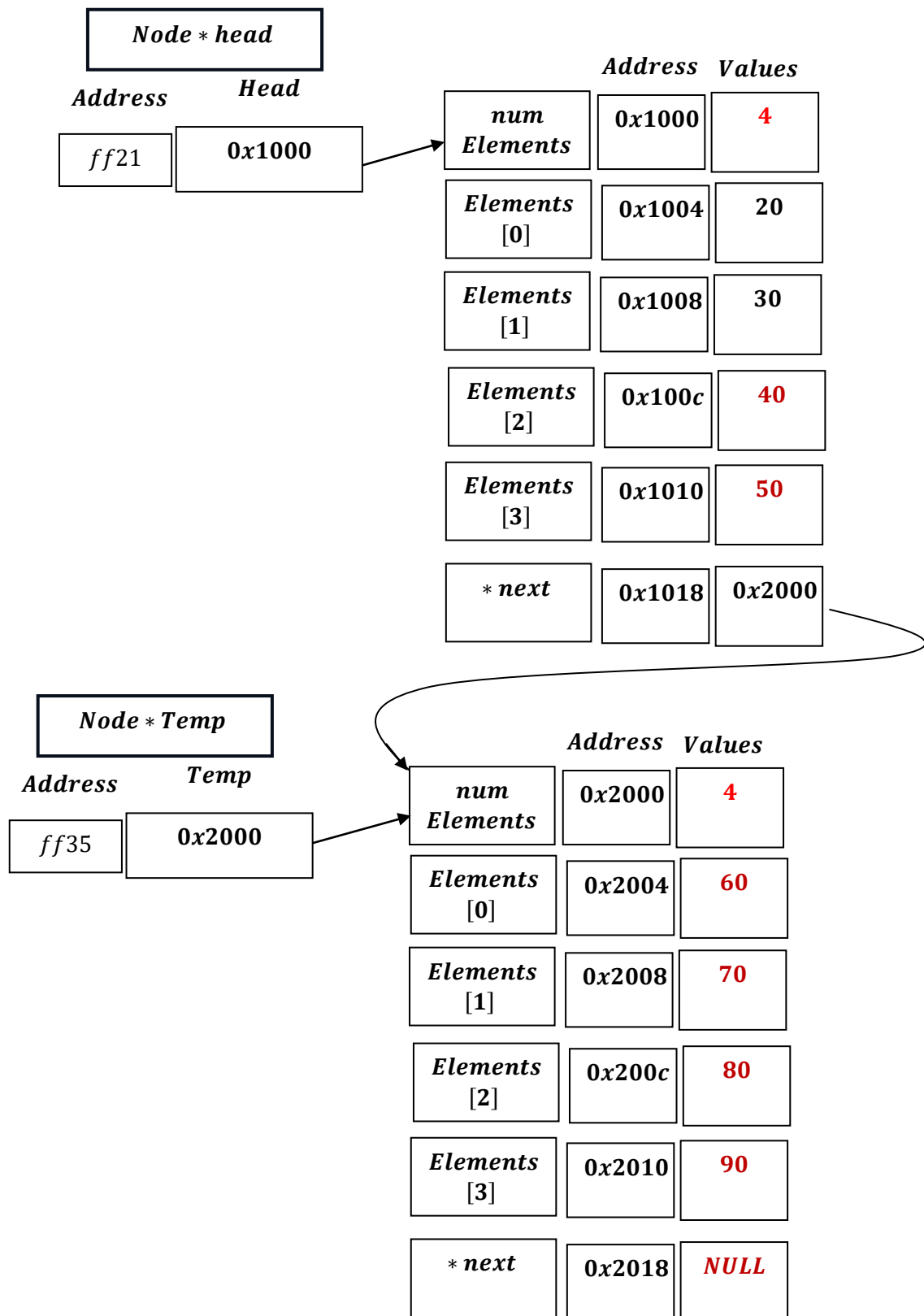
temp → *numElements* = *temp* → *numElements* + 1 = 1 + 1 = 2;

Print: ``Inserted `` *val*: 70 `` into the existing last node.``

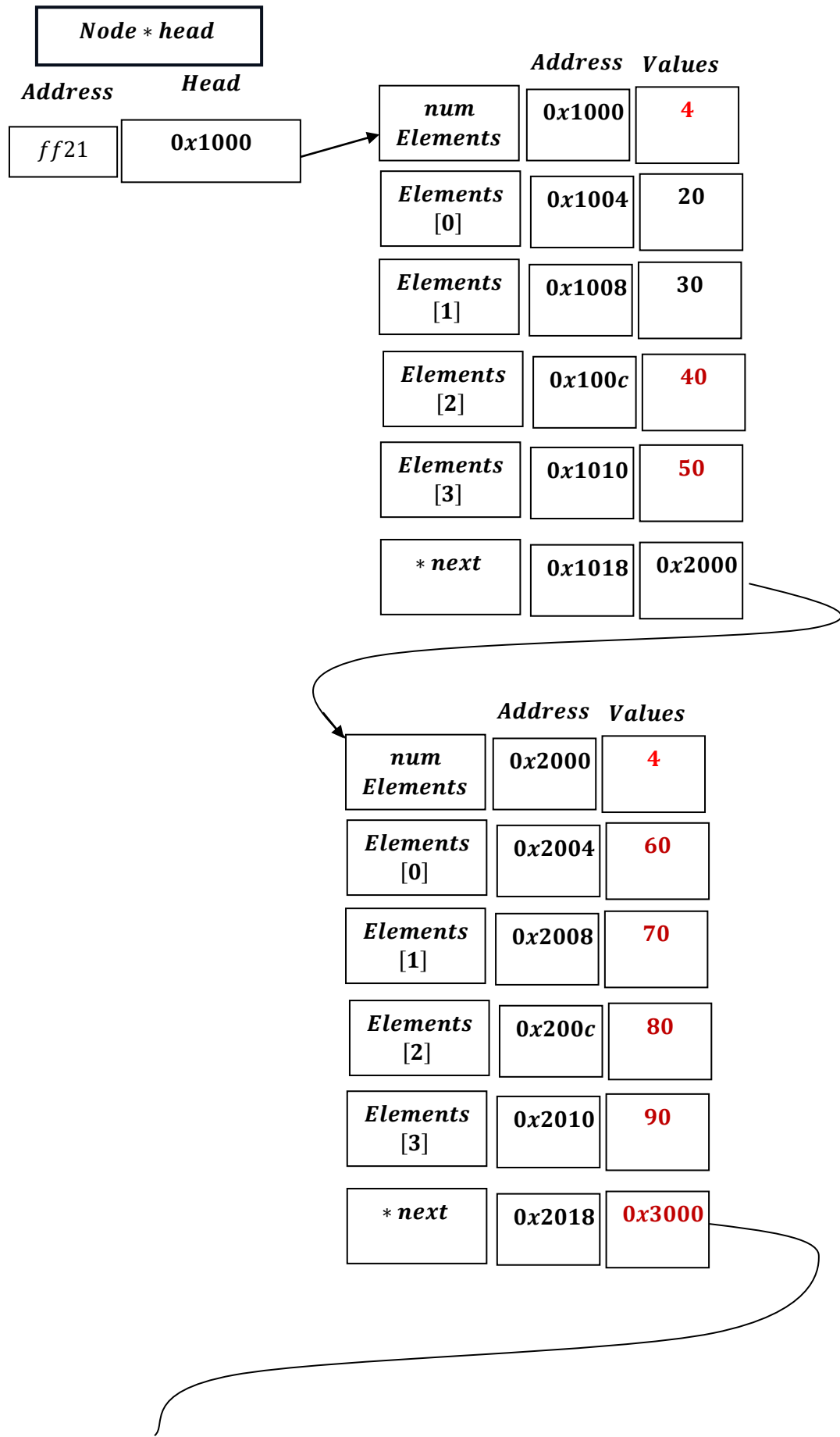
[Exit if structure]

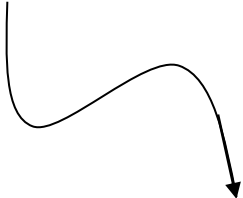


Like this we can get every thing updated as :



As soon as numElements becomes 4 , newNode is created and inserted:





	<i>Address</i>	<i>Values</i>
<i>num</i> <i>Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
* <i>next</i>	0x3018	NULL

1. b. Unrolled List
–Insert Element
–Time Complexity Analysis

Program:

```
#define MAX_ELEMENTS 4

// Structure for the Unrolled Linked List Node
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

// Global head pointer
struct Node* head = NULL;

void insertElement(int val) {
    if (head == NULL) {
        head = (struct Node*)malloc(sizeof(struct Node));
        head->numElements = 1;
        head->elements[0] = val;
        head->next = NULL;
        cout << "Inserted " << val << " into a new head node.\n";
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }
}
```

```

else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
}
...

case 1:
    cout << "Enter integer value to insert: ";
    cin >> value;
    insertElement(value);
    break;

```

Time Complexity Analysis

Best Case Time Complexity: $O(1)$ [**When head = NULL
And, A node is
added to Empty
List**]

- *The best case occurs when the unrolled linked list is completely empty (**head == NULL**).*
- *The program simply allocates memory for a new head node, assigns the value to the first array index, and returns immediately without executing any loops.*

Worst Case Time Complexity: $O(N)$

- *The worst case occurs when the list already contains one or more nodes.*
- *The algorithm uses a while loop (**while** (**temp** $\rightarrow$ **next** $\neq$ **NULL**)) to traverse from the head node all the way to the last node in the sequence.*
- *Because it must visit every node to find the end, the time it takes scales linearly with **N**, where **N** is the total number of nodes in the list.*

Average Case Time Complexity: $O(N)$

The insertElement function does not ask for a position. By its fundamental design, it only appends to the very end of the list.

Because of this hardcoded behavior, there are no probabilities to calculate. The algorithm only has two possible physical states:

1. ***The list is empty:*** Takes $O(1)$ time.
2. ***The list has nodes:*** The program must traverse every single existing node to reach the end.

Hence, there is no real average case, $\therefore$ Worst Case = Average Case = $O(N)$

As every insertion into a populated list forces the program to traverse the entire list to find the end. Therefore, the average performance remains bound by the $O(N)$ traversal time.

1.c. Unrolled List

–Insert Element

–Space Complexity Analysis

Program:

```
#define MAX_ELEMENTS 4

// Structure for the Unrolled Linked List Node
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
};

// Global head pointer
struct Node* head = NULL;

void insertElement(int val) {
    if (head == NULL) {
        head = (struct Node*)malloc(sizeof(struct Node));
        head->numElements = 1;
        head->elements[0] = val;
        head->next = NULL;
        cout << "Inserted " << val << " into a new head node.\n";
        return;
    }
    struct Node* temp = head;
    while (temp->next != NULL) {
        temp = temp->next;
    }

    if (temp->numElements < MAX_ELEMENTS) {
        temp->elements[temp->numElements] = val;
        temp->numElements++;
        cout << "Inserted " << val << " into the existing last
node.\n";
    }
}
```

```

else {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    newNode->numElements = 1;
    newNode->elements[0] = val;
    newNode->next = NULL;

    temp->next = newNode;
    cout << "Node full. Inserted " << val << " into a newly
created node.\n";
}
}
...

    case 1:
        cout << "Enter integer value to insert: ";
        cin >> value;
        insertElement(value);
        break;

```

Space Complexity Analysis

1. Input Space

1. a. Parametric Input Space: $O(1)$

- The *insertElement* function accepts exactly one parameter: an integer variable named *val*.
- The memory required to hold this single integer value is fixed and does not change regardless of the list's current size.
- Therefore, the parametric input space is strictly constant, resulting in a space complexity of $O(1)$.

1. b. Full Data Holistic Input Space: $O(N)$

- *The function operates on a global data structure accessed via the **head** pointer.*
- *The total memory occupied by this entire list scales linearly with the number of existing nodes.*
- *Each node allocates a fixed amount of memory: one integer (**numElements**), an array of 4 integers (**elements[$MAX_ELEMENTS$]**), and one pointer (**next**).*
- *For a list containing **N** nodes, the overall space occupied in memory by the data structure is $O(N)$.*

2. Auxiliary Space: $O(1)$

- *Auxiliary space defines the temporary, extra memory used explicitly to run the algorithm within the function's scope, distinct from the input space.*
- *To perform the traversal, the function declares a single temporary local pointer variable, **struct Node * temp**.*
- *If the final node's array is full ($temp \rightarrow numElements < MAX_ELEMENTS$ evaluates to false), the function allocates a fixed block of memory for exactly one new node using malloc and assigns it to a local pointer **struct Node* newNode**.*

- *Once the function terminates, the local pointer variables (**temp** and **newNode**) are destroyed.*
- *Because the function uses no recursion (no call stack growth) and creates no dynamic structures that scale with the size of the list, the temporary space utilized is strictly constant, yielding an auxiliary space complexity of **$O(1)$** .*

For Academic Answer = Auxiliary Space = $O(1)$.

"As we often cannot control the size of the input data, but we can control how much extra memory of our algorithm creates to process it."

Total Space Complexity or Total memory footprint:

***= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space***

= ($O(1) + O(N)$) + $O(1)$

= $O(N)$.

Algorithm: UnrolledINSElem(val)

*((Node *)malloc(sizeof(Node)) represents the availability of space for the data; let's name it AVAIL.))*

1. [CHECK IF LIST IS EMPTY]

- *If START = NULL, Then:*
 - *SET START := AVAIL (Allocating space for the new head node).*
 - *SET numElements[START] := 1*
 - *SET elements[START][0] := val*
 - *SET LINK[START] := NULL*
 - *Write: "Inserted ", val, " into a new head node."*
 - *RETURN*
- *[End of If structure.]*

2. [INITIALIZE TEMPORARY POINTER]

- *SET TEMPPTR := START*

(Note: The temporary pointer points to the first node to help traverse the loop.)

3. [LOOP TRAVERSAL]

- *REPEAT WHILE LINK[TEMPPTR] ≠ NULL:*
 - *SET TEMPPTR := LINK[TEMPPTR]*
- *[End of Loop]*

4. [CHECK NODE CAPACITY AND INSERT]

- *If numElements[TEMPPTR] < MAX_ELEMENTS, Then:*
 - *SET elements[TEMPPTR][numElements[TEMPPTR]] := val*
 - *SET numElements[TEMPPTR] := numElements[TEMPPTR] + 1*
 - *Write: "Inserted ", val, " into the existing last node."*
- *ELSE:*
 - *SET NEWPTR := AVAIL (Allocate memory for a newly created node).*
 - *SET numElements[NEWPTR] := 1*
 - *SET elements[NEWPTR][0] := val*
 - *SET LINK[NEWPTR] := NULL*

- $SET\ LINK[TEMPPTR] :=$
 $NEWPTR \left(\begin{array}{c} \text{Linking the new node with} \\ \text{the previous node after which} \\ \text{it is to be placed} \end{array} \right).$

- *Write: "Node full.Inserted ",val," into a newly created node."*

- *[End of If structure.]*

5. *END OF ALGORITHM*

2. Unrolled List

–Delete Element

–Program Analysis

Program:

```
void deleteElement(int val) {
    if (head == NULL) {
        cout << "The list is empty. Nothing to delete.\n";
        return;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {

                // Element found: Shift remaining elements left
                for (int j = i; j < temp->numElements - 1; j++) {
                    temp->elements[j] = temp->elements[j + 1];
                }
                temp->numElements--;

                cout << "Element " << val << " deleted
successfully.\n";

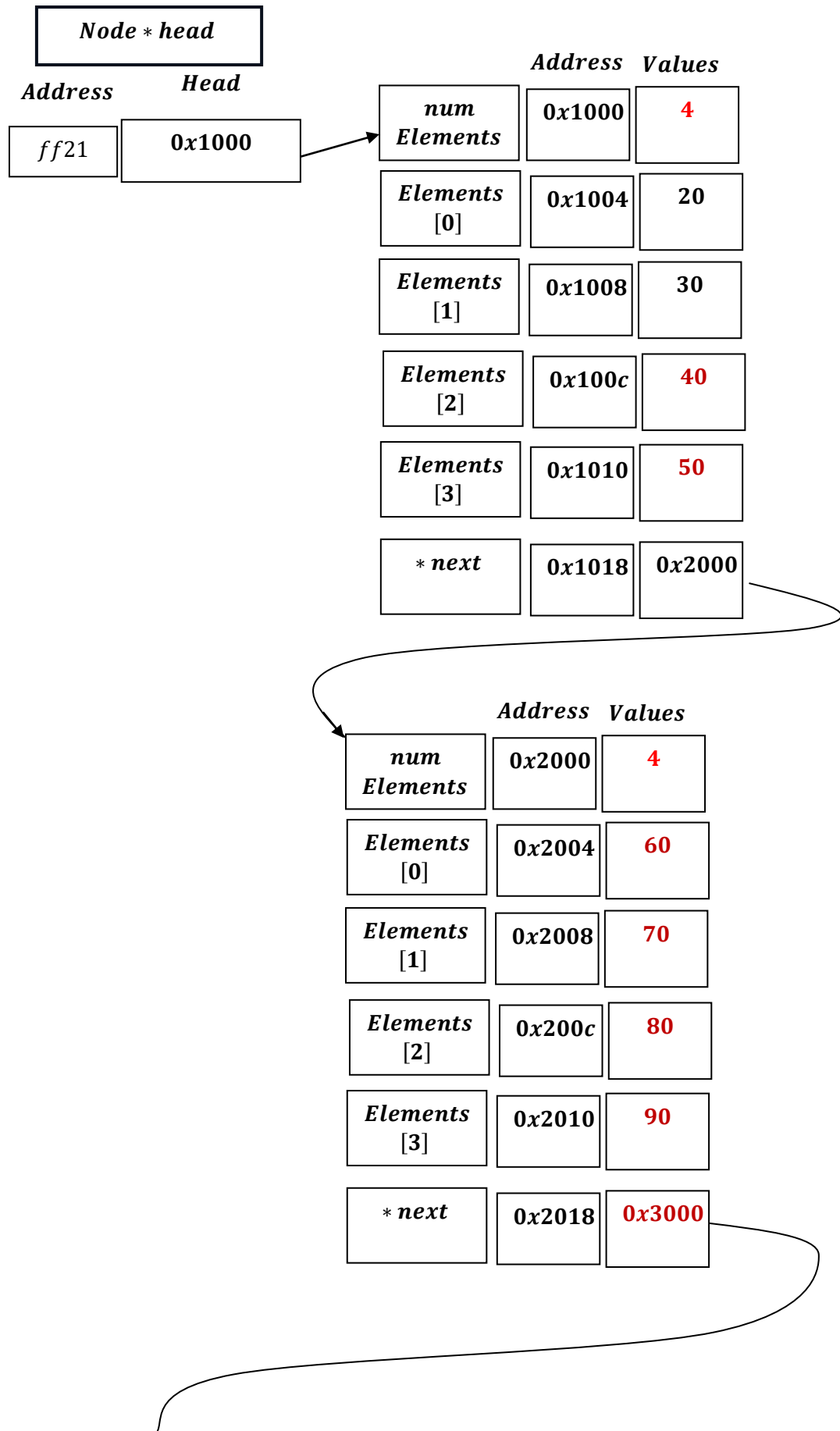
                // Check if the node is now completely empty
                if (temp->numElements == 0) {
                    if (prev == NULL) {
                        head = temp->next;
                        free(temp);
                    } else {
                        prev->next = temp->next;
                        free(temp);
                    }
                    cout << "An empty node was removed from the
list.\n";
                }
            }
        }
    }
}
```

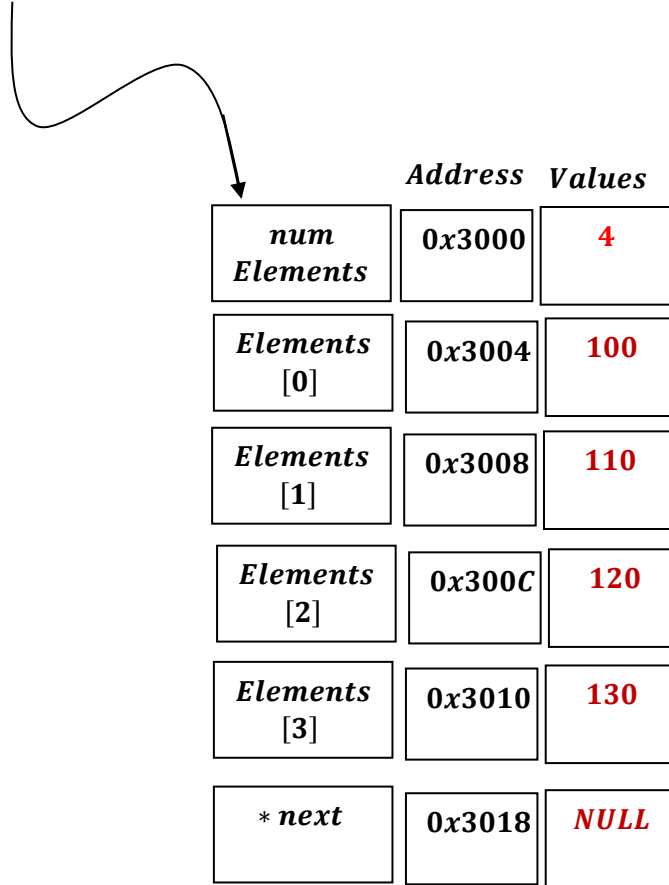
```
        return;
    }
}
prev = temp;
temp = temp->next;
}

cout << "Element " << val << " not found in the list.\n";
}
....

    case 2:
        cout << "Enter integer value to delete: ";
        cin >> value;
        deleteElement(value);
        break;
```

Program Analysis





	Address	Values
num Elements	0x3000	4
Elements [0]	0x3004	100
Elements [1]	0x3008	110
Elements [2]	0x300C	120
Elements [3]	0x3010	130
* next	0x3018	NULL

Function call:

```
case 2:
    cout << "Enter integer value to delete: ";
    cin >> value;
    deleteElement(value);
    break;
```

Say, value is : 110

```
void deleteElement(int val)
```

deleteElement(val: 110)

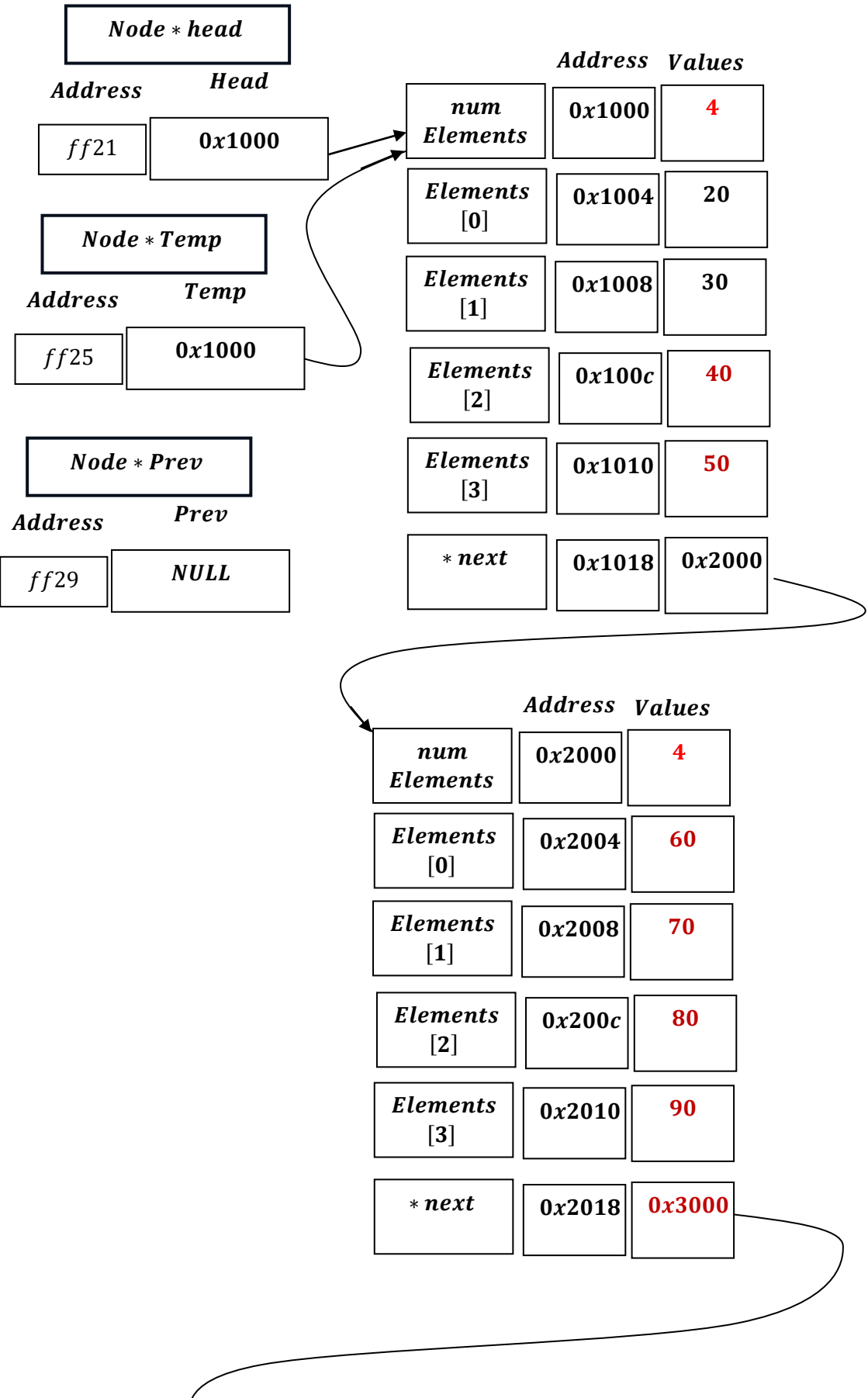
```
if (head == NULL) {
    cout << "The list is empty. Nothing to delete.\n";
    return;
}
```

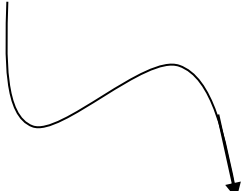
if (head = 0x1000 ≠ NULL) [Hence condition is False.]

```
struct Node* temp = head;
```

And,

```
struct Node* prev = NULL;
```





	Address	Values
<i>num Elements</i>	0x3000	4
<i>Elements [0]</i>	0x3004	100
<i>Elements [1]</i>	0x3008	110
<i>Elements [2]</i>	0x300C	120
<i>Elements [3]</i>	0x3010	130
<i>* next</i>	0x3018	NULL

```

while (temp != NULL) {
    for (int i = 0; i < temp->numElements; i++) {
        if (temp->elements[i] == val) {

            // Element found: Shift remaining elements left
            for (int j = i; j < temp->numElements - 1; j++) {
                temp->elements[j] = temp->elements[j + 1];
            }
            temp->numElements--;

            cout << "Element " << val << " deleted
successfully.\n";

            // Check if the node is now completely empty
            if (temp->numElements == 0) {
                if (prev == NULL) {
                    head = temp->next;
                    free(temp);
                } else {
                    prev->next = temp->next;
                    free(temp);
                }
                cout << "An empty node was removed from the
list.\n";

```

```

    }
    return;
}
}
prev = temp;
temp = temp->next;
}

```

$while(Temp = 0x1000 \neq NULL) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$for\ i = 0; i = 0 < temp \rightarrow numElements = 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$if(temp \rightarrow Elements[0]: 20 \neq 110) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 0 + 1 = 1.$

$for\ i = 1; i = 1 < temp \rightarrow numElements = 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$if(temp \rightarrow Elements[1]: 30 \neq 110) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 1 + 1 = 2.$

for $i = 2; i = 2 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[2]: 40 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 2 + 1 = 3.$

for $i = 3; i = 3 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[3]: 50 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

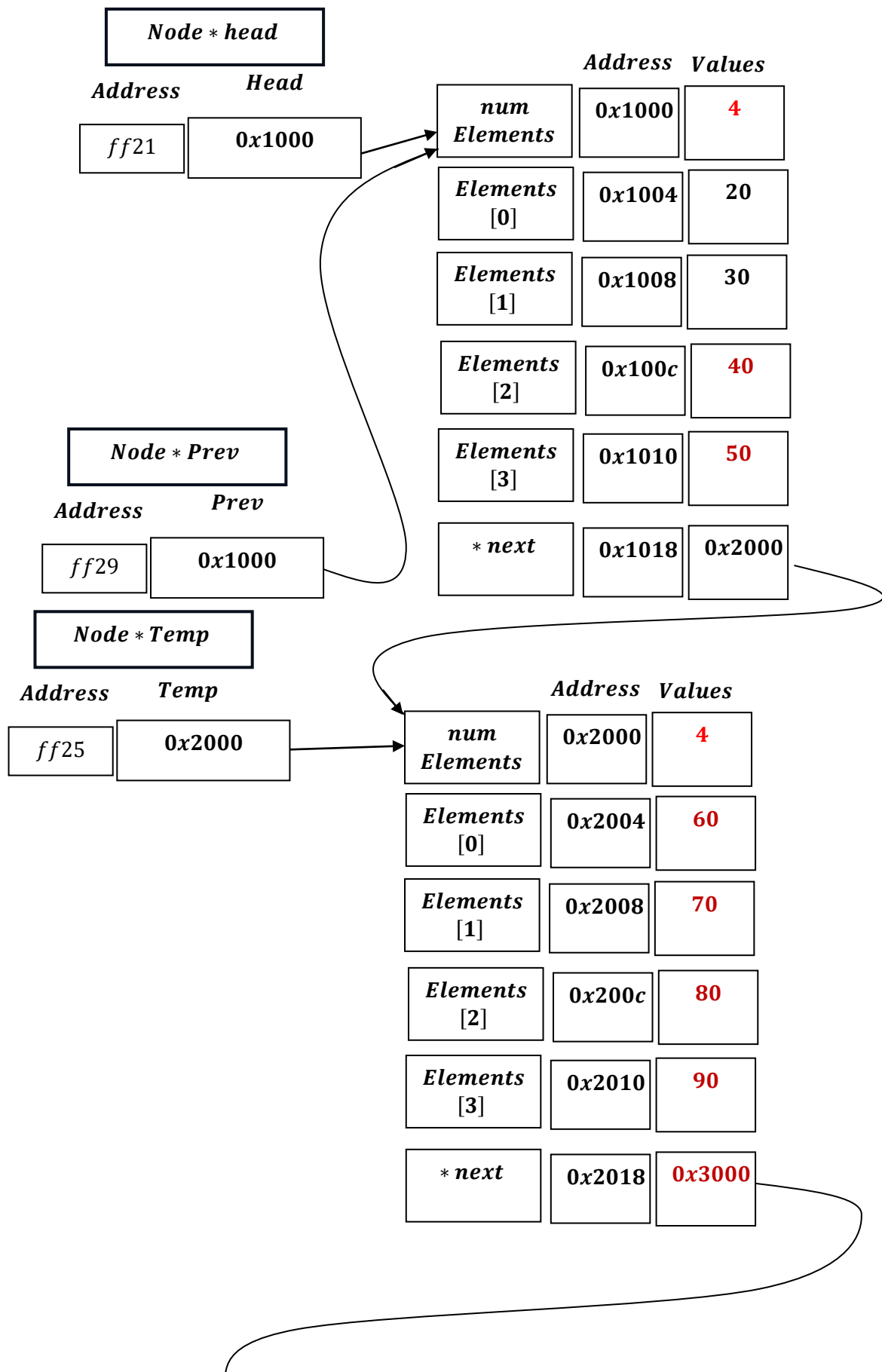
$i = i + 1 = 3 + 1 = 4.$

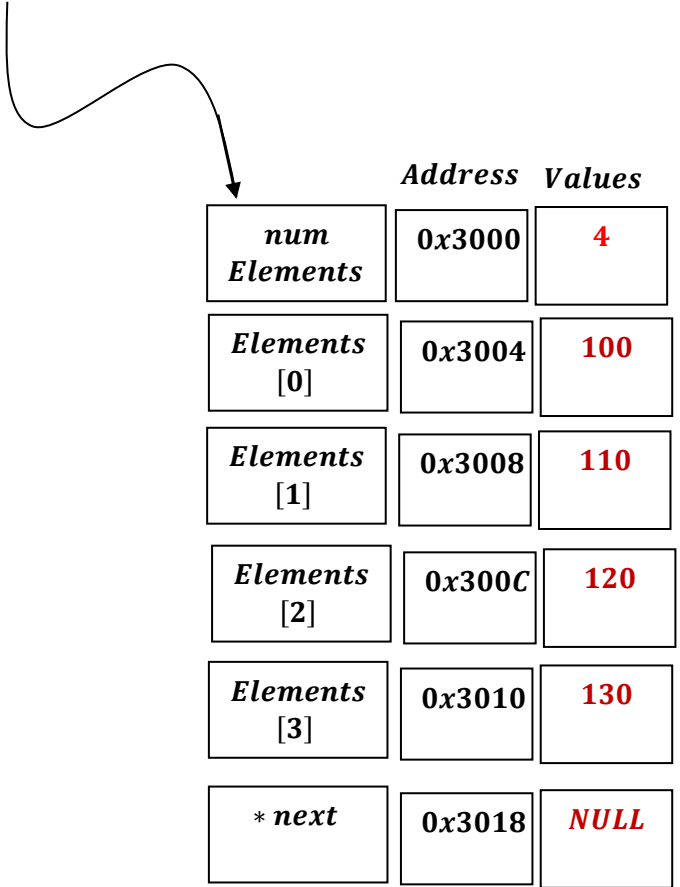
for $i = 4; i = 4 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

```
prev = temp;  
temp = temp->next;
```

prev = *temp* = 0x1000.

temp = *temp* $\rightarrow$ *next* = 0x2000.





	Address	Values
<i>num</i> <i>Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
* <i>next</i>	0x3018	NULL

$while(Temp = 0x2000 \neq NULL) \left[\begin{array}{c} \text{Condition} \\ is \\ True \end{array} \right]$

$for\ i = 0; i = 0 < temp \rightarrow numElements = 4 \left[\begin{array}{c} \text{Condition} \\ is \\ True \end{array} \right]$

$if(temp \rightarrow Elements[0]: 60 \neq 110) \left[\begin{array}{c} \text{Condition} \\ is \\ False \end{array} \right]$

$i = i + 1 = 0 + 1 = 1.$

for $i = 1; i = 1 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[1]: 70 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 1 + 1 = 2.$

for $i = 2; i = 2 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[2]: 80 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 2 + 1 = 3.$

for $i = 3; i = 3 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow Elements[3]: 90 \neq 110$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

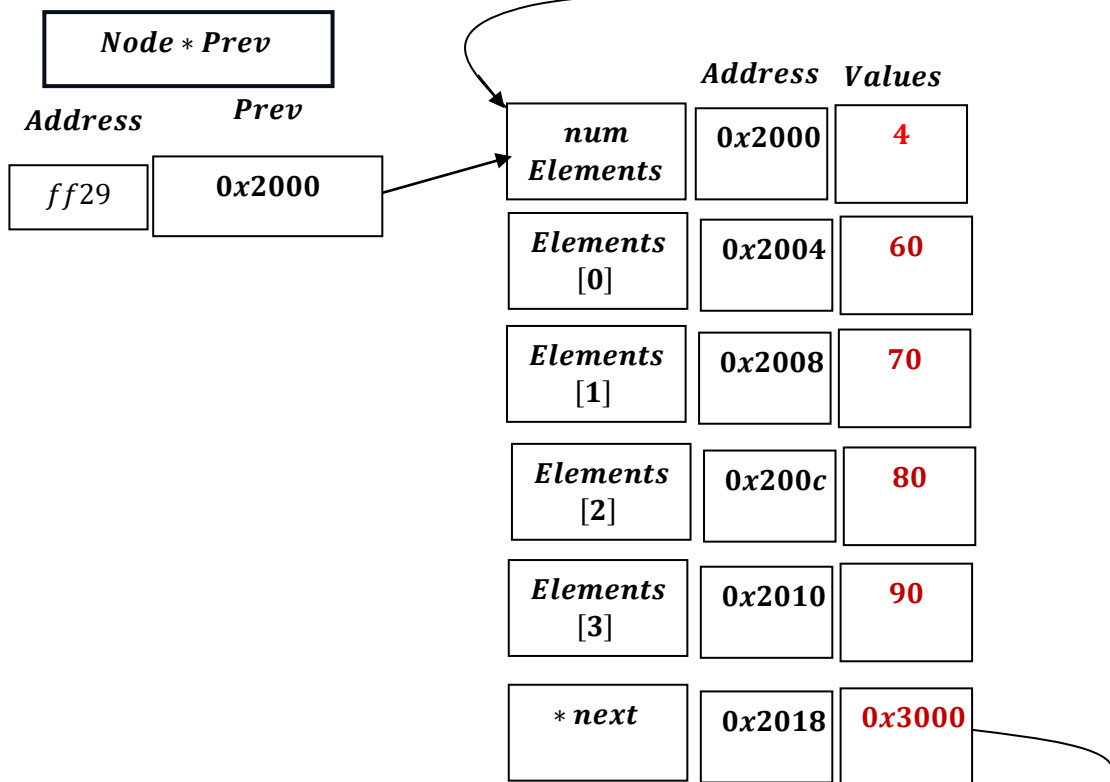
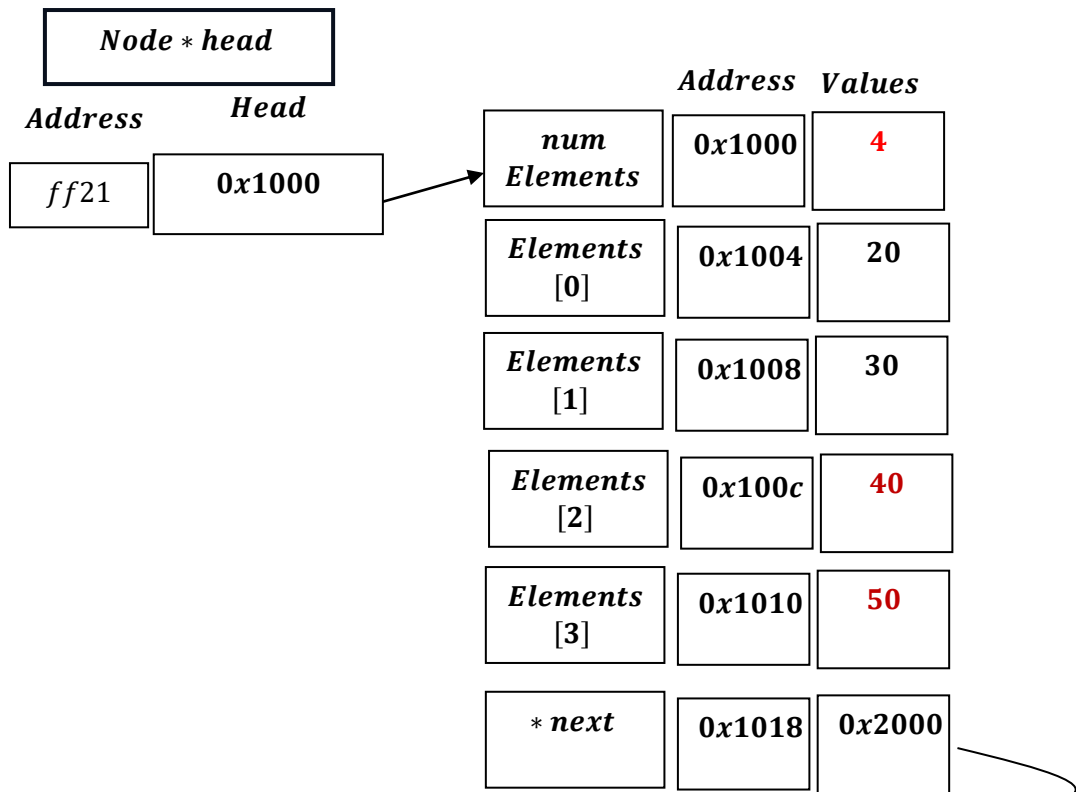
$i = i + 1 = 3 + 1 = 4.$

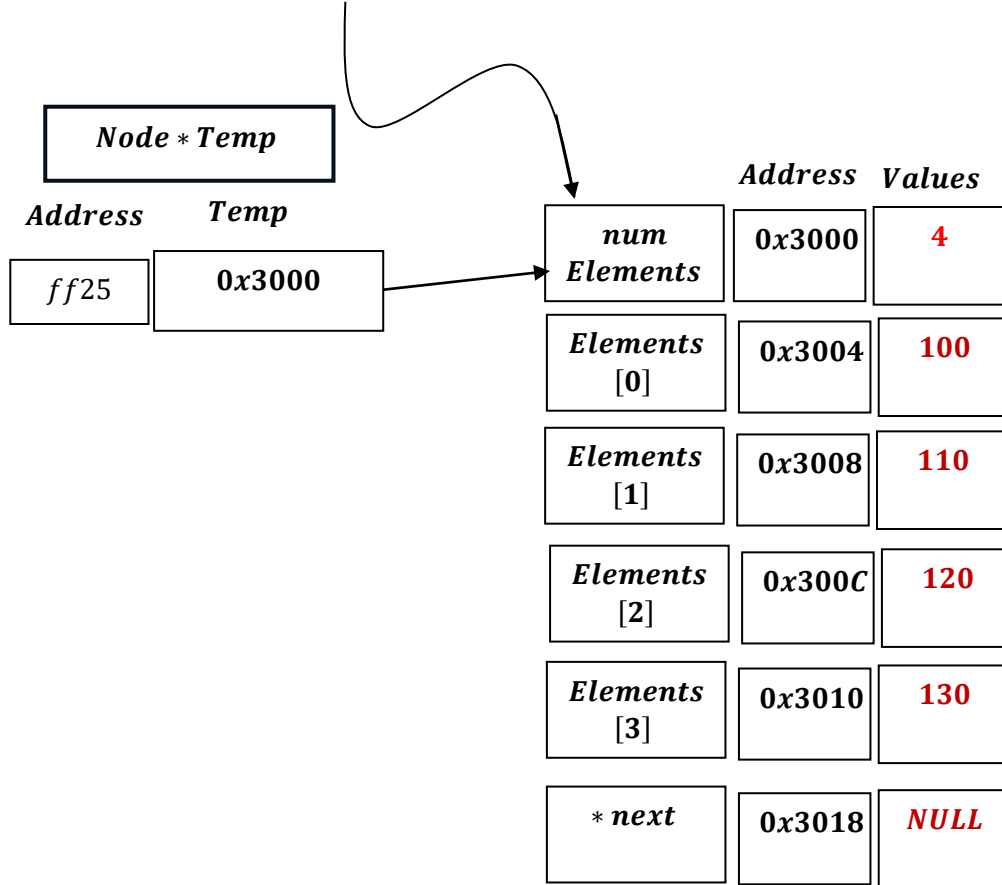
for $i = 4; i = 4 < temp \rightarrow numElements = 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

```
prev = temp;  
temp = temp->next;
```

prev = *temp* = 0x2000.

temp = *temp* $\rightarrow$ *next* = 0x3000.





$while(Temp = 0x3000 \neq NULL) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$for\ i = 0; i = 0 < temp \rightarrow numElements = 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$if(temp \rightarrow Elements[0]: 100 \neq 110) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 0 + 1 = 1.$

$for\ i = 1; i = 1 < temp \rightarrow numElements = 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$if(temp \rightarrow Elements[1]: 110 = 110) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

Shifting of Elements

```

for (int j = i; j < temp->numElements - 1; j++) {
    temp->elements[j] = temp->elements[j + 1];
}
temp->numElements--;

cout << "Element " << val << " deleted
successfully.\n";

```

$for : int j = i = 1 ; j < temp \rightarrow numElements - 1 [j < (4 - 1 = 3)] \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$temp \rightarrow element[j = 1] = temp \rightarrow element[j + 1 = 1 + 1 = 2] = 120$

$j = j + 1 = 1 + 1 = 2$

$for : int j = 2 ; j < temp \rightarrow numElements - 1 [j < (4 - 1 = 3)] \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

$temp \rightarrow element[j = 2] = temp \rightarrow element[j + 1 = 2 + 1 = 3] = 130$

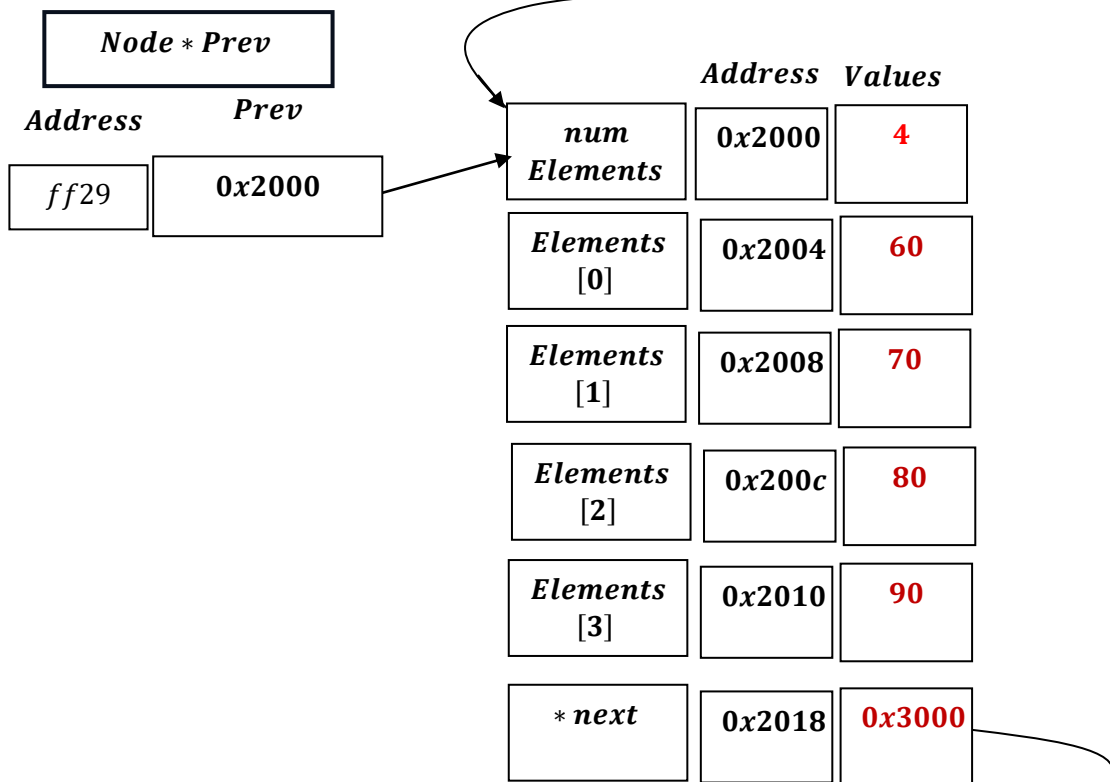
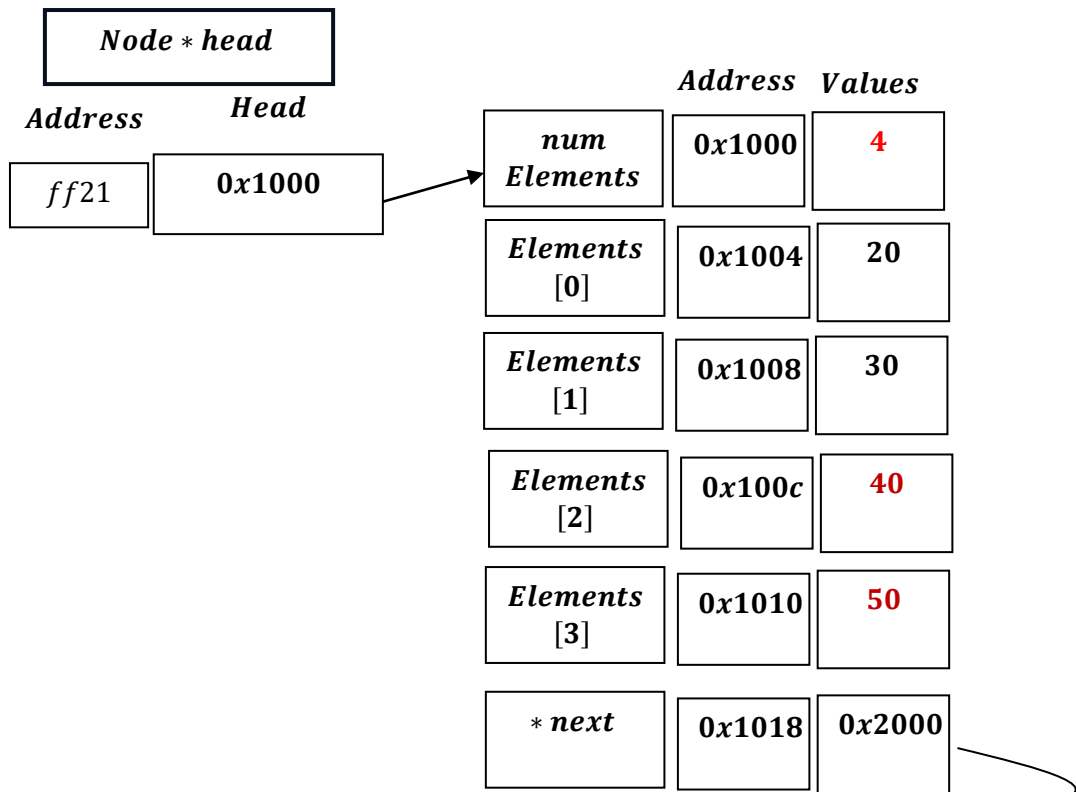
$j = j + 1 = 2 + 1 = 3$

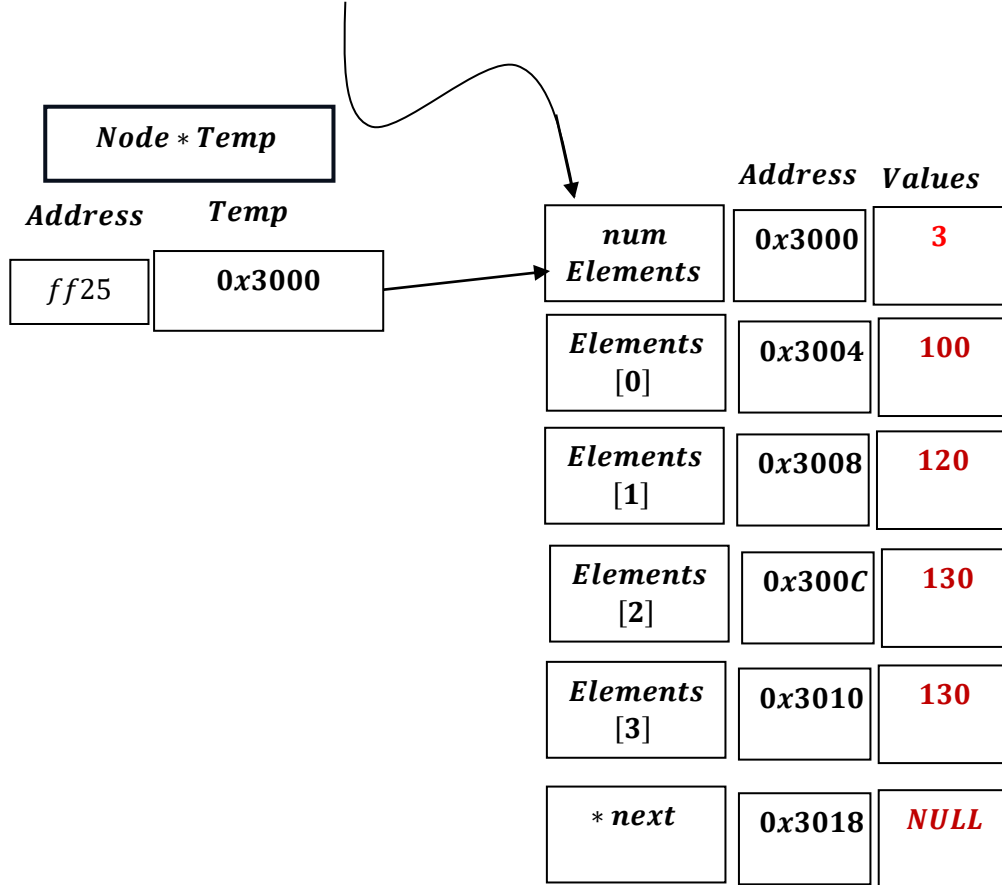
$for : int j = 3 ; j < temp \rightarrow numElements - 1 [j < (4 - 1 = 3)] \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$\left[\begin{array}{c} \text{Hence} \\ \text{loop} \\ \text{Exits} \end{array} \right]$

$temp \rightarrow numElement = temp \rightarrow numElements - 1 = 4 - 1 = 3.$

Print: `` Element `` val: 110 ``deleted successfully.``





return;

Return , will return **void** Return type
void delete
function. and exit from the loop.

Note, Element[3] → from Address: 0x3000, contains 130 ,
but UpperBound i. e. numElements has become **3** , therefore
next time temp , when it will be pointing at : 0x3000, for
loop will run:

for $i = 0; i = 0 < temp \rightarrow numElements = 3$ Condition
is
True

i. e. **loop condition will be true when :**

$i = 0, i = 1, i = 2$. and will not reach : Elements[3].

Now, when : temp → numElements = 0?

A. List has a single Node ,B. When A node's numElements become 0.

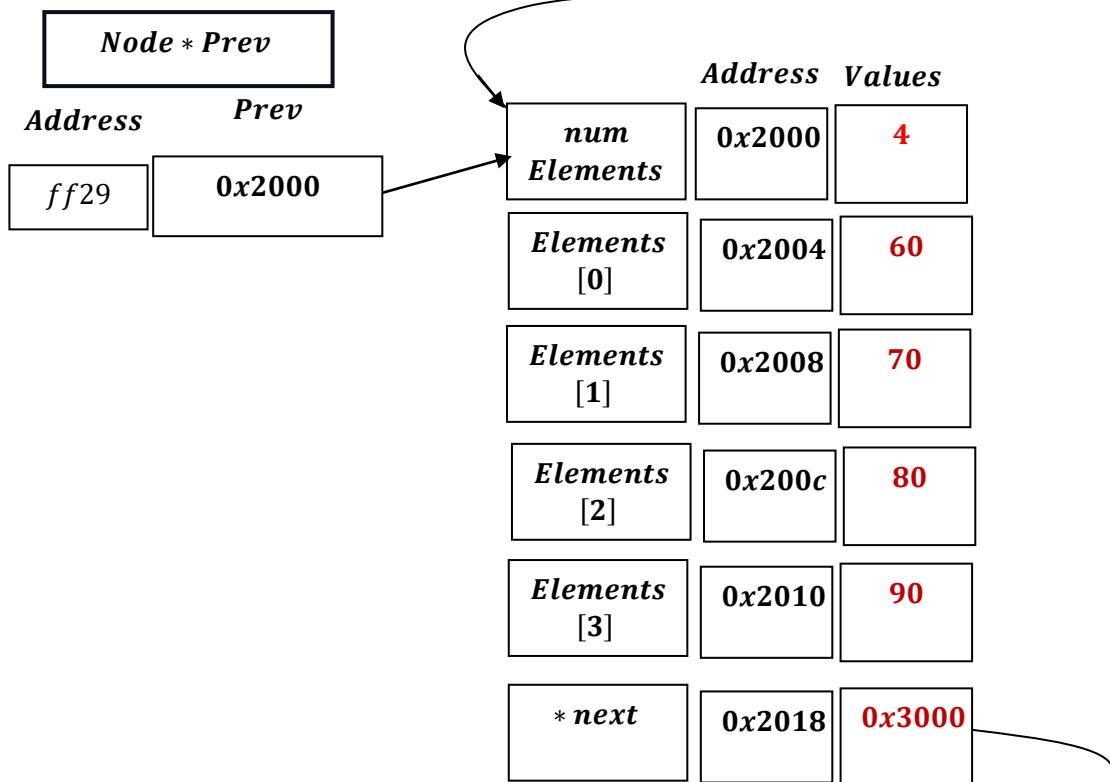
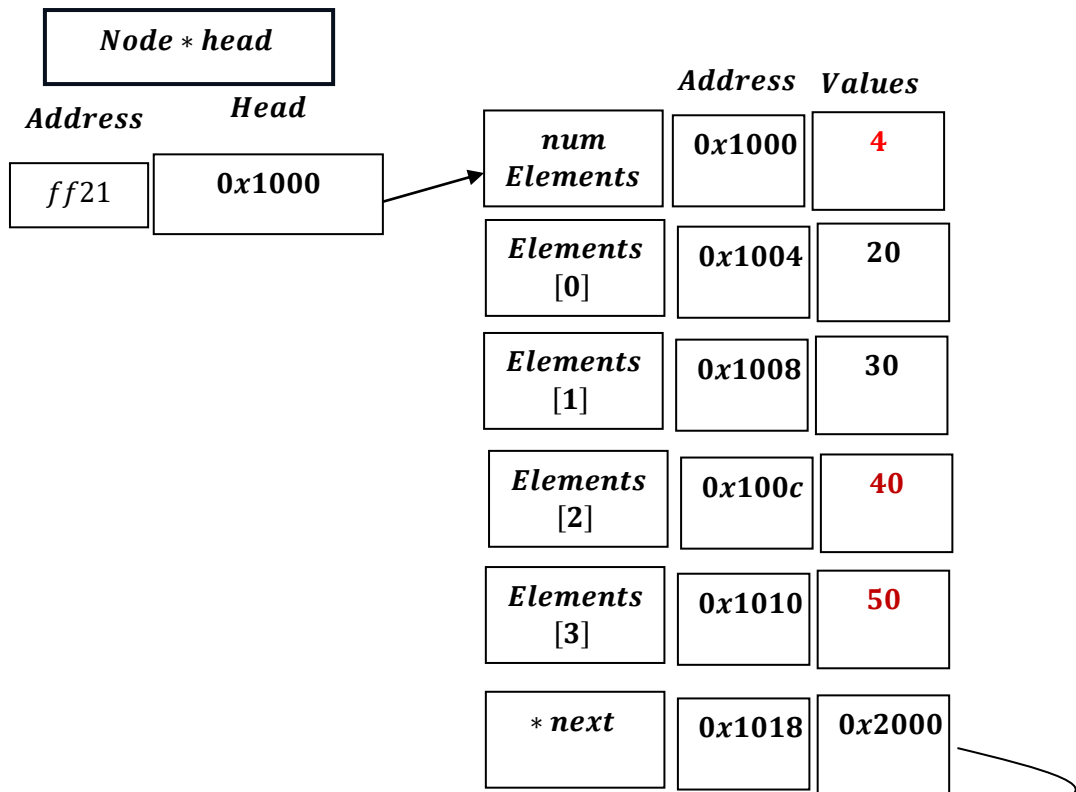
```
// Check if the node is now completely empty
if (temp->numElements == 0) {
    if (prev == NULL) {
        head = temp->next;
        free(temp);
    } else {
        prev->next = temp->next;
        free(temp);
    }
}
cout << "An empty node was removed from the list.\n";
}
```

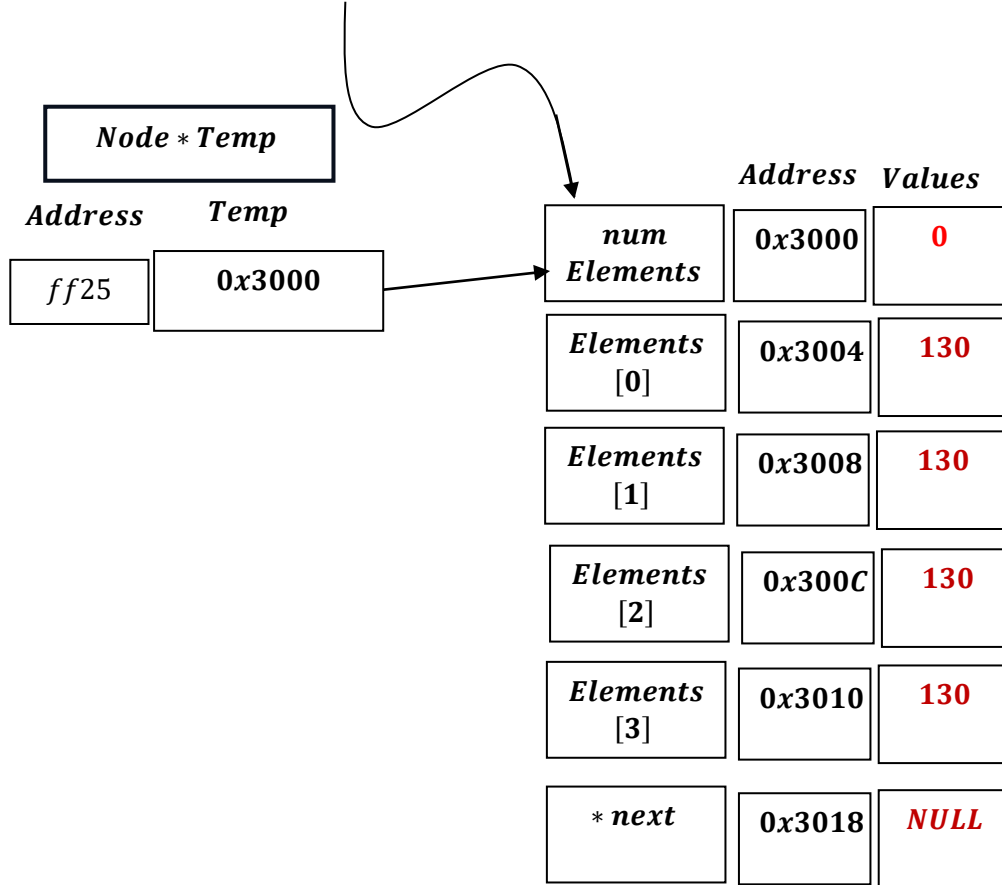
B. When a node's numElements become 0

i. e. no elements left in that node to get

Deleted.

Then : Prev ≠ NULL is True.





```

if (temp->numElements == 0) {
    if (prev == NULL) {
        head = temp->next;
        free(temp);
    } else {
        prev->next = temp->next;
        free(temp);
    }
}
cout << "An empty node was removed from the list.\n";
}

```

$$if (temp \rightarrow numElements = 0 = 0) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$$

$$if(prev: 0x2000 = NULL) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$$

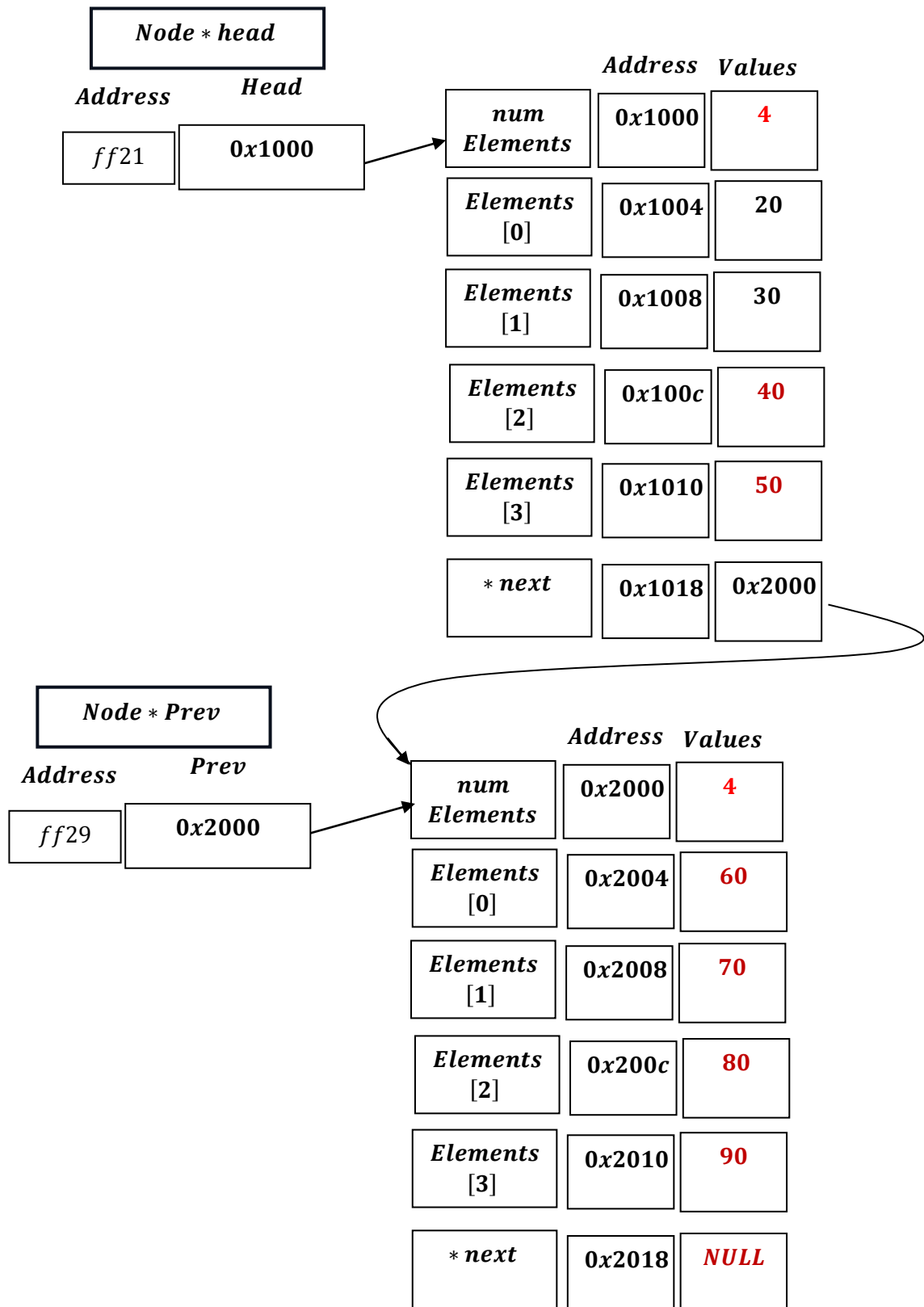
Hence , Else part:

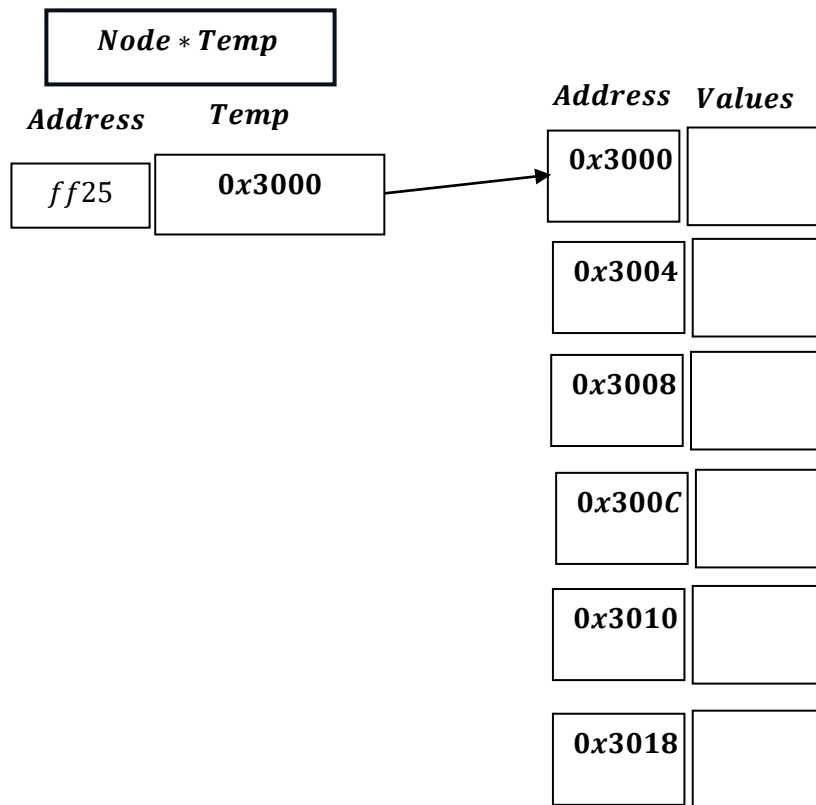
```
else {  
    prev->next = temp->next;  
    free(temp);  
}  
cout << "An empty node was removed from the list.\n";
```

prev: 0x2000 → next = temp: 0x3000 → next = NULL.

free(temp).

*Next it will return **void** Return type
void delete
function. and exit from the loop.*



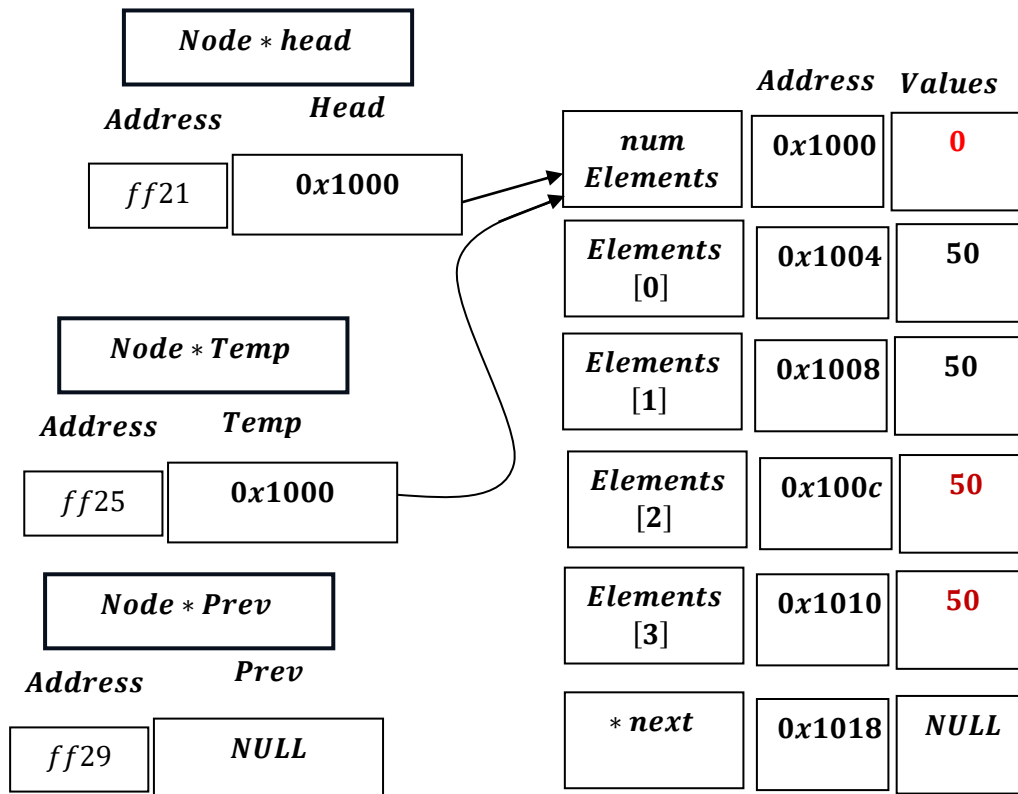


A. List has a single Node

and

temp → numElements = 0 i. e. No elements left in that single node to get deleted.

Then: Prev = NULL is True.



```

if (temp->numElements == 0) {
    if (prev == NULL) {
        head = temp->next;
        free(temp);
    } else {...}
cout << "An empty node was removed from the list.\n";
}

```

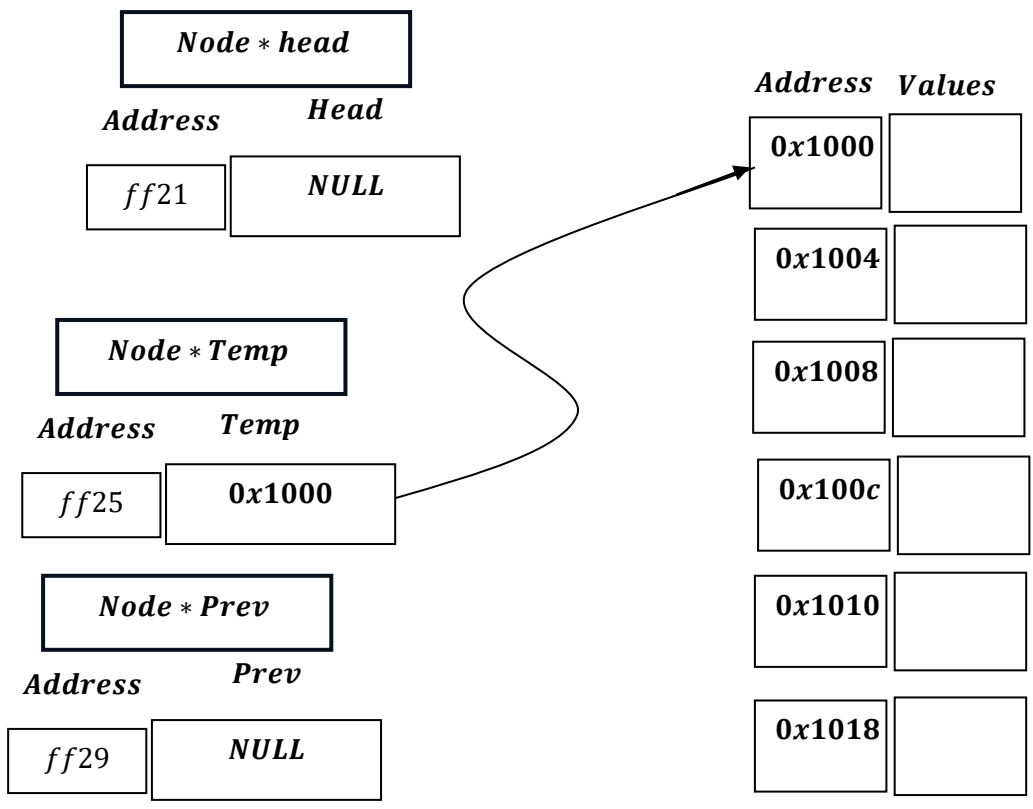
Temp → *numElements* = 0 [True]

Prev = NULL [True]

head = *temp* → *next* = NULL

free(temp);

Next it will return *void* [Return type
void delete
function.] and exit from the loop.



2. b. Unrolled List

–Delete Element

–Time Complexity Analysis

Program:

```
void deleteElement(int val) {
    if (head == NULL) {
        cout << "The list is empty. Nothing to delete.\n";
        return;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {

                // Element found: Shift remaining elements left
                for (int j = i; j < temp->numElements - 1; j++) {
                    temp->elements[j] = temp->elements[j + 1];
                }
                temp->numElements--;

                cout << "Element " << val << " deleted
successfully.\n";

                // Check if the node is now completely empty
                if (temp->numElements == 0) {
                    if (prev == NULL) {
                        head = temp->next;
                        free(temp);
                    } else {
                        prev->next = temp->next;
                        free(temp);
                    }
                    cout << "An empty node was removed from the
list.\n";
                }
            }
        }
    }
}
```

```
        return;
    }
}
prev = temp;
temp = temp->next;
}

cout << "Element " << val << " not found in the list.\n";
}
....
    case 2:
        cout << "Enter integer value to delete: ";
        cin >> value;
        deleteElement(value);
        break;
```

Time Complexity Analysis

Variable Definitions

- *Let n = total number of elements stored across the entire unrolled linked list.*
- *Let N = total number of nodes in the list.*
- *Let B = capacity of each node ($MAX_{ELEMENTS} = 4$).*

Note: Because each node holds a fixed constant capacity of elements

($B = 4$), the number of nodes $N \approx \left\lceil \frac{n}{4} \right\rceil$, meaning $O(N) = O(n)$.

Say, if we have 13 elements in the list, then $n = 13$ and these total of n

elements grouped into 4 blocks i.e. $\frac{n}{4} = \frac{13}{4} = 3.25$. As we cannot allocate

fraction of a node, hence we will use ceiling function $\left\lceil \frac{n}{4} \right\rceil = \left\lceil \frac{13}{4} \right\rceil$

= $\lceil 3.25 \rceil = 4$. That is number of Nodes ' N ' = 4 to store these $n = 13$

elements.

i.e. $N = \left\lceil \frac{n}{B} \right\rceil$, where B is $MAX_{ELEMENTS} = 4$.

1. *Best Case Time Complexity: $O(1)$*

The best – case scenario occurs when the function performs the minimum possible number of operations and terminates immediately:

1. **[Absolute Best Case]** *Empty List: If the list is completely empty (`head == NULL`), the function checks the condition, prints an error message, and returns immediately.*

2. **[Not Absolute Best Case]** *Element at the Very Beginning:*
If the target element is located at index 0 of the first node (`head → elements[0] == val`):
 - *The outer **while** loop runs only 1 time.*
 - *The search **for** loop finds the match on the first iteration (**$i = 0$**).*
 - *The shifting **for** loop shifts at most **$B - 1 = 4 - 1 = 3$** elements to the left, which takes constant time ($O(1)$).*
 - *Node deallocation (if the node becomes empty) also takes constant time ($O(1)$).*

2. Worst Case Time Complexity: $O(n)$ (or $O(N)$)

The worst – case scenario occurs when the algorithm is forced to perform the maximum possible number of comparisons and pointer traversals:

1. **[Absolute Worst Case]Element Not Found:** *The element does not exist in the list. The function must inspect every element across every node before reaching `temp == NULL` and printing Element not found.*

2. **[Not Absolute Worst Case]Element at the Very End:**
The element is located in the last position of the last node in the list.

[Absolute Worst Case] Element Not Found:

- *Let n = total number of elements stored across the entire unrolled linked list.*
- *Let N = total number of nodes in the list.*
- *Let B = capacity of each node ($MAX_{ELEMENTS} = 4$).*

```
while (temp != NULL) {
```

Nodes Traversal: The outer loop visits every single node. $Cost = c_n \times N$.

- c_n = The cost of hopping from one node to the next.

```
for (int i = 0; i < temp->numElements; i++) {
    if (temp->elements[i] == val) {
```

Elements Compared: The inner loop runs for every element accross all nodes.

$Costs = c_s \times n$.

- c_s = The cost of checking one element (the if comparison.)

```
for (int j = i; j < temp->numElements - 1; j++) {
    temp->elements[j] = temp->elements[j + 1];
}
```

Here we see `` $j < temp \rightarrow numElements - 1$ `` i.e. $j < B - 1$, but because the element is never found, the shifting loop never executes. $Cost = c_{sh} \times 0 = 0$.

- c_{sh} = The cost of shifting one element in the array.

```
temp->numElements--;

cout << "Element " << val << " deleted
successfully.\n";

// Check if the node is now completely empty
if (temp->numElements == 0) {
    if (prev == NULL) {
        head = temp->next;
        free(temp);
    } else {
        prev->next = temp->next;
        free(temp);
    }
}
```

```

        cout << "An empty node was removed from the
list.\n";
    }

```

Deletions: Nothing is deleted. Hence decrement occurs. Cost = $c_d \times 0 = 0$.

- c_d = The constant cost of bookkeeping (decrementing numElements, freeing a node)

Why is this considered a constant cost(c_d)?

No matter how many total elements (n) or nodes (N) exist in our list,

this specific block of code always takes the exact same number of steps to run:

- It does 1 subtraction ($temp \rightarrow numElements --$).
- It evaluates at most 2 if conditions.
- It does 1 pointer assignment (either $head = \dots$ or $prev \rightarrow next = \dots$).
- It performs 1 memory deallocation ($free(temp)$).

Mathematical Equation: $T_1 = (c_n \times N) + (c_s \times n) + 0 + 0$

As, $N \leq n$, the dominant term is n .

1. In terms of elements : $O(n)$.

Now,

```

while (temp != NULL) {
    for (int i = 0; i < temp->numElements; i++) {
        if (temp->elements[i] == val) {

```

That is we traverse each node blocks : N

And each node block elements traversed from 0 to $B - 1$ i.e. $\sum_{i=0}^{B-1} 1$

And number of comparisons also: $\sum_{i=0}^{B-1} 1$ times.

$$\sum_{i=0}^{B-1} 1 = 1 + 1 + \dots = (\text{Top Bound} - \text{Bottom Bound}) + 1 = (B - 1 - 0) + 1 = B.$$

That is $N \times B$ comparisons .

Relationship to n : *The variable n represents the exact number of active elements currently stored across the entire data structure. Because N nodes can each hold a maximum of B elements, the absolute physical capacity of the unrolled linked list is $N \times B$ slots. This gives us the mathematical guarantee that $n \leq N \times B$.*

```
while (temp != NULL) {
    for (int i = 0; i < temp->numElements; i++) {
        if (temp->elements[i] == val) {

            // Element found: Shift remaining elements left
            for (int j = i; j < temp->numElements - 1; j++) {
                temp->elements[j] = temp->elements[j + 1];
            }
        }
    }
}
```

The local shift:

```
for (int j = i; j < temp->numElements - 1; j++) {
    temp->elements[j] = temp->elements[j + 1];
}
```

1. *At most N nodes are visited.*
2. *Each nodes contains B elements.*
3. *Thus there can be at most NB element comparisons.*
4. *Then at most ' $B - 1$ ' local shift occurs. i. e.*

$$\sum_{i=0}^{B-2} 1 = (B - 2 - 0) + 1 = B - 1 \text{ local shift.}$$

[As shift starts from $j = 0$ to $j < B - 1$ i. e. 0 to $B - 2$]

$$\begin{aligned} \text{Therefore, } O(NB + B) + O(-1) \\ = O(NB + B) \end{aligned}$$

As , during , Absolute Worst Case, when Element Not Found , +B shifting term is completely absent(0).

$$\begin{aligned}\text{Therefore , } O(NB + 0) \\ = O(NB)\end{aligned}$$

Note: N is the dominant term and B is just constant , here 4.

$$\begin{aligned}\therefore O(4N) \\ = O(N)\end{aligned}$$

2. Hence , In terms of nodes and blocks : $O(N)$.

$\therefore$ [Absolute Worst Case] Element Not Found $:= O(n)$ or $O(N)$.

[Not Absolute Worst Case]

Element at the Very End

1. **Nodes Traversed** : The outer loop visits every single node. Cost = $c_n \times N$.

2. **Elements Compared**: The inner loop checks every element until the very last one.
 $= c_s \times n$.

3. **Elements Shifted**: Because the element is at the very end of the node's internal array, there are no elements left to get shifted . The shifting loop condition $j < temp \rightarrow numElements - 1$ immediately evaluates to false.

Hence 0 elements are shifted . Cost = $c_{sh} \times 0 \text{ times} = 0$.

4. **Deletions**: The node bookkeeping executes, constant amount of time . Costs
 $= c_d \times 1 \text{ time} = c_d$.

$$\therefore T_2 = (c_n \times N) + (c_s \times n) + 0 + c_d$$

As , $N \leq n$, the dominant term is n .

1. In terms of elements : $O(n)$.

Also , as shifting term : $+B$ doesnot exist here , therefore:

$$\begin{aligned} \text{Therefore , } O(NB + 0) \\ = O(NB) \end{aligned}$$

Note: N is the dominant term and B is just constant , here 4.

$$\begin{aligned} \therefore O(4N) \\ = O(N) \end{aligned}$$

2. Hence , In terms of nodes and blocks : $O(N)$.

Hence , Worst Case Time Complexity : $O(n)$ or $O(N)$.

Average Case Time Complexity

A. Average Comparisons

A. Based on Loop

In a Node:

```
while (temp != NULL) {
```

Inner loop comparisons:

```
for (int i = 0; i < temp->numElements; i++) {  
    if (temp->elements[i] == val) {
```

- *When we want to delete the 1st element, the inner loop comparison runs 1 time.*

i. e. temp → elements[0] == val [True]

- *When we want to delete the 2nd element, the inner loop comparison runs 2 times.*

i. e. temp → elements[0] == val [False]

temp → elements[1] == val [True]

- *When we want to delete the 3rd element, the inner loop comparison runs 3 times.*

i. e. temp → elements[0] == val [False]

temp → elements[1] == val [False]

temp → elements[2] == val [True]

- *When **n** element is to be deleted: loop runs **n** times.*

Note : we are doing comparisons like:

Node 1 (Elements 1 through 4)

- *Target is 1st element: 1 comparison.*
- *Target is 2nd element: 2 comparisons.*
- *Target is 3rd element: 3 comparisons.*
- *Target is 4th element: 4 comparisons.*

Node 2 (Elements 5 through 8)

- *Target is 5th element: 4 comparisons (from Node 1) + 1 comparison (in Node 2) = 5 total comparisons.*
- *Target is 6th element: 4 + 2 = 6 total comparisons.*
- *Target is 7th element: 4 + 3 = 7 total comparisons.*
- *Target is 8th element: 4 + 4 = 8 total comparisons.*

Hence ,here `n` elements represents total `n` elements of `N` node blocks.

***And,when `n` element is to be deleted ,we actually run loop
`n` number of times .***

*If there is **n** number of elements,we can delete data at **n** positions.*

- *For position 1,the probability of deletion is $\frac{1}{n}$.*
- *For position 2,the probability of deletion is $\frac{1}{n}$.*
- *For position n,the probability of deletion is $\frac{1}{n}$.*

Hence probability of deletion will always remain $\frac{1}{n}$.

Average Time Complexity is =

probability of deletion $\times$ (number of possible iterations):

$$= \left(\frac{1}{n} \times 1 + \frac{1}{n} \times 2 + \dots + \frac{1}{n} \times n \right)$$

$$= \frac{1}{n} \times \{1 + 2 + 3 + \dots + n\}$$

$$= \frac{1}{n} \times \left\{ \sum_{i=1}^n i \right\}$$

$$= \frac{1}{n} \left\{ \frac{n(n+1)}{2} \right\}$$

$$= \frac{n^2 + n}{2n}$$

$$= \frac{n^2}{2n} + \frac{n}{2n}$$

$$= \frac{n}{2} + \frac{1}{2}$$

$$= \frac{n+1}{2}$$

Alternate simplest approach :

B. Based on Position

- ***For position 1, the probability of deletion is $\frac{1}{n}$.***
- ***For position 2, the probability of deletion is $\frac{1}{n}$.***
- ***For position n, the probability of deletion is $\frac{1}{n}$.***

Hence probability of deletion will always remain $\frac{1}{n}$ for position 1 to n:

Average case complexity:

Probability of deletion $\times$ (Summation of number of Position).

$$\begin{aligned} &= \frac{1}{n} \times \left(\sum_{i=1}^n i \right) \\ &= \frac{1}{n} \times \left(\frac{n(n+1)}{2} \right) \\ &= \frac{n+1}{2} \end{aligned}$$

A. Total average comparison cost: $c_s \times \left(\frac{n+1}{2} \right)$

B. Average Node Traversal

The unrolled linked list consists of N total nodes. Finding a randomly distributed element requires hopping through the node blocks.

When searching for an element, the outer while (temp $\neq$ NULL) loop hops from node to node.

If list has a total of N nodes, the element must be in one of them:

- If the element is in the 1st node, the loop runs 1 time.***
- If the element is in the 2nd node, the loop runs 2 times.***
- If the element is in the N – th node, the loop runs N times.***

If we assume the elements are distributed evenly across the nodes ,picking a random element means it is equally likely to be located inside any of the N nodes. Therefore, the probability of the element being in any specific node is $\frac{1}{N}$.

$$\text{Average} = (\text{Prob. of Node 1} \times 1 \text{ hop}) + (\text{Prob. of Node 2} \times 2 \text{ hops}) + \dots + (\text{Prob. of Node } N \times N \text{ hops})$$

Substitute the probability $\left(\frac{1}{N}\right)$ into the equation:

$$\text{Average Traversals} = \left(\frac{1}{N} \times 1\right) + \left(\frac{1}{N} \times 2\right) + \left(\frac{1}{N} \times 3\right) + \dots + \left(\frac{1}{N} \times N\right)$$

$$= \frac{1}{N} \times (1 + 2 + 3 + \dots + N)$$

$$= \frac{1}{N} \times \sum_{i=1}^N i$$

$$= \frac{1}{N} \times \frac{N(N+1)}{2}$$

$$= \frac{N+1}{2}$$

$$\text{B. Total Average Traversal cost: } c_n \times \left(\frac{N+1}{2}\right)$$

C. Average Array Shift

```
for (int j = i; j < temp->numElements - 1; j++) {  
    temp->elements[j] = temp->elements[j + 1];  
}
```

Assuming if a node is 100% full, then it has B number of elements:

Loop generally traverse from 0^{th} index to $(B - 2)^{th}$ index, where last shift will be element at $B - 2^{th}$ index = $B - 1^{th}$ index.

Therefore, we can compute it as follows:

(Deleting at the very end i.e. any node's B^{th} element, sitting):
At Index $B - 1$

There are 0 elements to the right of it. Number of shifts = 0.

(Deleting at the second to last spot i.e. any node's $B - 1^{th}$ element, sitting):
At Index $B - 2$

There is 1 element to the right of it. Number of shifts = 1.

(Deleting at the third to last spot i.e. any node's $B - 2^{th}$ element, sitting):
At Index $B - 3$

There is 2 element to the right of it. Number of shifts = 2.

.
.

(Deleting at the very beginning i.e. any node's 1^{th} element, sitting):
At Index 0

There is $B - 1$ element to the right of it. Number of shifts = $B - 1$.

$$\text{Average of Array Shift} = \frac{\text{Sum of all possible shift counts}}{\text{Total number of possible positions}}$$

$$\text{Average of Array Shift} = \frac{0 + 1 + 2 + \dots + B - 1}{B}$$

$$= \frac{\sum_{i=0}^{B-1} i}{B}$$

$$= \frac{0 + \sum_{i=1}^{B-1} i}{B}$$

$$= \frac{\sum_{i=1}^{B-1} i}{B}$$

$$= \frac{\frac{(B-1)((B-1)+1)}{2}}{B} \left[\begin{array}{l} \sum_{i=1}^n i = \frac{1}{2} \times n \times (n+1), \\ \text{plugging } n = B-1, \text{ we get:} \\ \frac{1}{2} \times (B-1) \times ((B-1)+1) \end{array} \right]$$

$$= \frac{\frac{(B-1)(B)}{2}}{B}$$

$$= \frac{(B-1)(B)}{2} \times \frac{1}{B}$$

$$= \frac{B-1}{2}$$

The shifting operation only ever happens exactly one time during the entire function call. Therefore for a single node's average case is required here, not for continuous N^{th} node.

C. Total Average Shifting cost: $c_{sh} \times \left(\frac{B+1}{2}\right)$

D. Deletion Bookkeeping Costs

```
temp->numElements--;

        cout << "Element " << val << " deleted
successfully.\n";

        // Check if the node is now completely empty
        if (temp->numElements == 0) {
            if (prev == NULL) {
                head = temp->next;
                free(temp);
            } else {
                prev->next = temp->next;
                free(temp);
            }
            cout << "An empty node was removed from the
list.\n";
        }
```

The actual deletion process (decrementing numElements, evaluating if conditions, and potentially freeing an empty node) executes exactly one time when the element is successfully found.

This is a fixed constant cost.

$$\therefore \text{D.Total Deletion Cost} = c_d \times 1 = c_d .$$

$$\begin{aligned} T_{avg} = & \text{Total average comparison cost: } c_s \times \left(\frac{n+1}{2}\right) \\ & + \text{Total Average Traversal cost: } c_n \times \left(\frac{N+1}{2}\right) \\ & + \text{Total Average Shifting cost: } c_{sh} \times \left(\frac{B+1}{2}\right) \\ & + \text{Total Deletion Cost} = c_d \times 1 = c_d . \end{aligned}$$

$$T_{avg} = c_s \times \left(\frac{n+1}{2}\right) + c_n \times \left(\frac{N+1}{2}\right) + c_{sh} \times \left(\frac{B+1}{2}\right) + c_d$$

To express this in standard Big – O notation, we strip away all constant multipliers (like $\frac{1}{2}$), exact hardware costs (c_n, c_s, c_{sh}, c_d), and lower – order additive constants:

$$T_{avg} = O(n) + O(N) + O(B) + O(1)$$

- **Node Capacity is Constant:** Because $B = 4$,
the maximum shifting boundary $O(B)$ is just
 $O(4)$, which simplifies directly to $O(1)$.

- **N scales with n :** Because the total number of nodes is
 $N \approx \left\lceil \frac{n}{4} \right\rceil$, N is directly proportional to n .

This means traversing N nodes takes the same linear time
as searching n elements,
making $O(N)$ mathematically equivalent to $O(n)$.

$\therefore$ In terms of Elements(n): $T_{avg} = O(n) + O(n) + O(1) + O(1)$
 $= O(n) \left[\begin{array}{l} \text{As, } n \text{ is the} \\ \text{dominating} \\ \text{Term} \end{array} \right]$

Because $O(n)$ is the term with the highest growth rate, it absorbs all constants.

or, In terms of Nodes(N): $T_{avg} = O(N) + O(N) + O(1) + O(1)$
 $= O(N) \left[\begin{array}{l} \text{As, } N \text{ is the} \\ \text{dominating} \\ \text{Term} \end{array} \right]$

Because $O(N)$ is the term with the highest growth rate, it absorbs all constants.

Therefore, the **final Average Case Time Complexity** is definitively:

$O(n)$ or $O(N)$.

2.c.Unrolled List

–Delete Element

–Space Complexity Analysis and Algorithm

Program:

```
void deleteElement(int val) {
    if (head == NULL) {
        cout << "The list is empty. Nothing to delete.\n";
        return;
    }

    struct Node* temp = head;
    struct Node* prev = NULL;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {

                // Element found: Shift remaining elements left
                for (int j = i; j < temp->numElements - 1; j++) {
                    temp->elements[j] = temp->elements[j + 1];
                }
                temp->numElements--;

                cout << "Element " << val << " deleted
successfully.\n";

                // Check if the node is now completely empty
                if (temp->numElements == 0) {
                    if (prev == NULL) {
                        head = temp->next;
                        free(temp);
                    } else {
                        prev->next = temp->next;
                        free(temp);
                    }
                    cout << "An empty node was removed from the
list.\n";
                }
            }
        }
    }
}
```

```
        return;
    }
}
prev = temp;
temp = temp->next;
}

cout << "Element " << val << " not found in the list.\n";
}
....

    case 2:
        cout << "Enter integer value to delete: ";
        cin >> value;
        deleteElement(value);
        break;
```

Space Complexity Analysis

1. Input Space Analysis

The Input Space is the memory required to store the data that the algorithm operates on.

We can look at this from two perspectives:

- *the specific parameter passed to the function,*
- *and the entire data structure it acts upon.*

A. Parametric Input Space

This is the memory required to, store the explicit arguments passed directly into the function call.

Code: void deleteElement(int val)

Analysis: The function takes exactly one parameter: a single integer val. This integer requires a fixed, constant amount of memory (typically 4 bytes), regardless of how large the linked list is.

Complexity: $O(1)$ (Constant Space)

B. Full Data Holistic Input Space

This is the memory required to store the entire existing unrolled linked list that the function is searching through and modifying.

Analysis: *The data structure consists of N nodes. Each struct Node contains:*

- *An array of fixed capacity B ($MAX_ELEMENTS = 4$)*
- *An integer ($numElements$)*
- *A pointer ($*next$)*

Because each node holds a constant maximum of B elements (where $B = 4$), the memory size of a single node is fixed.

To store n total elements, the structure requires $N \approx \left\lceil \frac{n}{B} \right\rceil$ nodes.

The size of one node is constant ($O(1)$). Because there are N nodes in the list, the total memory consumed by the entire data structure scales linearly with the number of elements n .

Complexity: $O(N)$ or $O(n)$ (Linear Space)

2. Auxiliary Space Complexity: $O(1)$

Auxiliary Space measures the extra, temporary memory that the algorithm uses while executing, exclusively excluding the input data itself. This determines how much additional RAM that our function demands to do its job.

*Let's break down every temporary variable created inside the **deleteElement** block:*

- **Pointers:** The function initializes two local pointer variables to traverse the list:
 - `struct Node * temp = head;`
 - `struct Node * prev = NULL;`
- **Loop Counters:** The function uses standard integer variables for iterating:
 - `int i` for the search loop.
 - `int j` for the shifting loop.

Key Observations:

1. **No Dynamic Allocation:** The function never calls `malloc`, `new`, or creates any new nodes. It only modifies or frees existing memory (`free(temp)`).
2. **No Recursion:** The function uses iterative `while` and `for` loops. There are no recursive function calls, meaning the Call Stack does not grow. It stays at a fixed depth of 1 frame.
3. **Constant Memory:** The pointers (`temp, prev`) and integers (`i, j`) take a fixed amount of bytes.

Result: Because the extra memory required by the function is fixed and completely independent of the input size `n` or `N`, the Auxiliary Space Complexity is strictly $O(1)$ (Constant Space).

For Academic Answer = Auxiliary Space = $O(1)$.

"As we often cannot control the size of the input data, but we can control how much extra memory of our algorithm creates to process it."

Total Space Complexity or Total memory footprint:

***= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space***

= $(O(1) + O(N)) + O(1)$

= $O(N)$.

or,

$(O(1) + O(n)) + O(1)$

= $O(n)$.

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the list(N) scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The for loop is governed by the condition $i < N$, meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

*Inside the loop, the **malloc** function is called during every single iteration.*

***malloc** physically claims a brand – new block of memory from the computer's heap.*

3. Total Memory Consumed

*If **sizeof(struct Node)** takes a fixed constant amount of memory (for example, 24 bytes), the total extra memory generated by this loop is:*

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

*In Big – O notation, we drop the exact hardware constants (like 24 bytes) because we only care about how the memory requirements grow as N gets larger. Because the memory requested scales linearly with the variable N , the complexity is **$O(N)$** .*

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N * sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

Note: We came through this scenario earlier in

skip list:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * (level + 1));
```

if we have wrote:

```
n->forward = (struct Node **)malloc(sizeof(struct Node *) * N);
```

*here ,space complexity would have been : **$O(N)$***

But as it is: $(level + 1) = O(level)$ and level is capped with a hardcoded constant:

```
#define MAX_LEVEL 3
```

As it require `level + 1` , we can say space complexity = $O(level)$, but :

As, level = MAX_LEVEL = 3 is a constant ,

The function allocates dynamic memory on the heap.

Because $level \leq 3$, the forward array will allocate a maximum of 4 pointers.

Allocating a fixed maximum amount of memory, is $O(1)$ space.

Algorithm: UnrolledDELElem(val)

1. [CHECK IF LIST IS EMPTY]

- **IF $START = NULL$, then:**
 - **Write: "The list is empty. Nothing to delete."**
 - **RETURN**
- **[End of If structure.]**

2. [INITIALIZE TRAVERSAL POINTERS]

- **SET $TEMPPTR := START$**
- **SET $PREVPTR := NULL$**

3. [OUTER LOOP: NODE TRAVERSAL]

- **REPEAT WHILE $TEMPPTR \neq NULL$:**

4. [INNER LOOP: ELEMENT SEARCH]

- **SET $i := 0$**
- **REPEAT WHILE $i < numElements[TEMPPTR]$:**

5. [CHECK FOR MATCH AND DELETE]

- **IF $elements[TEMPPTR][i] = val$, Then:**
 - **Write: "Element ", val , " deleted successfully."**

(Shift remaining elements left)

- **SET $j := i$**
- **REPEAT WHILE $j < numElements[TEMPPTR] - 1$:**
 - **SET $elements[TEMPPTR][j] := elements[TEMPPTR][j + 1]$**
 - **SET $j := j + 1$**

[End of Shifting while Loop]

(Decrement element count)

▪ *SET numElements[TEMPPTR] := numElements[TEMPPTR] - 1*

(Check if the node is now completely empty)

▪ *IF numElements[TEMPPTR] = 0, Then:*

▪ *IF PREVPTR = NULL, Then:*

▪ *SET START := LINK[TEMPPTR]*

▪ *ELSE:*

▪ *SET LINK[PREVPTR] := LINK[TEMPPTR]*

[End of inner If structure.]

▪ *FREE(TEMPPTR) (Return node space to memory)*

▪ *Write: "An empty node was removed from the list."*

▪ *[End of empty node check If structure.]*

▪ *RETURN (Exit the algorithm immediately)*

▪ *[End of Match If structure.]*

▪ *SET i := i + 1*

○ *[End of Inner Search Loop]*

(Move to the next node if not found in the current node)

○ *SET PREVPTR := TEMPPTR*

○ *SET TEMPPTR := LINK[TEMPPTR]*

• *[End of Outer Node Traversal Loop]*

6. [ELEMENT NOT FOUND]

• *Write: "Element ", val, " not found in the list."*

7. END OF ALGORITHM

Unrolled List
–Display List
–Program Analysis

Program

```
void displayList() {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    cout << "\n--- Unrolled Linked List Content ---\n";
    while (temp != NULL) {
        cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
        for (int i = 0; i < temp->numElements; i++) {
            cout << temp->elements[i];
            if (i < temp->numElements - 1) {
                cout << ", ";
            }
        }
        cout << "]\n";

        temp = temp->next;
        nodeCount++;
    }
    cout << "-----\n";
}

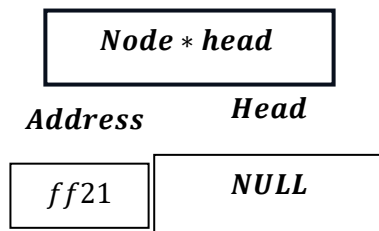
.....

    case 3:
        displayList();
        break;
```

Program Analysis

1. List is Empty

```
if (head == NULL) {  
    cout << "The list is empty.\n";  
    return;  
}
```

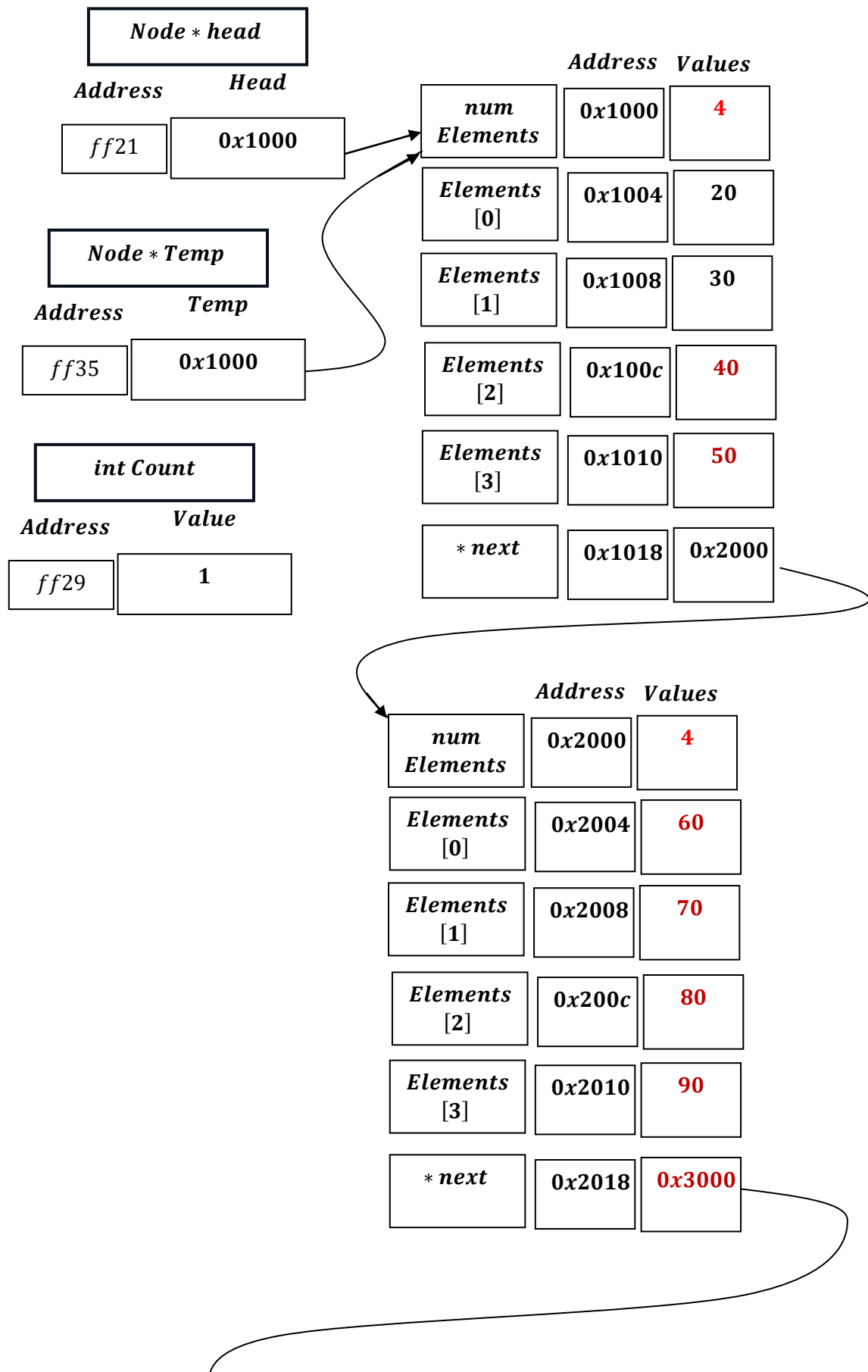


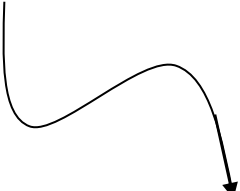
As head = NULL, it prints : `` The list is empty.``

Return void and exit from the function.

2. List is not Empty

```
struct Node* temp = head;  
int nodeCount = 1;
```





	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements [0]</i>	0x3004	100
<i>Elements [1]</i>	0x3008	110
<i>Elements [2]</i>	0x300C	120
<i>Elements [3]</i>	0x3010	130
<i>* next</i>	0x3018	NULL

```

cout << "\n--- Unrolled Linked List Content ---\n";
while (temp != NULL) {
    cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
    for (int i = 0; i < temp->numElements; i++) {
        cout << temp->elements[i];
        if (i < temp->numElements - 1) {
            cout << ", ";
        }
    }
    cout << "]\n";

    temp = temp->next;
    nodeCount++;
}

```

Print `` --- Unrolled Linked List Content --- ``

While(Temp: 0x1000 $\neq$ NULL) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

Print : Node 1 (temp $\rightarrow$ numElements: 4 \ MAX_ELEMENTS: 4 elements): [

for: int $i = 0$; $i < temp \rightarrow numElements: 4$

Print: temp $\rightarrow$ elements[0] := 20

if($i = 0 < temp \rightarrow numElements: 4 - 1 = 3$)

Print: `` , ``

$i = i + 1 = 0 + 1 = 1$

for: int $i = 1$; $i < temp \rightarrow numElements: 4$

Print: temp $\rightarrow$ elements[1] := 30

if($i = 1 < temp \rightarrow numElements: 4 - 1 = 3$)

Print: `` , ``

$i = i + 1 = 1 + 1 = 2$

for: int $i = 2$; $i < temp \rightarrow numElements: 4$

Print: temp $\rightarrow$ elements[2] := 40

if($i = 2 < temp \rightarrow numElements: 4 - 1 = 3$)

Print: `` , ``

$i = i + 1 = 2 + 1 = 3$

for: int $i = 3$; $i < temp \rightarrow numElements: 4$

Print: temp $\rightarrow$ elements[3] := 50

if($i = 3 < temp \rightarrow numElements: 4 - 1 = 3$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

$i = i + 1 = 3 + 1 = 4$

for: int i = 4; i < temp → numElements: 4 $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

```
cout << "]\n";
```

Print: ``]`` and next : newline i.e. end of line and start from a newline.

And finally we have print:

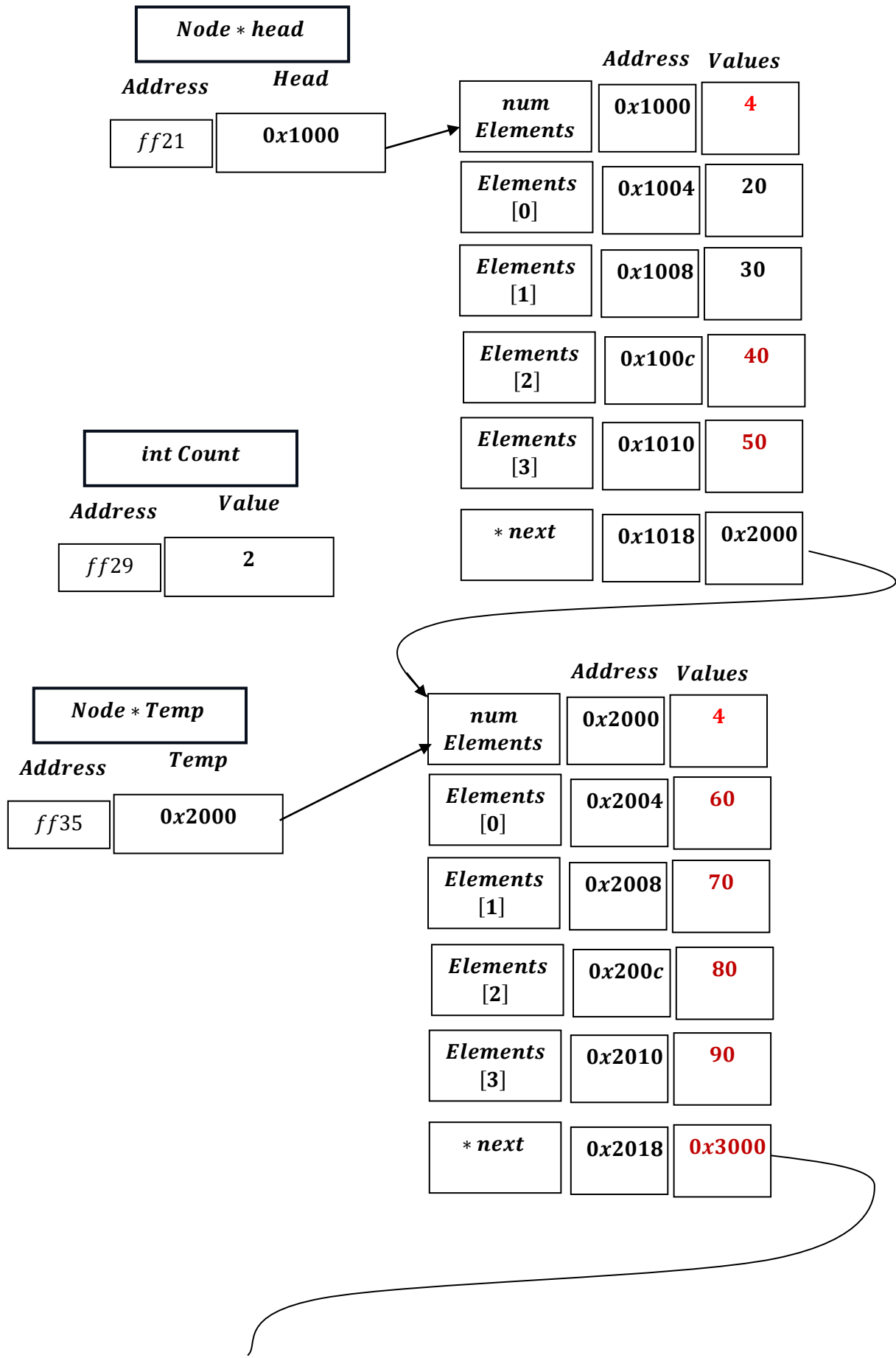
— — —Unrolled Linked List Content — — —

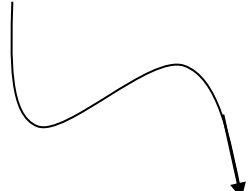
Node 1 (4 /4 elements): [20, 30, 40, 50]

```
temp = temp->next;  
nodeCount++;
```

Temp = Temp → Next : 0x2000

nodeCount = nodeCount + 1 = 1 + 1 = 2.





	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements [0]</i>	0x3004	100
<i>Elements [1]</i>	0x3008	110
<i>Elements [2]</i>	0x300C	120
<i>Elements [3]</i>	0x3010	130
<i>* next</i>	0x3018	NULL

While(*Temp*: 0x2000 $\neq$ *NULL*) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

Print : Node 2 (*temp* $\rightarrow$ *numElements*: 4 \ *MAX_ELEMENTS*: 4 elements): [

for: *int i* = 0; *i* < *temp* $\rightarrow$ *numElements*: 4

Print: *temp* $\rightarrow$ *elements*[0] := 60

if(*i* = 0 < *temp* $\rightarrow$ *numElements*: 4 – 1 = 3)

Print: `` , ``

i = *i* + 1 = 0 + 1 = 1

for: *int i* = 1; *i* < *temp* $\rightarrow$ *numElements*: 4

Print: *temp* $\rightarrow$ *elements*[1] := 70

if(*i* = 1 < *temp* $\rightarrow$ *numElements*: 4 – 1 = 3)

Print: `` , ``

i = *i* + 1 = 1 + 1 = 2

for: int i = 2; i < temp → numElements: 4

Print: temp → elements[2] := 80

if(i = 2 < temp → numElements: 4 - 1 = 3)

Print: `` , ``

i = i + 1 = 2 + 1 = 3

for: int i = 3; i < temp → numElements: 4

Print: temp → elements[3] := 90

if(i = 3 < temp → numElements: 4 - 1 = 3) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

i = i + 1 = 3 + 1 = 4

for: int i = 4; i < temp → numElements: 4 $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

```
cout << "]\n";
```

Print: `` ] `` and next : newline i.e. end of line and start from a newline.

And finally we have print:

— — —Unrolled Linked List Content — — —

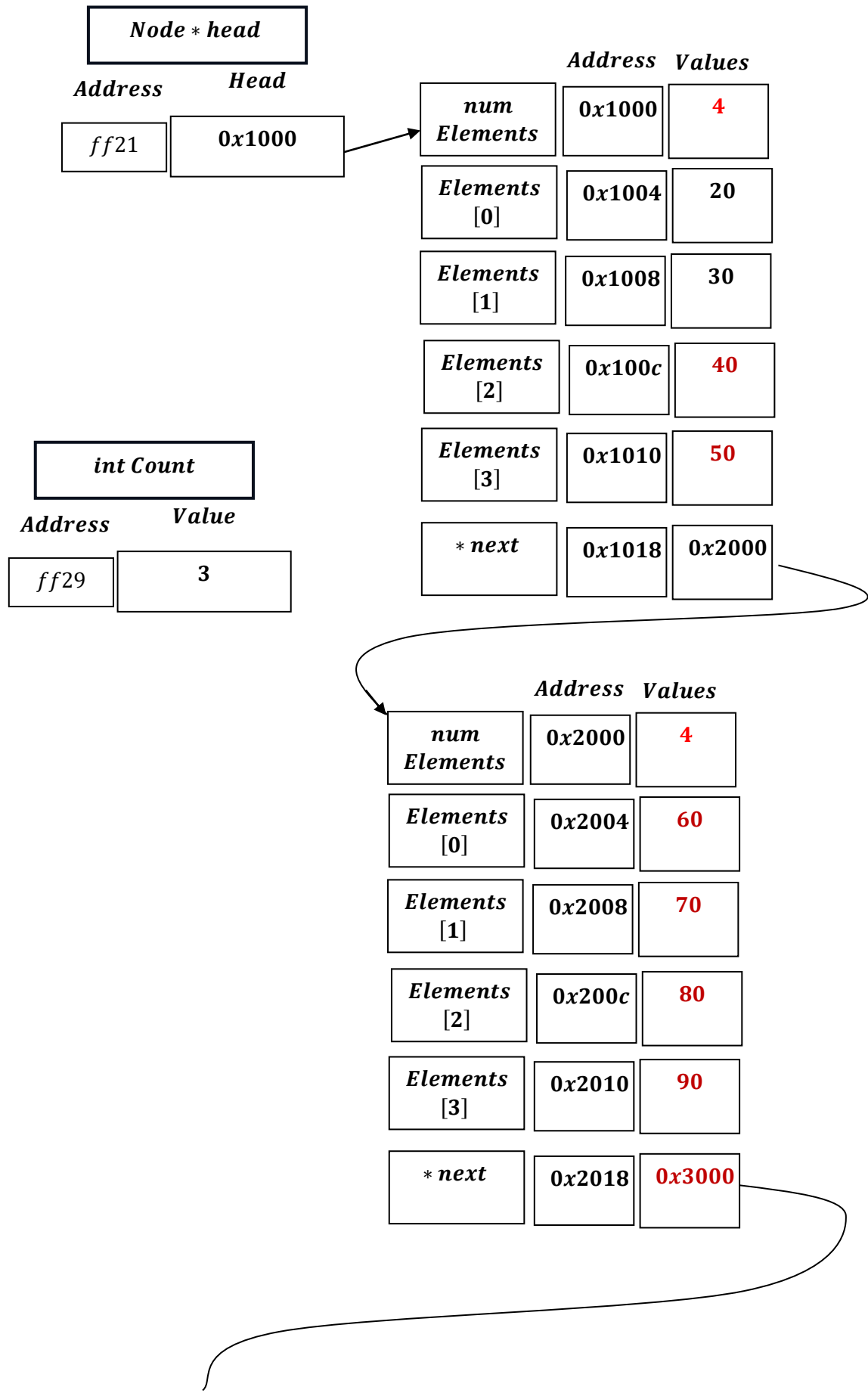
Node 1 (4 /4 elements): [20,30,40,50]

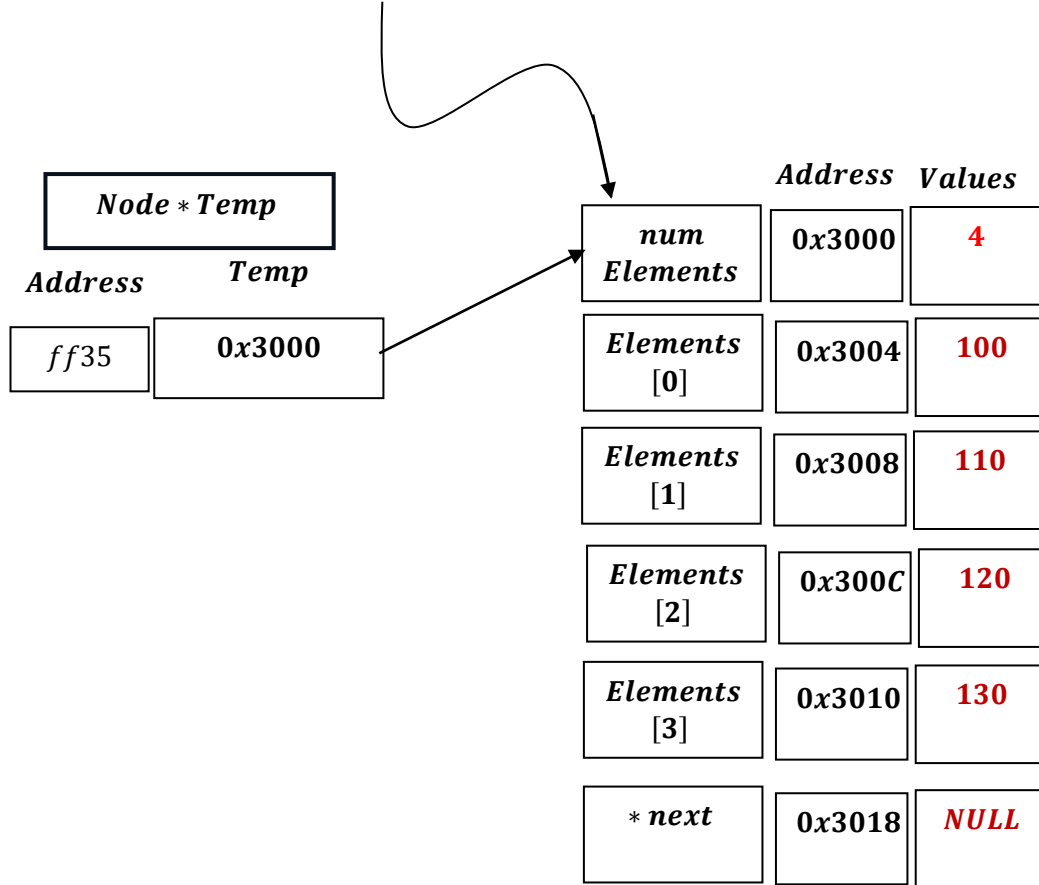
Node 2 (4 /4 elements): [60,70,80,90]

```
temp = temp->next;  
nodeCount++;
```

Temp = Temp → Next : 0x3000

nodeCount = nodeCount + 1 = 2 + 1 = 3.





While(Temp: 0x3000 ≠ NULL) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

Print : Node 3 (temp → numElements: 4 \ MAX_ELEMENTS: 4 elements): [

for: int $i = 0; i < temp \rightarrow numElements: 4$

Print: temp → elements[0] := 100

if($i = 0 < temp \rightarrow numElements: 4 - 1 = 3$)

Print: `` , ``

$i = i + 1 = 0 + 1 = 1$

for: int $i = 1; i < temp \rightarrow numElements: 4$

Print: temp → elements[1] := 110

if($i = 1 < temp \rightarrow numElements: 4 - 1 = 3$)

Print: `` , ``

$i = i + 1 = 1 + 1 = 2$

for: int i = 2; i < temp → numElements: 4

Print: temp → elements[2] := 120

if(i = 2 < temp → numElements: 4 – 1 = 3)

Print: `` , ``

i = i + 1 = 2 + 1 = 3

for: int i = 3; i < temp → numElements: 4

Print: temp → elements[3] := 130

if(i = 3 < temp → numElements: 4 – 1 = 3) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

i = i + 1 = 3 + 1 = 4

for: int i = 4; i < temp → numElements: 4 $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

```
cout << "]\n";
```

Print: `` ] `` and next : newline i.e. end of line and start from a newline.

And finally we have print:

– – –Unrolled Linked List Content – – –

Node 1 (4 /4 elements): [20, 30, 40, 50]

Node 2 (4 /4 elements): [60, 70, 80, 90]

Node 3 (4 /4 elements): [100, 110, 120, 130]

```
temp = temp->next;  
nodeCount++;
```

Temp = Temp → Next : NULL

nodeCount = nodeCount + 1 = 3 + 1 = 4.

While(Temp: NULL $\neq$ NULL) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \end{array} \right]$

And , then we print:

```
cout << "-----\n";
```

Therefore , final output:

— — —Unrolled Linked List Content — — —

Node 1 (4 /4 elements): [20,30,40,50]

Node 2 (4 /4 elements): [60,70,80,90]

Node 3 (4 /4 elements): [100,110,120,130]

Unrolled List
–Display List
–Time Complexity Analysis

Program:

```
void displayList() {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    cout << "\n--- Unrolled Linked List Content ---\n";
    while (temp != NULL) {
        cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
        for (int i = 0; i < temp->numElements; i++) {
            cout << temp->elements[i];
            if (i < temp->numElements - 1) {
                cout << ", ";
            }
        }
        cout << "]\n";

        temp = temp->next;
        nodeCount++;
    }
    cout << "-----\n";
}

.....

    case 3:
        displayList();
        break;
```

Time Complexity Analysis

- *n*: The total number of elements currently stored across the entire list.
- *N*: The total number of node blocks currently chained together in the list.
- *B*: The maximum capacity of each node (**MAX_ELEMENTS**), which is fixed at 4.

Because each node holds up to *B* elements, the number of nodes is bounded by

$N = \left\lceil \frac{n}{B} \right\rceil$. Since *B* is a constant (4), *N* scales directly and proportionally

with *n*.

Therefore, each node holds up to 4 elements, *N*

$\approx \left\lceil \frac{n}{4} \right\rceil$, meaning *N* scales proportionally with *n*.

Therefore, $O(N)$ is mathematically equivalent to $O(n)$ i.e $O(N) \equiv O(n)$.

1. Best Case Time Complexity: $O(1)$

The absolute best – case scenario occurs when the function

is called on an empty data structure.

- **Condition**: if (**head** == **NULL**) evaluates to true.
- **Execution**: The program prints The list is empty. and immediately executes a **return**; statement, exiting the function.
- **Cost**: Because it performs a single check and terminates

without traversing any loops, it executes

in strictly constant time: **$O(1)$** .

2. Worst & Average Case Time Complexity: $O(n)$ (or $O(N)$)

For the `displayList` function, the worst – case and average – case time complexities are identical.

There is no probability or early exit to average out; the function is forced to perform a complete, exhaustive traversal of the unrolled linked list(When not Empty).

A. Node Traversal (Outer Loop)

```
while (temp != NULL) {
    cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
    ...
}
```

- The outer loop **while (temp ≠ NULL)** visits every single node from **head** to **NULL**.

```
cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
```

- *Inside this loop, it prints the node header information*
(e.g., Node 1 (4/4 elements)) takes $1 + 1 + 1 \dots N = N$ times.

```
cout << "]\n";
```

- Prints ``` and goes to new line takes $1 + 1 + 1 \dots N = 'N'$ times.

```
temp = temp->next;
```

- **Temp = Temp → Next**, this assignment runs, till **Temp = NULL** hence : $1 + 1 + 1 \dots N = N$ times.

```
nodeCount++;
```

- Increments the **nodeCount** till nodecounts, runs: $1 + 1 + 1 \dots N = N$ times.

That is all these **4** line of codes runs inside **while (temp ≠ NULL)** : **4N** times, where, **N** is the total number of node blocks currently chained together in the list.

- **Cost**: Let c_n be the cost of hopping to the next node and printing the header. The total traversal cost is $c_n \times N$ or **4N**.

B. Element Display (Inner Loop)

```
for (int i = 0; i < temp->numElements; i++) {  
    cout << temp->elements[i];  
    if (i < temp->numElements - 1) {  
        cout << ", ";  
    }  
}
```

Within each visited node, the inner loop executes. While it runs a maximum of **B** times per individual node, across the entire execution of the program, it must process every single element in the data structure exactly once.

```
cout << temp->elements[i];
```

Prints the actual integer value. Since it prints every active element across the entire list, this takes $1 + 1 + 1 \dots n = n$ times.

Where, **n**, is the total number of elements currently stored across the entire list.

Note : we are printing like:

Node 1 (Elements 1 through 4)

- *Target is 1st element: Prints 1st element.*
- *Target is 2nd element: Prints 2nd element.*
- *Target is 3rd element: Prints 3rd element.*
- *Target is 4th element: Prints 4th element.*

Node 2 (Elements 5 through 8)

- *Target is 5th element: Prints 5th element.*
- *Target is 6th element: Prints 6th element.*
- *Target is 7th element: Prints 7th element.*
- *Target is 8th element: Prints 8th element.*

Hence , here `n` elements represents total `n` elements of `N` node blocks.

```
if (i < temp->numElements - 1)
```

Evaluates whether the current element is the last one in the node to determine if a comma is needed. This condition is checked for every single element, taking $1 + 1 + 1 \dots n = n$ times.

```
cout << ", ";
```

Let's represent the number of elements in each node as variables:

- *Node 1 has k_1 elements.*
- *Node 2 has k_2 elements.*
- *Node N has k_N elements.*

The total number of elements in the entire list is:

$$n = k_1 + k_2 + \dots + k_N$$

Now, let's calculate the total number of commas printed based on the code's rule (print commas for all elements minus 1 per node):

- Commas in Node 1 = $k_1 - 1$
- Commas in Node 2 = $k_2 - 1$
- Commas in Node N = $k_N - 1$

To find the global total of commas printed, we add them all together:

$$\text{Total Commas} = (k_1 - 1) + (k_2 - 1) + \dots + (k_N - 1)$$

We can rearrange the terms by grouping the k variables together and the 1s together:

$$\text{Total Commas} = (k_1 + k_2 + \dots + k_N) - (1 + 1 + \dots \text{repeated } N \text{ times})$$

Substitute n back into the first group, and simplify the second group:

$$\text{Total Commas} = n - N$$

That is, all these lines of code run inside the inner for loop a total of:

$$n + n + (n - N) = 3n - N \text{ times,}$$

where n is the total number of elements and N is the total number of nodes.

Cost: Let c_p be the combined constant cost of checking and printing a single element. The total element display cost is $c_p \times n$ or roughly $3n$.

$$T(n) = \text{Total Traversal Cost} + \text{Total Element Display Cost}$$

$$T(n) = (c_n \times N) + (c_p \times n)$$

or, represented by the exact execution counts:

$$T(n) = 4N + (3n - N) = 3N - 3n$$

$$\text{Where, } c_n = 3 \text{ and } c_p = 3.$$

To convert this into asymptotic Big – O notation, we drop the constant multipliers (3):

$$T(n) = O(N) + O(n)$$

As, $N \approx \left\lceil \frac{n}{B} \right\rceil$, here, $N \approx \left\lceil \frac{n}{4} \right\rceil$, meaning:

"N "scales directly and proportionally with "n".

*Therefore, **$O(N)$** is mathematically equivalent to **$O(n)$** , making the final Worst & Average Case Time Complexity definitively **$O(n)$** (or **$O(N)$**).*

***Note:** Fundamentally, an Unrolled Linked List still relies on sequential searching. Therefore, the Worst and Average Case Time Complexity always resolves to **$O(n)$** also, **$O(N)$** .*

There is no real Average Case here as:

Whether the list is completely full or mostly empty, if it has n elements, the computer must traverse N nodes and print n elements.

Hence : Worst = Average = $O(n)$ or $O(N)$.

Unrolled List

–Display List

–Space Complexity Analysis and Algorithm

Program:

```
void displayList() {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    cout << "\n--- Unrolled Linked List Content ---\n";
    while (temp != NULL) {
        cout << "Node " << nodeCount << " (" << temp->numElements <<
"/" << MAX_ELEMENTS << " elements): [";
        for (int i = 0; i < temp->numElements; i++) {
            cout << temp->elements[i];
            if (i < temp->numElements - 1) {
                cout << ", ";
            }
        }
        cout << "]\n";

        temp = temp->next;
        nodeCount++;
    }
    cout << "-----\n";
}

.....

    case 3:
        displayList();
        break;
```

Space Complexity Analysis

1. Input Data Space

The input space measures the total memory occupied by the data that the algorithm needs to process. We look at this from two perspectives:

A. Parametric Input Space: $O(1)$

This measures the memory required exclusively for the arguments passed directly into the function signature.

- *The function signature is `void displayList()`.*
- *Unlike `insertElement(int val)` or `deleteElement(int val)`, this function accepts zero parameters.*
- *Because no parameters are passed into the function, it requires absolutely no memory for arguments.*
- **Result:** *The parametric input space is strictly $O(1)$ (Constant Space).*

B. Full Data Holistic Space: $O(N)$ or $O(n)$

This measures the memory consumed by the actual data structure the algorithm operates on (the unrolled linked list).

- *The function traverses a globally defined data structure accessed via the `head` pointer.*
- *The total memory occupied by this entire list scales linearly with the number of existing nodes.*

- *For a list containing n elements, the data is grouped into*

$$N \approx \left\lceil \frac{n}{4} \right\rceil \text{ nodes.}$$

- *Because the total memory footprint physically grows as more elements and nodes are added to the list, the memory requirement scales linearly.*
- *Result: The holistic input space is $O(N)$ (which is mathematically equivalent to $O(n)$ i. e. $[O(N) \equiv O(n)]$).*

2. Auxiliary Space Complexity: $O(1)$

Auxiliary space measures the temporary, extra memory that the algorithm actively creates to perform its task, entirely separate from the input data itself.

*Let's look at the exact variables declared inside the **displayList** function to see how much extra memory it asks the computer for:*

- *One temporary pointer: **struct Node * temp = head;** used to traverse the list.*
- *One counter variable: **int nodeCount = 1;** used to track the current node number.*
- *One loop iterator: **int i = 0;** used for the inner **for** loop to print elements.*

Key Observations:

1. **No Dynamic Allocation:** The function only reads data. It never uses `malloc()` or `new` to create additional nodes or arrays.
2. **No Recursion:** The function relies on simple iterative loops (while and for), meaning the Call Stack does not grow.

Result: Because the extra memory required by the function is fixed and does not change based on the size of the list, the Auxiliary Space Complexity is strictly **$O(1)$ (Constant Space)**.

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the `list(N)` scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The *for* loop is governed by the condition $i < N$, meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

Inside the loop, the *malloc* function is called during every single iteration.

malloc physically claims a brand – new block of memory from the computer's heap.

3. Total Memory Consumed

If *sizeof(struct Node)* takes a fixed constant amount of memory (for example, 24 bytes), the total extra memory generated by this loop is:

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

In Big – O notation, we drop the exact hardware constants (like 24 bytes) because we only care about how the memory requirements grow as N gets larger. Because the memory requested scales linearly with the variable N , the complexity is $O(N)$.

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N *  
sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

For Academic Answer = Auxiliary Space = $O(1)$.

*"As we often cannot control the size of the input data, but we
can control how much extra memory of our algorithm
creates to process it."*

Total Space Complexity or Total memory footprint:

***= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space***

$$= (O(1) + O(N)) + O(1)$$

$$= O(N).$$

or,

$$(O(1) + O(n)) + O(1)$$

$$= O(n).$$

Algorithm: UnrolledDISPList()

1. [CHECK IF LIST IS EMPTY]

- **IF START = NULL, Then:**
 - **Write: "The list is empty."**
 - **RETURN**
- **[End of If structure.]**

2. [INITIALIZE TRAVERSAL VARIABLES]

- *SET TEMP_PTR := START*
- *SET nodeCount := 1*
- *Write: " — — — Unrolled Linked List Content — — — "*

3. [OUTER LOOP: NODE TRAVERSAL]

- **REPEAT WHILE *TEMPPTR* $\neq$ NULL:**

4. *[PRINT NODE HEADER]*

- **Write:** `Node`, `nodeCount`, `(, numElements[TEMPPTR],/, MAX_ELEMENTS, elements):` [

5. [INNER LOOP: ELEMENT DISPLAY]

- *SET* $i := 0$
- *REPEAT WHILE* $i < \text{numElements}[\text{TEMPPTR}]$:
 - *Write*: $\text{elements}[\text{TEMPPTR}][i]$

(Check if a comma is needed)

- *IF* $i < \text{numElements}[\text{TEMPPTR}] - 1$, *Then:*
 - *Write:* ", "
 - [*End of inner If structure.*]
 - *SET* $i := i + 1$
- [*End of Inner Display Loop*]

6. [FINALIZE NODE AND MOVE TO NEXT]

- ***Write:***] (*Closes the bracket for the current node and starts a new line*)
 - ***SET TEMPPTR := LINK[TEMPPTR]*** (*Move to the next node block*)
 - ***SET nodeCount := nodeCount + 1*** (*Increment the node tracker*)
 - [***End of Outer Node Traversal Loop***]
- Write:*** " _ _ _ _ _ "

7. [END OF ALGORITHM]

***Unrolled List –
Search Element
–Program Analysis***

Program

```
// Function to search for an element
void searchElement(int val) {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {
                cout << "Element " << val << " found in Node " <<
nodeCount << " at index " << i << ".\n";
                return;
            }
        }
        temp = temp->next;
        nodeCount++;
    }

    cout << "Element " << val << " not found in the list.\n";
}

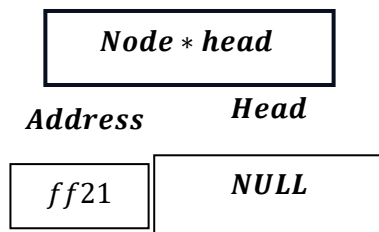
....

    case 4:
        cout << "Enter integer value to search: ";
        cin >> value;
        searchElement(value);
        break;
```

Program Analysis

1. List is Empty

```
if (head == NULL) {  
    cout << "The list is empty.\n";  
    return;  
}
```

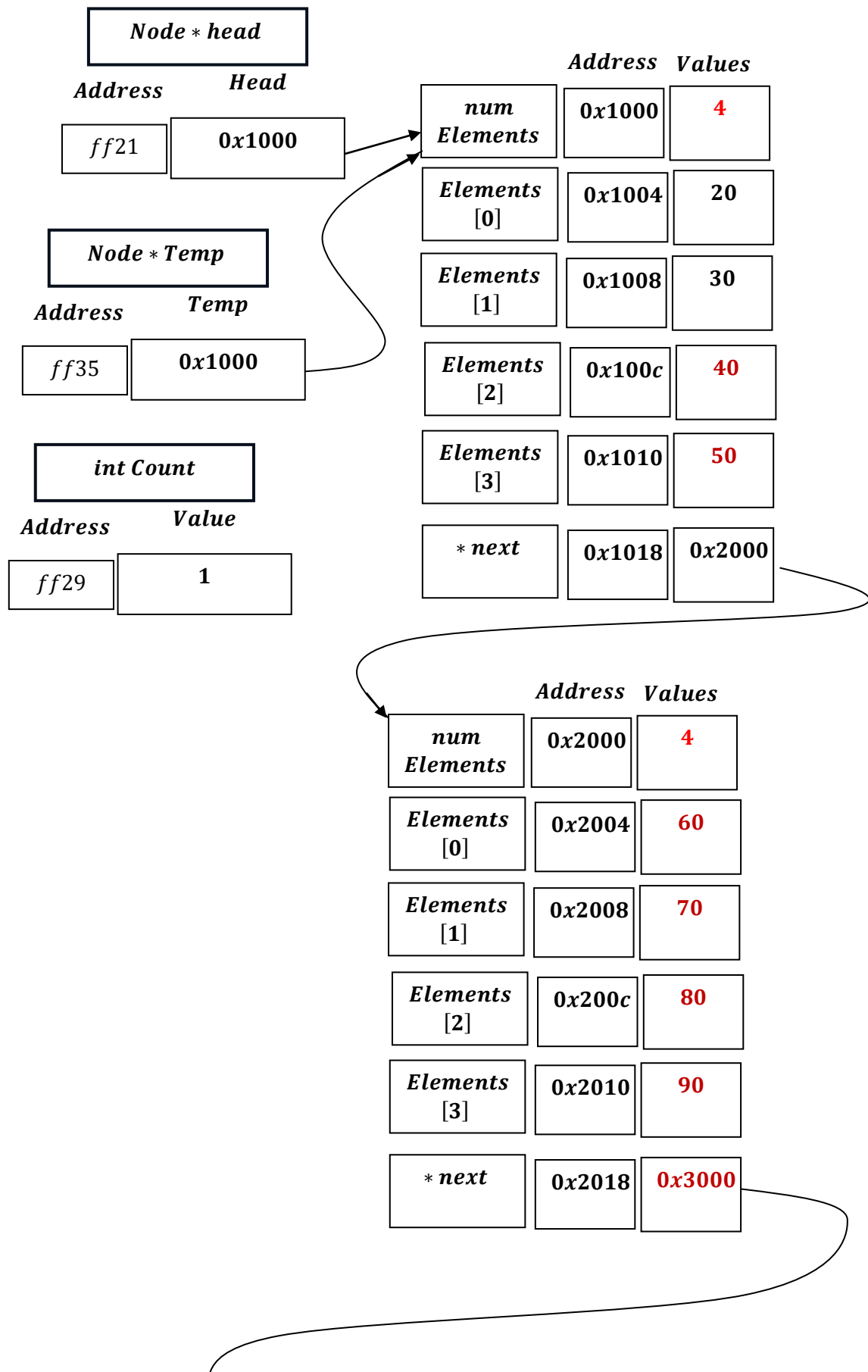


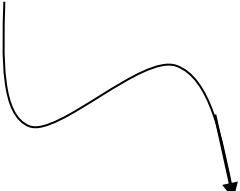
As head = NULL, it prints : `` The list is empty.``

Return void and exit from the function.

2. List is not Empty

```
struct Node* temp = head;  
int nodeCount = 1;
```





	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
<i>* next</i>	0x3018	NULL

```
cin >> value;
searchElement(value);
break;
```

Say, we are searching Element value : 60.

i. e. Function call: searchElement(value: 60).

```
while (temp != NULL) {
    for (int i = 0; i < temp->numElements; i++) {
        if (temp->elements[i] == val) {
            cout << "Element " << val << " found in Node " <<
nodeCount << " at index " << i << ".\n";
            return;
        }
    }
    temp = temp->next;
    nodeCount++;
}
```

while(*Temp*: 0x1000 ≠ *NULL*) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right] :$

for: *i* = 0; *i* < *temp* → *numElements*: 4 $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

if(*temp* → *elements*[0]: 20 = *val*: 60) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{False} \\ \textit{and if block exits} \end{array} \right]$

i = *i* + 1 = 0 + 1 = 1.

for: *i* = 1; *i* < *temp* → *numElements*: 4 $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

if(*temp* → *elements*[1]: 30 = *val*: 60) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{False} \\ \textit{and if block exits} \end{array} \right]$

i = *i* + 1 = 1 + 1 = 2.

for: *i* = 2; *i* < *temp* → *numElements*: 4 $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$

if(*temp* → *elements*[2]: 40 = *val*: 60) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{False} \\ \textit{and if block exits} \end{array} \right]$

i = *i* + 1 = 2 + 1 = 3.

$for: i = 3; i < temp \rightarrow numElements: 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$
 $if(temp \rightarrow elements[3]: 50 = val: 60) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \\ \text{and if block exits} \end{array} \right]$

$i = i + 1 = 3 + 1 = 4.$

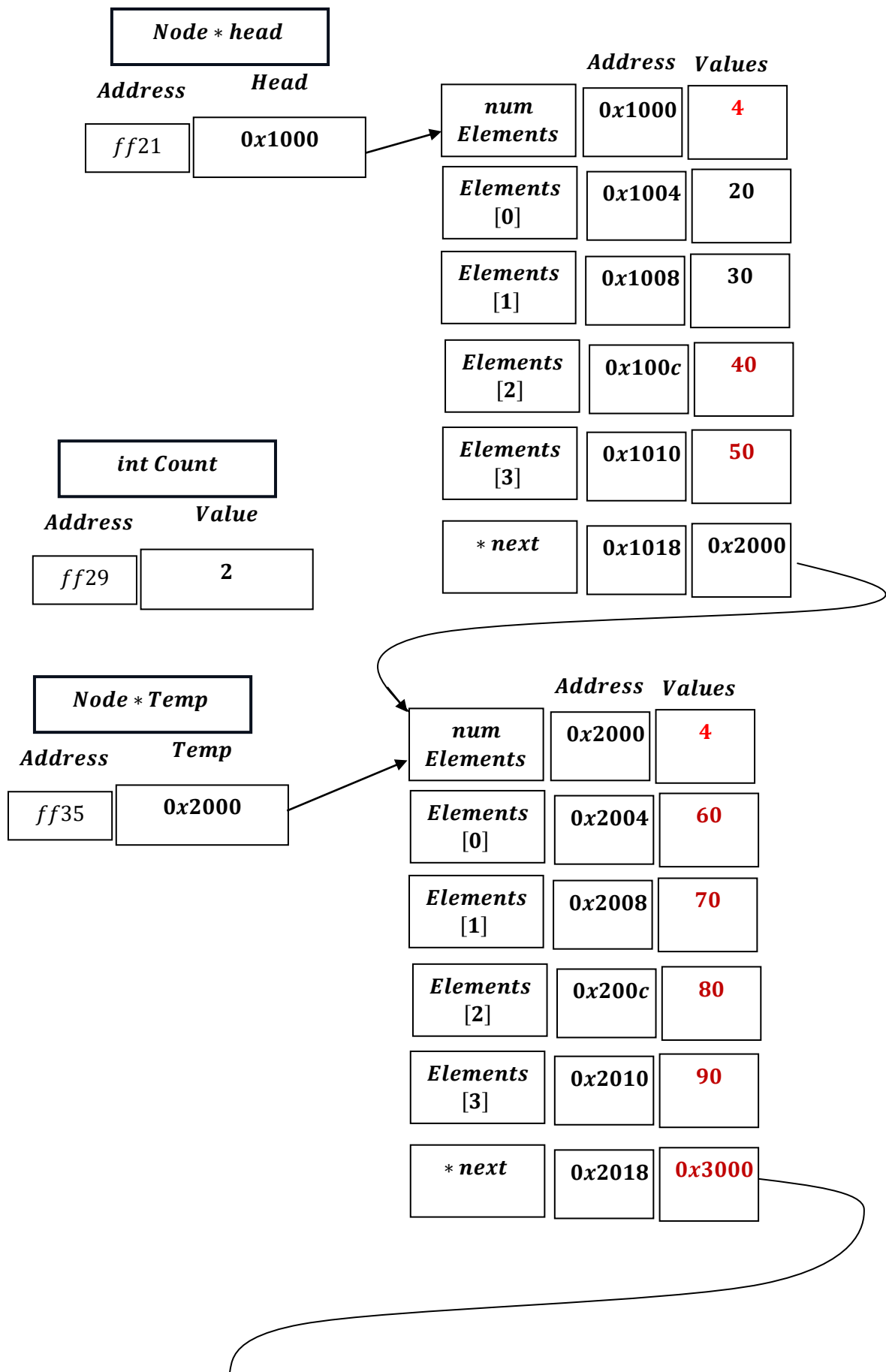
$for: i = 4; i < temp \rightarrow numElements: 4 \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{False} \\ \text{and for loop} \\ \text{Exits} \end{array} \right]$

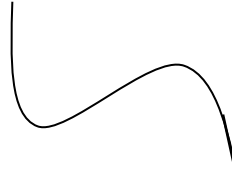
Now,

```
temp = temp->next;
nodeCount++;
```

$Temp = Temp \rightarrow next = 0x2000$

$nodeCount = nodeCount + 1 = 1 + 1 = 2.$





	Address	Values
<i>num Elements</i>	0x3000	4
<i>Elements [0]</i>	0x3004	100
<i>Elements [1]</i>	0x3008	110
<i>Elements [2]</i>	0x300C	120
<i>Elements [3]</i>	0x3010	130
<i>* next</i>	0x3018	NULL

while(Temp: 0x2000 $\neq$ NULL) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$:

for: $i = 0; i < temp \rightarrow numElements: 4$ $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$

if($temp \rightarrow elements[0]: 60 = val: 60$) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \\ \text{and if block runs} \end{array} \right]$

```
cout << "Element " << val << " found in Node " <<
nodeCount << " at index " << i << ".\n";
```

Print `` Element`` val: 60 ``found in Node`` nodeCount: 2 ``at index`` i: 0 .

```
return;
```

return will return void $\left[\begin{array}{c} \text{Return} \\ \text{Type} \end{array} \right]$ and exit from the function.

∴ Finally we have:

Element 60 found in Node 2 at index 0 .

Unrolled List

–Search Element

–Time Complexity Analysis

Program

```
// Function to search for an element
void searchElement(int val) {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {
                cout << "Element " << val << " found in Node " <<
nodeCount << " at index " << i << ".\n";
                return;
            }
        }
        temp = temp->next;
        nodeCount++;
    }

    cout << "Element " << val << " not found in the list.\n";
}

... .

    case 4:
        cout << "Enter integer value to search: ";
        cin >> value;
        searchElement(value);
        break;
```

Time Complexity Analysis

- *n*: Total number of elements.
- *N*: Total number of nodes ($N \approx \left\lceil \frac{n}{4} \right\rceil$).
- *B*: Maximum node capacity (4).

1. Best Case Time Complexity: $O(1)$

The best – case scenario occurs when the algorithm performs the minimum possible number of operations.

- **Absolute Best Case (Empty List)**: *The function checks `if (head == NULL)`. If true, it prints an error message and immediately executes a `return;` statement.*

This takes strictly constant time.

- **Not Absolute Best Case (First Element Match)**: *The list is populated, and the target value happens to be **located at index 0** of the very first node (`temp → elements[0] == val`). The inner loop matches on the first iteration, prints the success message, and exits immediately via `return;`.*

Line of Code → Absolute Best Case < Not Absolute Best Case.

- **Result**: *Because the loops barely execute or do not execute at all, the time complexity is $O(1)$.*

2. Worst Case Time Complexity: $O(n)$ (or $O(N)$)

1. Node Traversal (Outer Loop)

```
while (temp != NULL) {  
... *  
    temp = temp->next;  
    nodeCount++;  
}
```

→ Because the element is never found, the function never hits the **return;** statement inside the loop.

→ Therefore, the outer loop must visit every single node in the list from **head** until **temp** becomes **NULL**.

```
temp = temp->next;
```

- **Temp = Temp → Next**, this assignment runs, till **Temp = NULL** hence : $1 + 1 + 1 \dots N = N$ times.

```
nodeCount++;
```

- Increments the **nodeCount** till nodecounts, runs: $1 + 1 + 1 \dots N = N$ times.

→ **Cost:** Let c_n be the physical cost of hopping to the next node and incrementing the counter.

$$\text{Nodes Traversed Cost} = c_n \times N$$

$$\text{or, Exactly: } N + N = 2N, \text{ where } c_n = 2.$$

2. Element Comparisons (Inner Loop)

```
for (int i = 0; i < temp->numElements; i++) {  
    if (temp->elements[i] == val)
```

- *Inside each visited node, the inner loop executes for every active element.*
- *Because the if condition always evaluates to False, the loop never breaks early. It checks k_1 elements in Node 1, k_2 elements in Node 2, all the way up to Node N.*
 - $1 + 1 + 1 \dots n \text{ times} = 1 \times n = n.$
- *The sum of all these elements across all N nodes is exactly n.*
- **Cost:** *Let c_s be the cost of running the if comparison.*

$$\text{Elements Compared Cost} = c_s \times n$$

or, Exactly: $1 \times n = n$, where $c_s = 1$.

3. Success Execution (Inside the if block)

```
cout << "Element " << val << " found in Node " <<  
nodeCount << " at index " << i << ".\n";  
return;
```

- *Because the element is not found, the if block is entirely bypassed.*
- **Cost:** 0

$$T_{\text{worst}} = \text{Nodes Traversed Cost} + \text{Elements Compared Cost} \\ + \text{Success Execution Cost}$$

$$T_{\text{worst}} = (c_n \times N) + (c_s \times n) + 0$$

Or , exactly,

$$T_{\text{worst}} = 2N + 1n + 0$$

$$\Rightarrow T_{\text{worst}} = 2N + n$$

Applying Big – O and dropping the constants we get:

$$T_{\text{worst}} = O(N) + O(n)$$

Consolidate Proportional Variables: $N \approx \left\lceil \frac{n}{4} \right\rceil$, meaning the total number of node blocks scales directly and proportionally with the total number of elements. Because **N** and **n** grow at the exact same linear rate, **O(N)** is mathematically equivalent to **O(n)**. They absorb into each other.

Therefore, by combining the **2N** node operations with the **n** element comparisons, the final Worst Case Time Complexity definitively simplifies to **O(n)** (or **O(N)**).

Average Case Time Complexity

A. Average Element Comparisons (Inner Loop)

A. 1. Based on Loop

```
for (int i = 0; i < temp->numElements; i++) {  
    if (temp->elements[i] == val) {  
        cout << "Element " << val << " found in Node " <<  
nodeCount << " at index " << i << ".\n";  
        return;  
    }  
}
```

*If there are **n** total elements across the entire unrolled linked list,
the target element could be located at any of the **n** positions.*

- If we search for the 1st element,*

```
for (int i = 0; i < temp->numElements; i++)
```

*the inner loop comparison runs **1** time.*

```
if (temp->elements[i] == val)
```

- If we search for the 2nd element, comparison runs **2** times.*
- If we search for the **n** – th element, comparison runs **n** times.*

Note : we are doing comparisons like:

Node 1 (Elements 1 through 4)

- Target is 1st element: 1 comparison.*
- Target is 2nd element: 2 comparisons.*
- Target is 3rd element: 3 comparisons.*
- Target is 4th element: 4 comparisons.*

Node 2 (Elements 5 through 8)

- **Target is 5th element: 4 comparisons (from Node 1) + 1 comparison (in Node 2) = 5 total comparisons.**
- **Target is 6th element: 4 + 2 = 6 total comparisons.**
- **Target is 7th element: 4 + 3 = 7 total comparisons.**
- **Target is 8th element: 4 + 4 = 8 total comparisons.**

Hence, here `n` elements represents total `n` elements of `N` node blocks.

Assuming a uniform distribution, the probability of the target element

being at any specific position is exactly $\frac{1}{n}$.

If there is n number of elements, we can search the target at n positions.

- **For position 1, the probability of searching target is $\frac{1}{n}$.**
- **For position 2, the probability of searching target is $\frac{1}{n}$.**
- **For position n , the probability of searching target is $\frac{1}{n}$.**

Hence probability of searching target will always remain $\frac{1}{n}$.

To find the average time complexity, we multiply the probability of the element's location by the number of iterations required to find it:

$$\text{Average} = \left(\frac{1}{n} \times 1\right) + \left(\frac{1}{n} \times 2\right) + \dots + \left(\frac{1}{n} \times n\right)$$

$$\text{Average} = \frac{1}{n} \times \{1 + 2 + 3 + \dots + n\}$$

Using the sum of an arithmetic progression formula $\left(\sum_{i=1}^n i\right)$:

$$Average = \frac{1}{n} \times \left\{ \frac{n(n+1)}{2} \right\}$$

$$Average = \frac{n+1}{2}$$

Alternate simplest approach :

A.2. Based on Position

*If there are **n** total elements in the data structure, they occupy absolute positions from **1 to n**.*

Because we assume the target element could be anywhere, every single position has the exact same, equal chance of being the target:

- *For position 1, the probability of searching is $\frac{1}{n}$.*
- *For position 2, the probability of searching is $\frac{1}{n}$.*
- *For position n, the probability of searching is $\frac{1}{n}$.*

*Hence, the probability always remains exactly $\frac{1}{n}$ for any position from **1 to n**.*

$$Average = Probability \times (Summation\ of\ number\ of\ Position)$$

By substituting our values into the equation:

$$Average = \frac{1}{n} \times \left(\sum_{i=1}^n i \right)$$

*Using the standard mathematical formula for the sum of the first **n** natural numbers:*

$$Average = \frac{1}{n} \times \left(\frac{n(n+1)}{2} \right)$$

The **n** in the numerator and the **n** in the denominator perfectly cancel each other out:

$$\text{Average} = \frac{n + 1}{2}$$

Cost: Let **c_s** be the cost of checking one element (the **if** comparison).
The total average comparison cost is **c_s** × $\left(\frac{n + 1}{2}\right)$.

B. Average Node Traversals (Outer Loop)

The list consists of **N** total node blocks. The outer **while** (**temp** ≠ **NULL**) loop hops from node to node to locate the block containing the target element.

- If the element is in the 1st node, the loop runs 1 time.
- If the element is in the 2nd node, the loop runs 2 times.
- If the element is in the **N** – th node, the loop runs **N** times.

Because the elements are distributed evenly, the probability of the element being in any specific node is $\frac{1}{N}$.

We calculate the average using the exact same statistical logic as the elements:

$$\text{Average Traversals} = \left(\frac{1}{N} \times 1\right) + \left(\frac{1}{N} \times 2\right) + \dots + \left(\frac{1}{N} \times N\right)$$

$$\text{Average Traversals} = \frac{1}{N} \times \sum_{i=1}^N i$$

$$\text{Average Traversals} = \frac{1}{N} \times \frac{N(N + 1)}{2}$$

$$\text{Average Traversals} = \frac{N + 1}{2}$$

Cost: Let c_n be the cost of hopping from one node to the next. The total average traversal cost is $c_n \times \left(\frac{N+1}{2}\right)$.

To find the total average time (T_{avg}), we add the traversal cost and comparison cost together:

$$T_{avg} = c_n \times \left(\frac{N+1}{2}\right) + c_s \times \left(\frac{n+1}{2}\right)$$

To express this in standard Big – O notation, we strip away all constant multipliers (like $\frac{1}{2}$), hardware costs (c_n, c_s), and lower – order additive constants:

$$T_{avg} = O(N) + O(n)$$

Because the total number of nodes is $N \approx \left\lceil \frac{n}{4} \right\rceil$, N is directly proportional to n . Traversing N nodes takes the same linear time as searching n elements, making $O(N)$ mathematically equivalent to $O(n)$.

Because $O(n)$ is the term with the highest growth rate, it absorbs all constants. Therefore, the final Average Case Time Complexity for searching the list is definitively $O(n)$ (or $O(N)$).

Unrolled List
–Search Element
–Space Complexity Analysis

Program:

```
// Function to search for an element
void searchElement(int val) {
    if (head == NULL) {
        cout << "The list is empty.\n";
        return;
    }

    struct Node* temp = head;
    int nodeCount = 1;

    while (temp != NULL) {
        for (int i = 0; i < temp->numElements; i++) {
            if (temp->elements[i] == val) {
                cout << "Element " << val << " found in Node " <<
nodeCount << " at index " << i << ".\n";
                return;
            }
        }
        temp = temp->next;
        nodeCount++;
    }

    cout << "Element " << val << " not found in the list.\n";
}

....

    case 4:
        cout << "Enter integer value to search: ";
        cin >> value;
        searchElement(value);
        break;
```

Space Complexity Analysis

1. Input Data Space

The input space measures the memory required to store the data that the algorithm interacts with.

1. A. Parametric Input Space: $O(1)$

This refers exclusively to the memory needed for the arguments passed directly into the function signature.

- *The function signature is: `void searchElement(int val)`.*
- *It accepts exactly one parameter: a single integer `val`.*
- *Because a single integer requires a fixed, constant amount of memory (**typically 4 bytes**) regardless of how large the list grows, the memory footprint of the parameters never changes.*
- *Result: The parametric input space is strictly $O(1)$ (**Constant Space**).*

1. B. Full Data Holistic Space: $O(N)$ or $O(n)$

This measures the memory consumed by the actual data structure the algorithm is searching through.

- *The function searches a globally defined unrolled linked list accessed via the **head** pointer.*
- *To store **n** total elements, the structure requires $N \approx \left\lceil \frac{n}{4} \right\rceil$ nodes.*
- *Therefore, the total memory consumed by the data structure physically scales linearly with the number of elements.*

- **Result:** The holistic input space is $O(N)$ (which is mathematically equivalent to $O(n)$ i. e. $[O(N) \equiv O(n)]$).

2. Auxiliary Space Complexity: $O(1)$

Auxiliary Space measures the temporary, extra memory that the algorithm uses exclusively to perform its task, entirely separate from the input data.

*Let's trace every local variable created inside the **searchElement** block:*

- One traversal pointer: **`struct Node * temp = head;`**.
- One node tracker: **`int nodeCount = 1;`**.
- One loop iterator: **`int i = 0;`** (used inside the for loop).

Key Observations:

1. **No Dynamic Allocation:** The function only reads data. It never uses **`malloc()`** or **`new`** to create additional nodes or arrays.
2. **No Recursion:** The function relies on simple iterative loops (while and for), meaning the Call Stack does not grow.

The Call Stack depth remains at exactly 1.

Result: Because the extra memory required by the function is fixed and does not change based on the size of the list, i. e. independent of the input size **`n`**,

*The Auxiliary Space Complexity is strictly **$O(1)$** (Constant Space).*

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the list(N) scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The for loop is governed by the condition $i < N$, meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

*Inside the loop, the **malloc** function is called during every single iteration.*

***malloc** physically claims a brand – new*

block of memory from the computer's heap.

3. Total Memory Consumed

*If **sizeof(struct Node)** takes a fixed constant amount of memory (for example, 24 bytes), the total extra memory generated by this loop is:*

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

*In Big – O notation, we drop the exact hardware constants (like 24 bytes) because we only care about how the memory requirements grow as N gets larger. Because the memory requested scales linearly with the variable N, the complexity is **$O(N)$** .*

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N *  
sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

For Academic Answer = Auxiliary Space = $O(1)$.

*"As we often cannot control the size of the input data, but we
can control how much extra memory of our algorithm
creates to process it."*

Total Space Complexity or Total memory footprint:
= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space

$$= (O(1) + O(N)) + O(1)$$

$$= O(N).$$

or,

$$(O(1) + O(n)) + O(1)$$

$$= O(n).$$

Key Notation Conventions

- **START**: Represents the global **head** pointer.
- **LINK[PTR]**: Represents the **next** pointer of a specific node block.
- **numElements[PTR] / elements[PTR][i]**: Represents accessing the attributes inside the specific node.

Algorithm: UnrolledSEARCHElem(val)

1. [CHECK IF LIST IS EMPTY]

- **IF START = NULL, Then:**
 - **Write: "The list is empty."**
 - **RETURN**
- **[End of If structure.]**

2. [INITIALIZE TRAVERSAL VARIABLES]

- **SET TEMP_PTR := START**
- **SET NODECOUNT := 1**

3. [OUTER LOOP: NODE TRAVERSAL]

- **REPEAT WHILE TEMP_PTR ≠ NULL:**

4. [INNER LOOP: ELEMENT SEARCH]

- **SET i := 0**
- **REPEAT WHILE i < numElements[TEMP_PTR]:**

5. [CHECK FOR MATCH]

- **IF elements[TEMP_PTR][i] = val, Then:**
 - **Write: "Element ", val, " found in Node ", NODECOUNT, " at index ", i, "."**
 - **RETURN (Exit the algorithm immediately upon success)**
- **[End of Match If structure.]**
- **SET i := i + 1 (Move to the next element in the current node)**
- **[End of Inner Search Loop]**

6. [MOVE TO NEXT NODE]

- **SET TEMP_PTR := LINK[TEMP_PTR] (Advance the pointer to the next node block)**
- **SET NODECOUNT := NODECOUNT + 1 (Increment the node tracker)**
- **[End of Outer Node Traversal Loop]**

7. [ELEMENT NOT FOUND]

*(If the outer loop finishes without
executing the inner return statement)*

- *Write: "Element ", val, " not found in the list."*

8. END OF ALGORITHM

Unrolled List
–Destroy List
–Program Analysis

Program

```
void destroyList() {
    if (head == NULL) {
        cout << "The list is already empty.\n";
        return;
    }

    struct Node* temp = head;
    while (temp != NULL) {
        struct Node* nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }

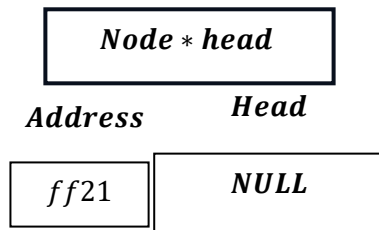
    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";
}

...
    case 5:
        destroyList();
        break;
```

Program Analysis

1. List is Empty

```
if (head == NULL) {  
    cout << "The list is empty.\n";  
    return;  
}
```

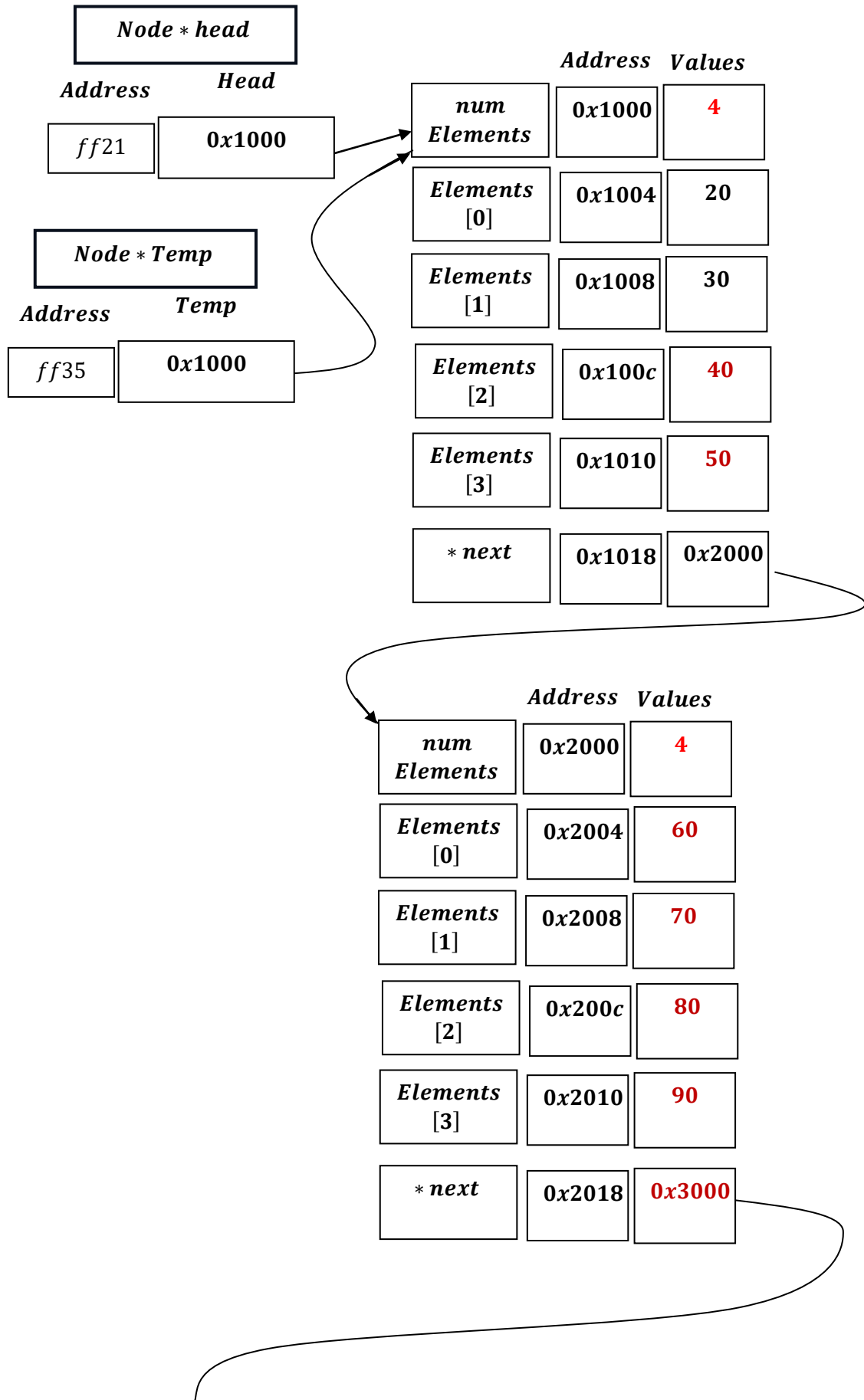


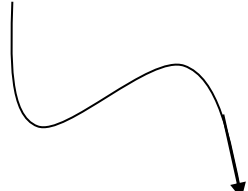
As head = NULL, it prints : `` The list is empty.``

Return void and exit from the function.

2. List is not Empty

```
struct Node* temp = head;
```





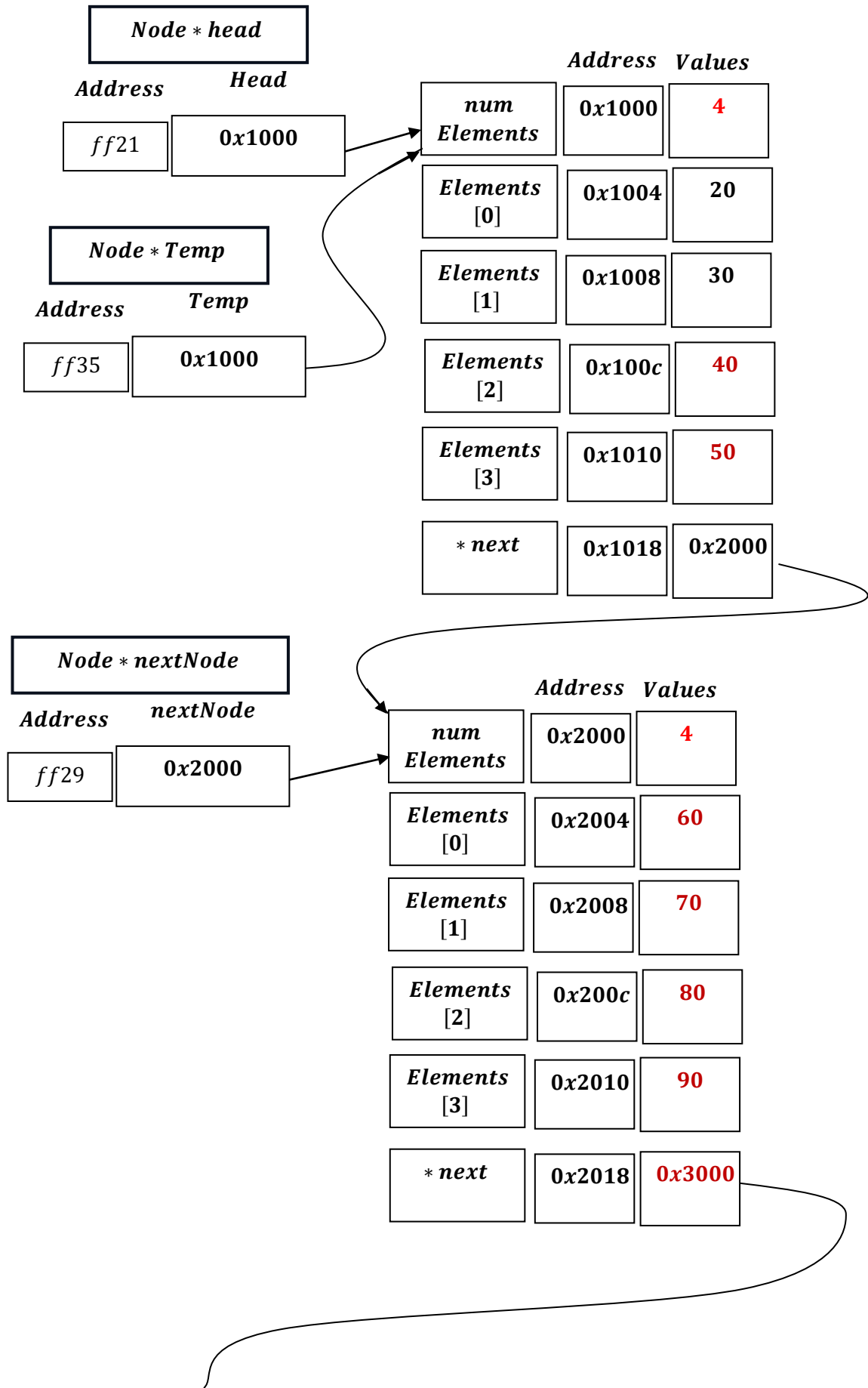
	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
<i>* next</i>	0x3018	NULL

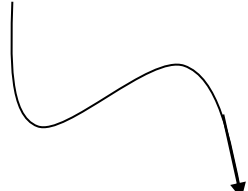
```

while (temp != NULL) {
    struct Node* nextNode = temp->next;
    free(temp);
    temp = nextNode;
}

```

While(*temp*: 0x1000 $\neq$ NULL) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$
*struct Node * nextNode = temp* $\rightarrow$ *next*: 0x2000



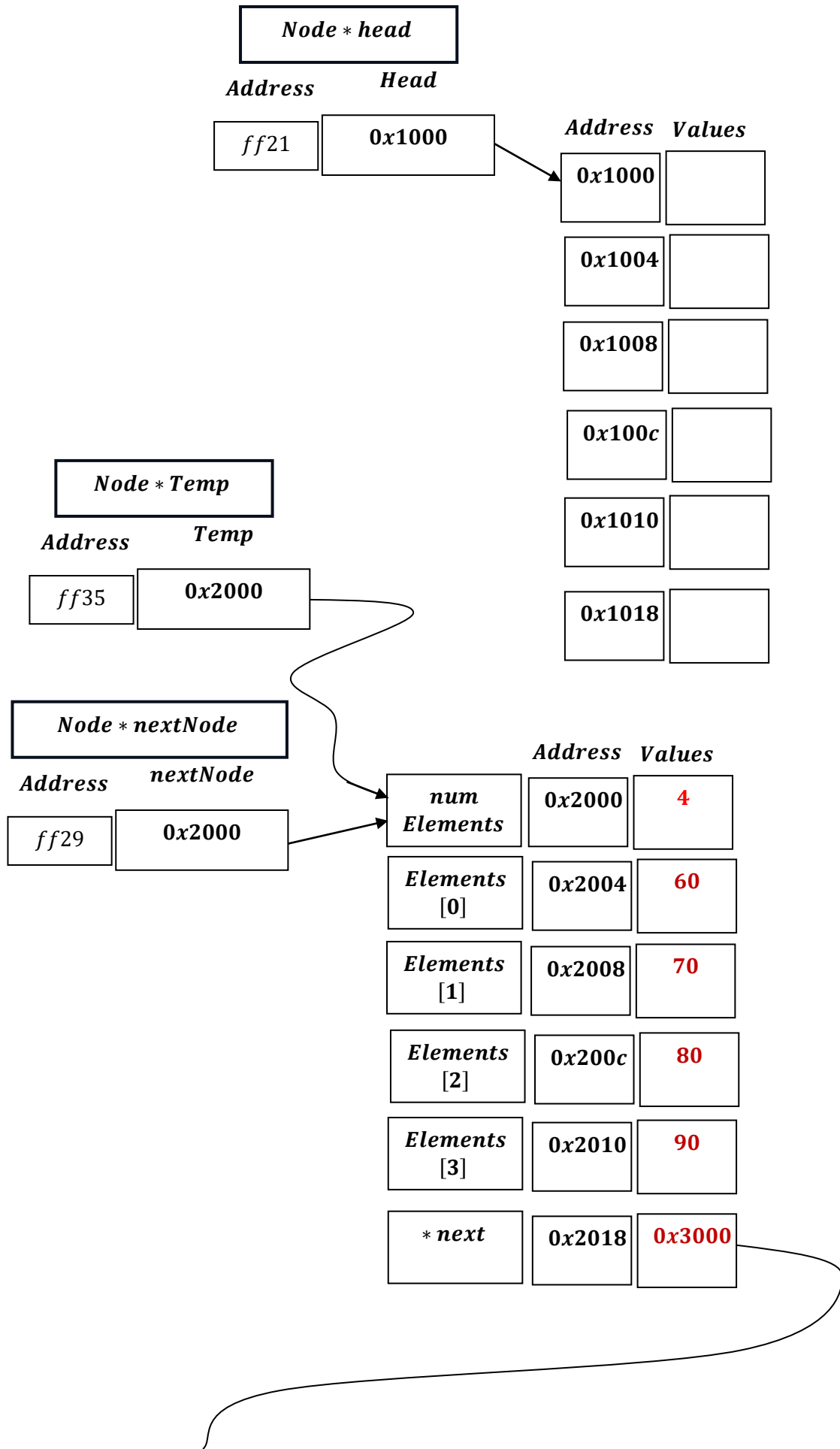


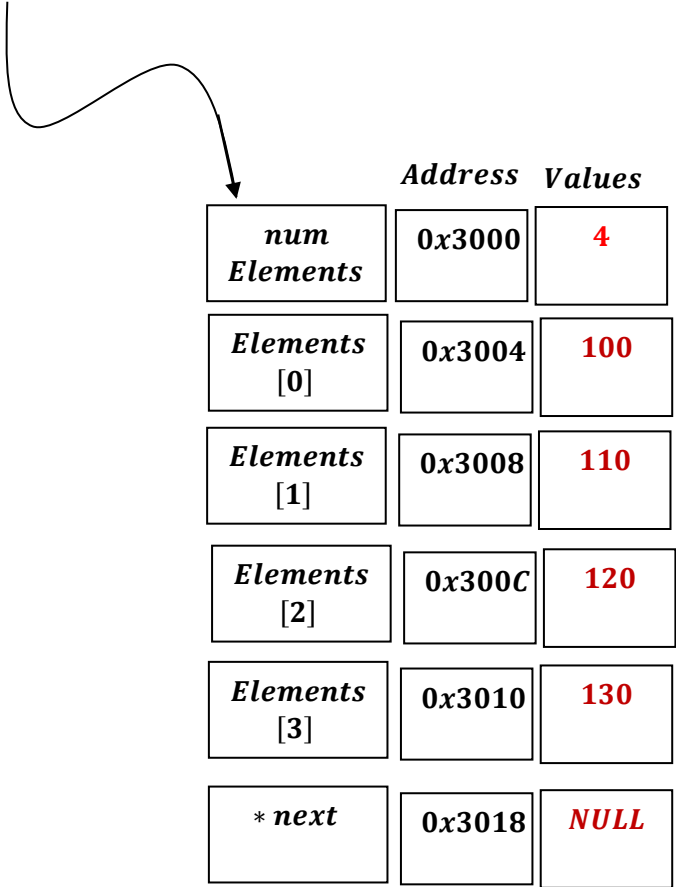
	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements [0]</i>	0x3004	100
<i>Elements [1]</i>	0x3008	110
<i>Elements [2]</i>	0x300C	120
<i>Elements [3]</i>	0x3010	130
<i>* next</i>	0x3018	NULL

That is:

While(*temp*: 0x1000 $\neq$ NULL) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$
 struct Node * *nextNode* = *temp* $\rightarrow$ *next*: 0x2000
 free(*temp*: 0x1000)
 temp = *nextNode* = 0x2000

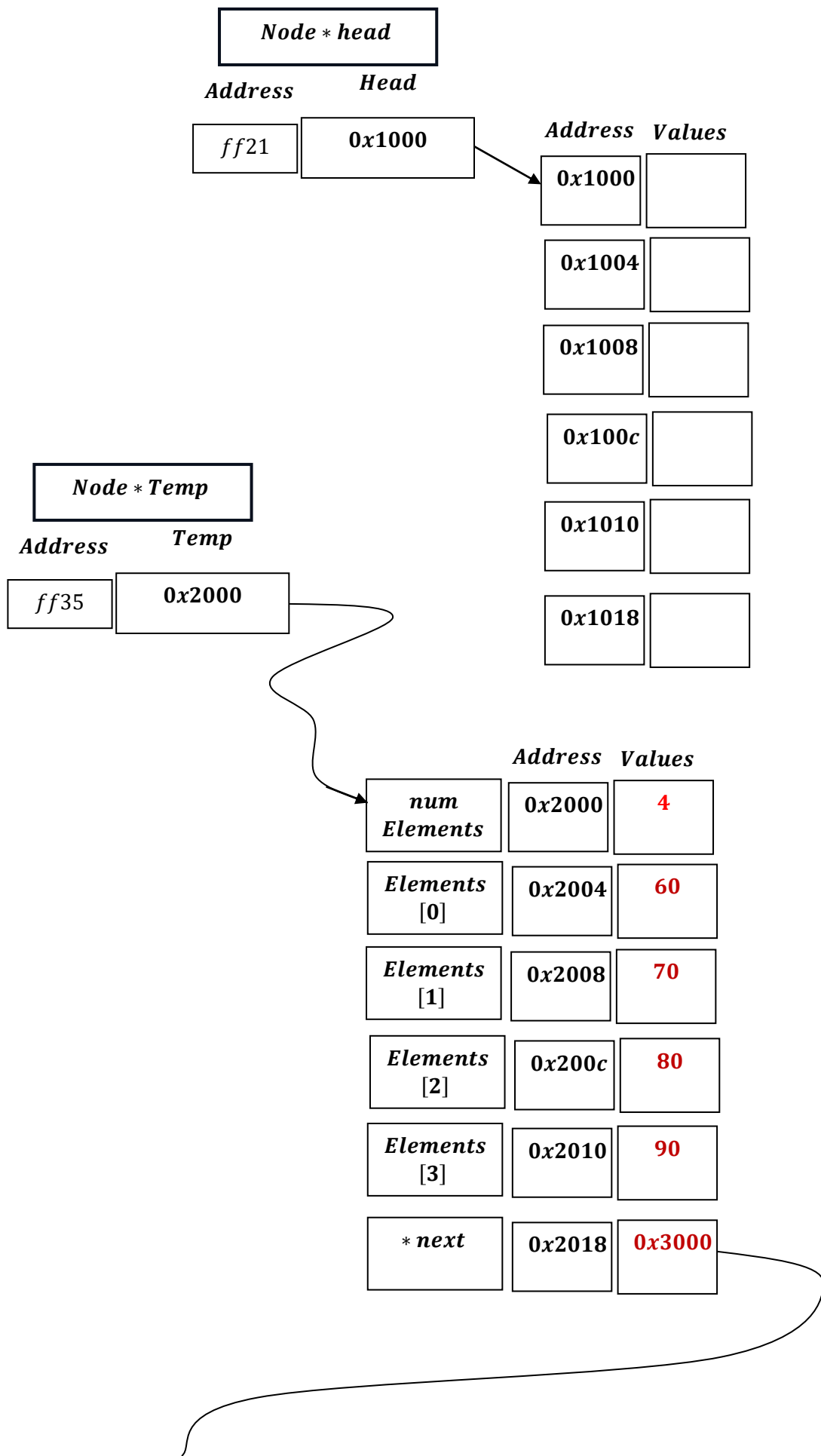
Note: *free*(*temp*: 0x1000), this will freeing memory : 0x1000
 starting from 0x1000 and its associated memory allocations
 ended till : 0x1018 are freed all at once .

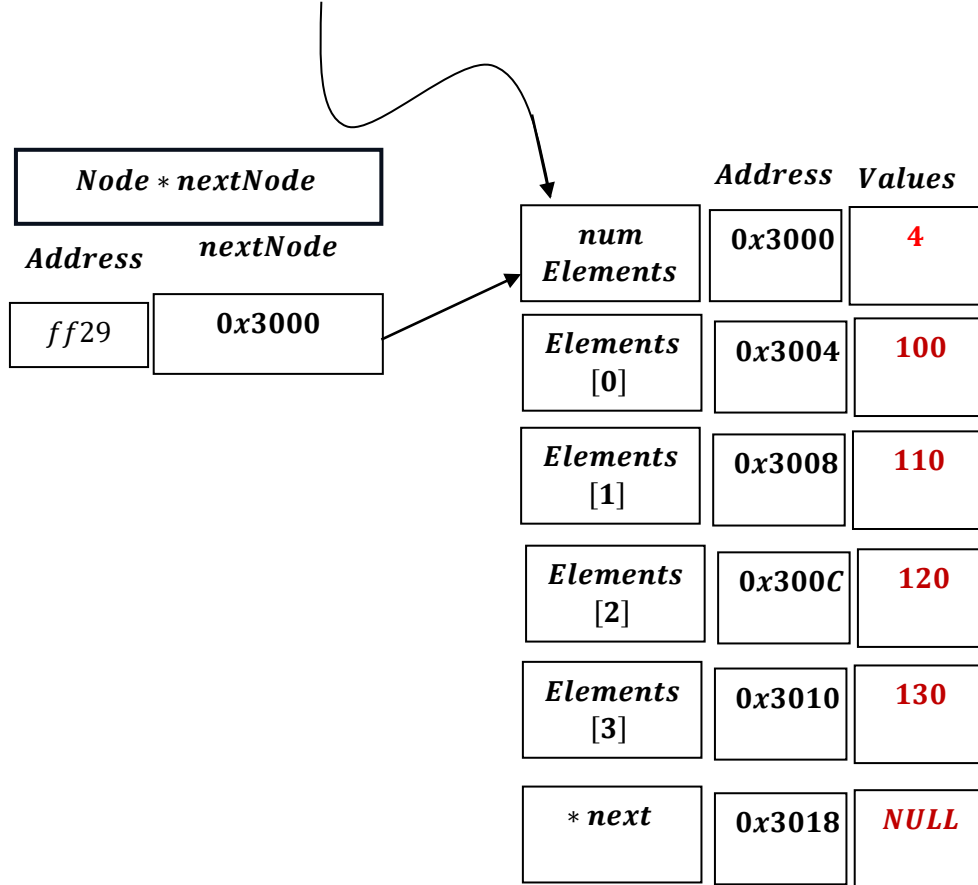




	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
<i>* next</i>	0x3018	NULL

$While(temp: 0x2000 \neq NULL) \left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{True} \end{array} \right]$
*struct Node * nextNode = temp → next: 0x3000*



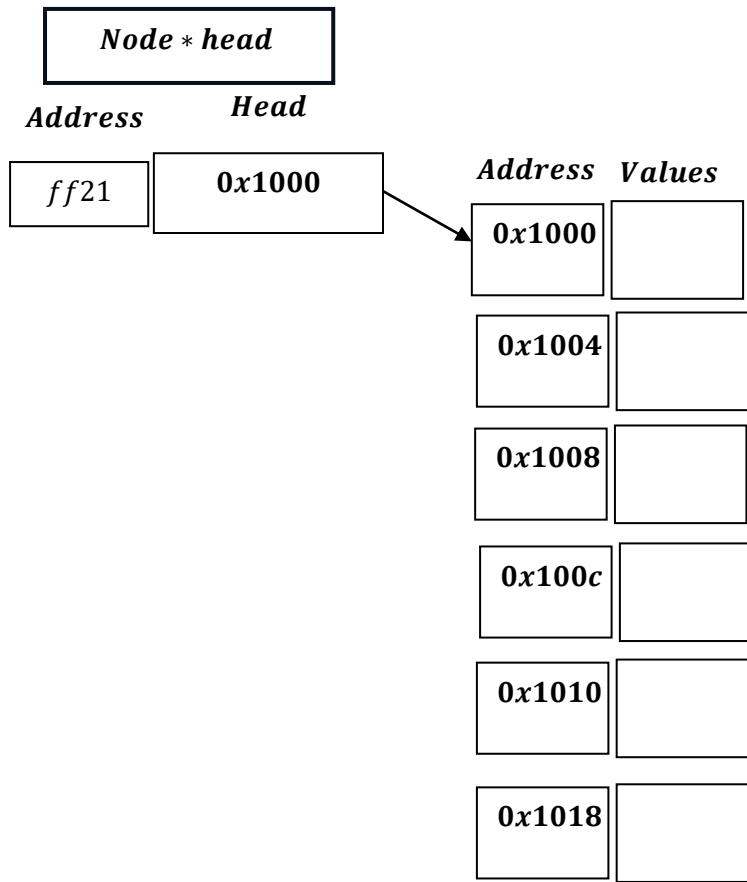


That is:

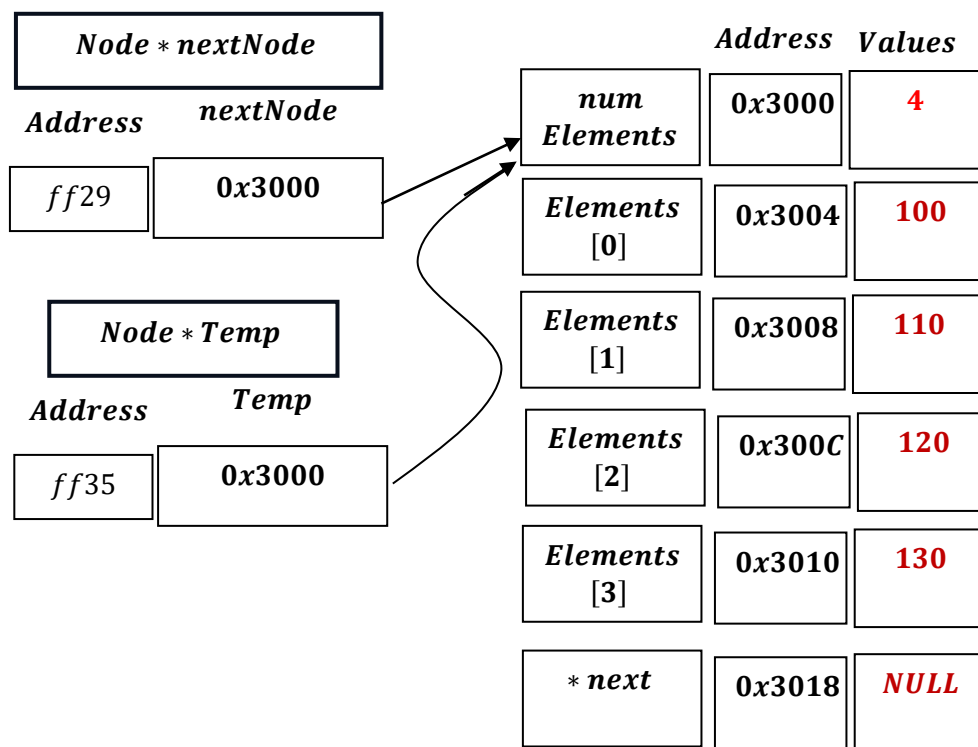
```

While(temp: 0x2000 ≠ NULL)  $\left[ \begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$ 
    struct Node * nextNode = temp → next: 0x3000
    free(temp: 0x2000)
    temp = nextNode = 0x3000
  
```

*Note: **free(temp: 0x2000)**, this will freeing memory : 0x2000 starting from 0x2000 and its associated memory allocations ended till : 0x2018 are freed all at once .*

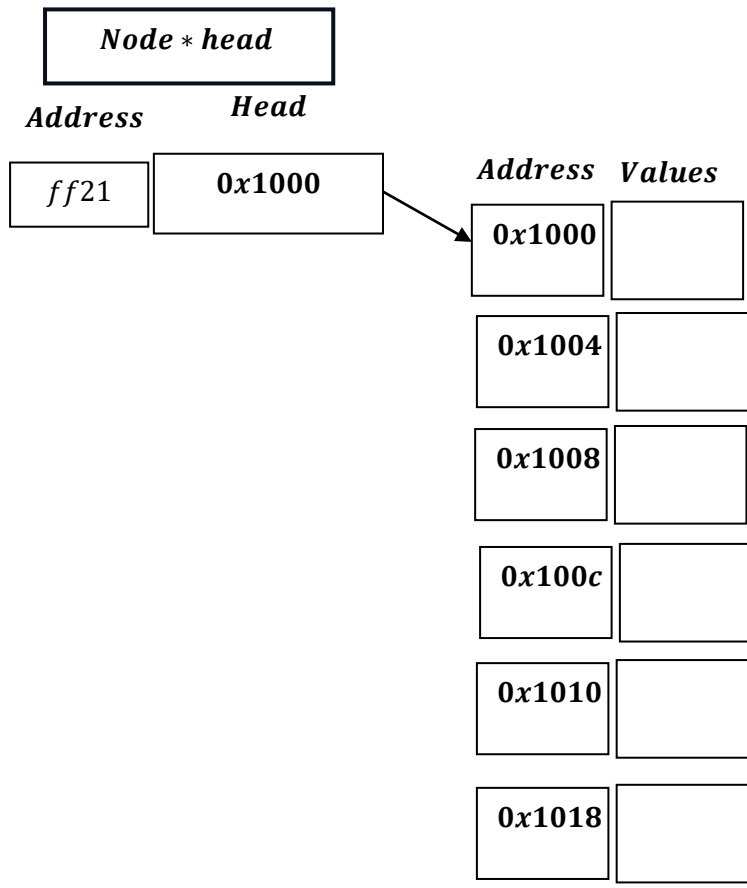


Address	Values
0x2000	
0x2004	
0x2008	
0x200c	
0x2010	
0x2018	

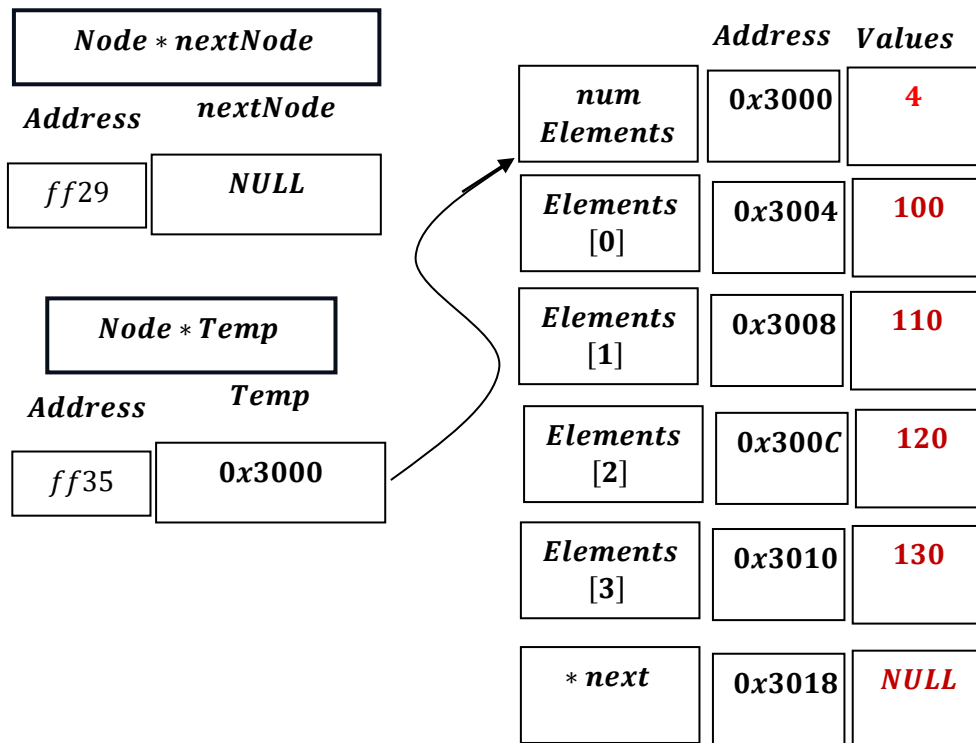


$$\text{While}(\text{temp: } 0x3000 \neq \text{NULL}) \left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$$

$$\text{struct Node * nextNode} = \text{temp} \rightarrow \text{next: NULL}$$



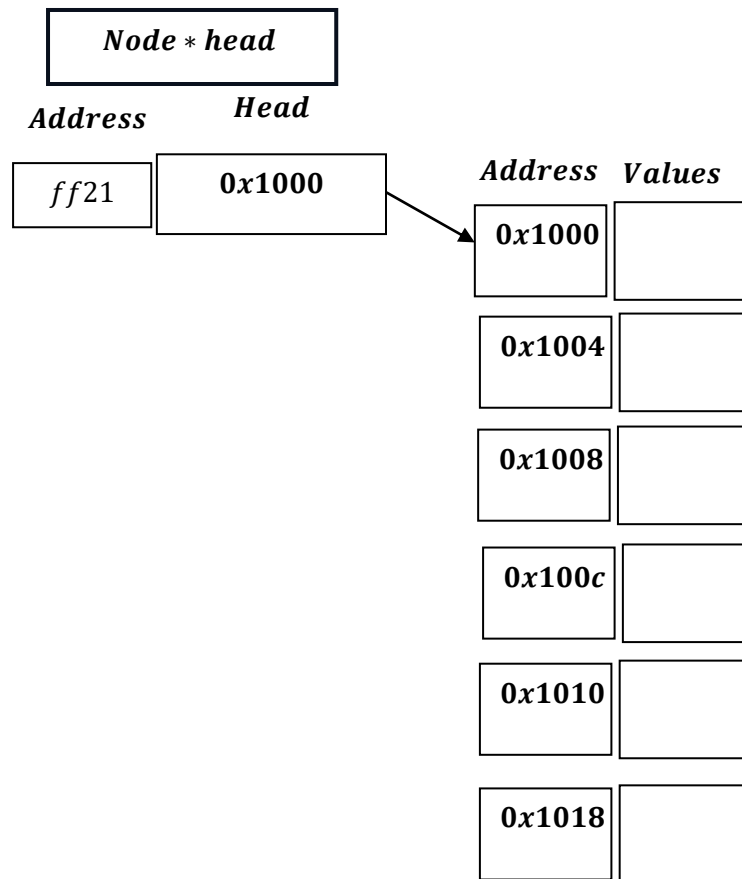
Address	Values
0x2000	
0x2004	
0x2008	
0x200c	
0x2010	
0x2018	



That is:

*While(temp: 0x3000 $\neq$ NULL) $\left[\begin{array}{c} \text{Condition} \\ \text{is} \\ \text{True} \end{array} \right]$
 *struct Node * nextNode = temp $\rightarrow$ next: NULL*
 free(temp: 0x3000)
 *temp = nextNode = NULL**

*Note: **free(temp: 0x3000)**, this will freeing memory : **0x3000** starting from **0x3000** and its associated memory allocations ended till : **0x3018** are freed all at once .*



Address	Values
0x2000	
0x2004	
0x2008	
0x200c	
0x2010	
0x2018	

<i>Node * nextNode</i>

Address nextNode

<i>ff29</i>	<i>NULL</i>
-------------	-------------

<i>Node * Temp</i>

Address Temp

<i>ff35</i>	<i>NULL</i>
-------------	-------------

Address Values

<i>0x3000</i>	
---------------	--

<i>0x3004</i>	
---------------	--

<i>0x3008</i>	
---------------	--

<i>0x300C</i>	
---------------	--

<i>0x3010</i>	
---------------	--

<i>0x3018</i>	
---------------	--

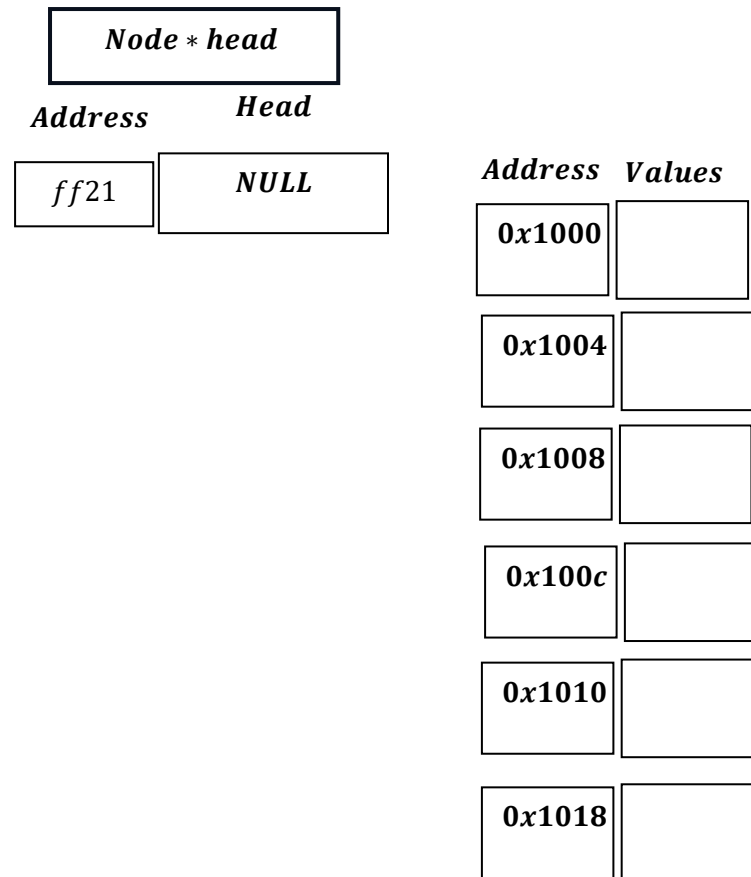
Now , While(temp: NULL $\neq$ NULL) $\left[\begin{array}{c} \textit{Condition} \\ \textit{is} \\ \textit{False} \end{array} \right]$

As, condition is false – while loop exits.

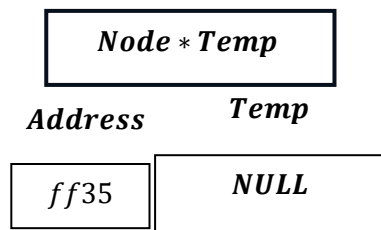
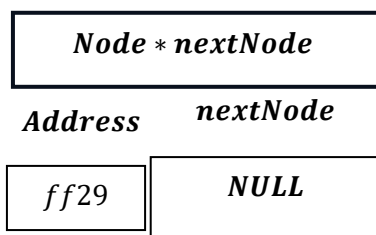
```

    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";

```



Address	Values
0x2000	
0x2004	
0x2008	
0x200c	
0x2010	
0x2018	



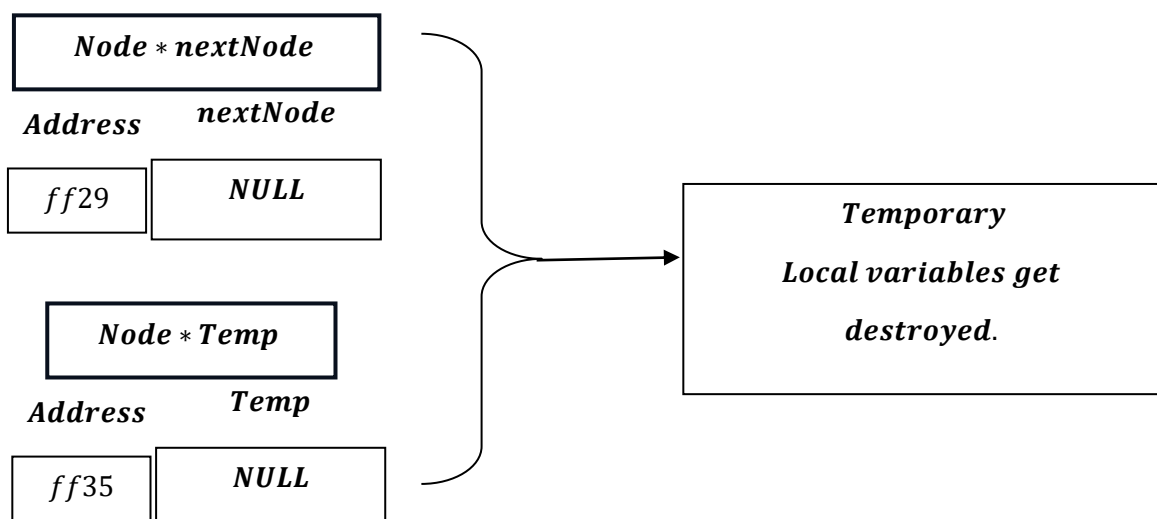
Address	Values
0x3000	
0x3004	
0x3008	
0x300C	
0x3010	
0x3018	

And now it will print:

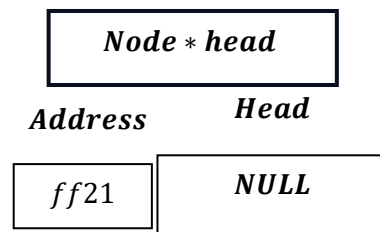
```
cout << "The entire list has been destroyed and memory
freed.\n";
```

Print: ``The entire list has been destroyed and memory freed.``

As , functions ends :



And we are left with:



That is list is empty.

Unrolled List
–Destroy List
–Time Complexity Analysis

Program

```
void destroyList() {
    if (head == NULL) {
        cout << "The list is already empty.\n";
        return;
    }

    struct Node* temp = head;
    while (temp != NULL) {
        struct Node* nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }

    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";
}

...

    case 5:
        destroyList();
        break;
```

Time Complexity Analysis

- ***n***: Total number of active elements.
- ***N***: Total number of node blocks ($N \approx \left\lceil \frac{n}{4} \right\rceil$).
- ***B***: Node capacity (***B*** = 4).

1. Best Case Time Complexity: ***O*(1)**

The absolute best – case scenario occurs when the function is called on a list that has no nodes.

- **The Condition**: The function evaluates ***if (head == NULL)***.
- **The Execution**: Because ***head*** points to ***NULL***, the condition is true. The program prints a message stating the list is empty and immediately executes a ***return***; statement, exiting the function.
- **Result**: This single check and return process requires strictly constant time, making the Best Case ***O*(1)**.

2. Worst & Average Case Time Complexity: ***O*(N)** (or ***O*(n)**)

If the list is populated, the ***destroyList*** function must completely wipe the data structure from memory. Because it must traverse and delete every single node to prevent memory leaks, there is no "early exit."

Thus, the worst – case and average – case times are mathematically identical.

Node Traversal (The while Loop)

```
while (temp != NULL)
```

- The function uses a `while (temp != NULL)` loop to traverse the list.
- If the list has N nodes (like the 3 like in address: `0x1000`, `0x2000`, and `0x3000`), the loop evaluates to **True** exactly N times.

```
struct Node* nextNode = temp->next;  
free(temp);  
temp = nextNode;
```

1. `struct Node * nextNode = temp → next;` (Temporarily saves the address of the next node block).

It runs : $1 + 1 + \dots N \text{ times} = 1 \times N = N$.

2. `free(temp);` (Deallocates the current node)

It runs : $1 + 1 + \dots N \text{ times} = 1 \times N = N$.

3. `temp = nextNode;` (Moves the pointer forward).

It runs : $1 + 1 + \dots N \text{ times} = 1 \times N = N$.

That is total time complexity of the loop : $N + N + N = 3N$

`free(temp)`: it frees the starting address (e.g., `0x1000`) and all of its associated memory allocations (ending at `0x1018`) all at once. Because the entire array of elements and the node metadata are destroyed simultaneously in one massive chunk, the algorithm does not need to iterate through the individual **B** elements inside.

C. Post – Loop Operations

```
    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";
}
```

- *Once the loop exits (when **temp** becomes **NULL**), the function sets **head = NULL**;*
- *It then prints a final confirmation message ("The entire list has been destroyed and memory freed. ").*
- *Finally, the temporary local variables (**nextNode** and **temp**) are automatically destroyed as the function ends.*
- *Cost: $O(1)$.*

$$T = \text{Total Loop Cost} + \text{Post-Loop Cost}$$

$$T = 3N + O(1)$$

Applying Big – O , we get:

$$\Rightarrow T = 3 \times O(N) + O(1)$$

$$\Rightarrow T = O(N) + O(1)$$

$$\Rightarrow T = O(N)$$

Because the total number of nodes N scales directly and linearly with the total number of elements n (where $N \approx \left\lceil \frac{n}{4} \right\rceil$), traversing N nodes takes the exact same linear time as processing n elements.

Therefore, $O(N)$ is mathematically equivalent to $O(n)$, making the final Time Complexity definitively $O(N)$ (or $O(n)$).

Unrolled List

–Destroy List

–Space Complexity Analysis and Algorithm

Program:

```
void destroyList() {
    if (head == NULL) {
        cout << "The list is already empty.\n";
        return;
    }

    struct Node* temp = head;
    while (temp != NULL) {
        struct Node* nextNode = temp->next;
        free(temp);
        temp = nextNode;
    }

    head = NULL;
    cout << "The entire list has been destroyed and memory
freed.\n";
}
...
    case 5:
        destroyList();
        break;
```

Space Complexity Analysis

1. Input Data Space

The input space measures the memory required to hold the data that the algorithm interacts with before it begins processing.

A. Parametric Input Space: $O(1)$

This refers exclusively to the memory needed for the arguments passed directly into the function signature.

- *The function signature is **void destroyList()**.*
- *It accepts absolutely zero parameters.*
- *Because no values are passed into the function, it requires zero extra memory for arguments.*
- **Result:** *The parametric input space is strictly $O(1)$ (Constant Space).*

B. Full Data Holistic Space: $O(N)$ or $O(n)$

This measures the memory consumed by the actual data structure the algorithm operates on.

- *The function interacts with the global unrolled linked list consisting of **N** node blocks to store **n** active elements.*
- *Even though the goal of the function is to delete this memory, at the exact moment the function is called, the data structure fully exists in the computer's RAM.*
- *Because the total memory occupied by the list scales linearly with the number of elements, the holistic space requirement at the start of the function is proportional to the size of the list.*
- **Result:** *The holistic input space is $O(N)$ (which is mathematically equivalent to $O(n)$).*

2. Auxiliary Space Complexity: $O(1)$

Auxiliary Space measures the temporary, extra memory that the algorithm uses exclusively to perform its task, entirely separate from the input data.

Let's trace every single local variable created inside the `destroyList` block:

- *One traversal pointer: `struct Node * temp = head;` declared at the beginning of the function.*
- *One temporary holding pointer: `struct Node * nextNode = temp → next;` declared inside the `while` loop to safely hold the next address before the current one is deleted.*

Key Observations:

1. **Memory Destruction, Not Allocation:** *The function explicitly uses `free(temp)` to deallocate memory. It never uses `malloc` or `new` to create new structures.*
 2. **No Recursion:** *The function uses a simple, iterative `while` loop.*
- The Call Stack does not grow; it remains at a depth of 1.*
-

Extra:

When, dynamically memory allocation to cause $O(N)$ auxiliary space, the amount of memory requested must scale up as the size of the list(N) scales up.

1. Looping N times to allocate N nodes.

```
// If a function did this, it would take  $O(N)$  auxiliary space
for (int i = 0; i < N; i++) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct
Node));
    // ... link the node ...
}
```

1. The Loop Execution

The for loop is governed by the condition $i < N$, meaning it will execute exactly N times.

2. The Dynamic Memory Allocation

*Inside the loop, the **malloc** function is called during every single iteration.*

***malloc** physically claims a brand – new*

block of memory from the computer's heap.

3. Total Memory Consumed

*If **sizeof(struct Node)** takes a fixed constant amount of memory (for example, 24 bytes), the total extra memory generated by this loop is:*

$$\text{Total Memory} = N \times 24 \text{ bytes.}$$

In Big – O notation, we drop the exact hardware constants (like 24 bytes) because we only care about how the memory requirements grow as N gets larger. Because the memory requested scales linearly with the variable N, the complexity is $O(N)$.

2. Allocating a massive block of memory in a single line:

```
struct Node* array_of_nodes = (struct Node*)malloc(N *  
sizeof(struct Node));
```

*Even though malloc is only called one time here,
the total size of the memory requested is multiplied by N.
If N gets larger, the memory block gets larger.*

$$\text{Space Complexity} = O(N)$$

For Academic Answer = Auxiliary Space = $O(1)$.

*"As we often cannot control the size of the input data, but we
can control how much extra memory of our algorithm
creates to process it."*

Total Space Complexity or Total memory footprint:
*= Input Space (Parametric Input Space + Full Data Holistic Input Space)
+ Auxiliary Space*

$$= (O(1) + O(N)) + O(1)$$

$$= O(N).$$

or,

$$(O(1) + O(n)) + O(1)$$

$$= O(n).$$

Key Notation Conventions

- **START**: Represents the global **head** pointer.
- **LINK[PTR]**: Represents the **next** pointer of a specific node block.
- **FREE(PTR)**: Represents the **free()** function used to deallocate the memory back to the system.

Algorithm: UnrolledDESTROYList()

1. [CHECK IF LIST IS EMPTY]

- IF **START** = **NULL**, Then:
 - Write: "The list is already empty."
 - RETURN
- [End of If structure.]

2. [INITIALIZE TRAVERSAL POINTER]

- SET **TEMPPTR** := **START**

3. [OUTER LOOP: NODE TRAVERSAL AND DEALLOCATION]

- REPEAT WHILE **TEMPPTR** ≠ **NULL**:

4. [SAVE NEXT ADDRESS]

- SET **NEXTPTR** := **LINK[TEMPPTR]** $\left(\begin{array}{l} \text{Temporarily saves the address} \\ \text{of the next node block so it is} \\ \text{not lost upon deletion} \end{array} \right)$

5. [FREE MEMORY]

- **FREE(TEMPPTR)** $\left(\begin{array}{l} \text{Deallocates the current node,} \\ \text{including its array of elements and} \\ \text{metadata, all at once} \end{array} \right)$

6. [ADVANCE POINTER]

- SET **TEMPPTR** := **NEXTPTR** $\left(\begin{array}{l} \text{Moves the pointer forward} \\ \text{to the saved address} \end{array} \right)$
- [End of Node Traversal Loop]

7. [RESET GLOBAL POINTER]

- SET **START** := **NULL** $\left(\begin{array}{l} \text{Ensures the global head pointer is} \\ \text{safely nullified to prevent} \\ \text{dangling pointer errors} \end{array} \right)$

8. Write: "The entire list has been destroyed and memory freed."

9. *[END OF ALGORITHM]*

Unrolled List – Why N linearly scales with n ?

- *Let n = total number of elements stored across the entire unrolled linked list.*
- *Let N = total number of nodes in the list.*
- *Let B = capacity of each node ($MAX_ELEMENTS = 4$)*

if $N \approx \left\lceil \frac{n}{4} \right\rceil$, then mathematically, $n \approx 4N$.

*In everyday language, we sometimes think linear means exactly equal (1: 1).
But in mathematics and computer science, linear scaling means something very specific.*

Here is exactly how, $n = 4N$ proves that N scales linearly with n .

In mathematics, two variables scale linearly (or are directly proportional) if one can multiply one by a fixed constant to get the other.

It means the equation does not have any exponents (like n^2) or logarithms (like $\log n$).

In a linear relationship, the variables do not have exponents like n^2 or n^3 , nor do they have logarithms like $\log n$ or $\log_2 n$.

*Instead, the variable is only ever raised to the **power of 1** (which is simply written as **n**). Because the equation is free of higher powers, square roots, or logarithmic curves, graphing it will always produce a perfectly straight line.*

The standard equation for a linear relationship is a straight line:

$$y = m \times x$$

In our unrolled linked list:

$$n = 4 \times N$$

*Because 4 is just a fixed constant (B), the relationship between **n** and **N** forms a perfectly straight line on a graph.*

- If $N = 1, n = 4$
- If $N = 10, n = 40$
- If $N = 100, n = 400$

Big – O notation is designed to classify algorithms by the shape of their growth curve, completely ignoring those constant multipliers. It only cares about the variables.

Because $n \approx 4N$, we can isolate N :

$$\Rightarrow N \approx \frac{1}{4}n$$

$$\Rightarrow O(N) \approx O\left(\frac{1}{4}n\right)$$

$$\Rightarrow O(N) \approx O(n)$$

The actual statement is:

$$1 \leq \text{numElements of each non – empty node} \leq B$$

Let the number of elements in node i be m_i :

$$1 \leq m_i \leq B$$

Suppose there are N non – empty nodes. The total number of elements is:

$$n = m_1 + m_2 + \cdots + m_N$$

- $m_1 = \text{number of elements in node 1.}$
- $m_2 = \text{number of elements in node 2.}$
- $m_3 = \text{number of elements in node 3.}$
- .
- .
- $m_N = \text{number of elements in node } N.$

Since every $m_i \leq B$, then:

$$\bullet \quad m_1 + m_2 + \cdots + m_N \leq B + B + B + \cdots N \text{ times}$$

$$\circ \quad n \leq NB$$

Hence,

$$N \geq \frac{n}{B}$$

or,

$$N \geq \left\lceil \frac{n}{B} \right\rceil$$

Lets say, we have:

Node 1 Elements = [1 3 4] numElements = 3	Node 2 Elements = [5 6 7 8] numElements = 4	Node 3 Elements = [9 10 11 12] numElements = 4
--	--	---

Here , $n = 3 + 4 + 4 = 11$ and $B(MAX_ELEMENTS) = 4$

$$\therefore N = \left\lceil \frac{n}{B} \right\rceil = \left\lceil \frac{11}{4} \right\rceil = \lceil 2.75 \rceil = 3$$

Now say we have :

Node 1 Elements = [1] numElements = 1	Node 2 Elements = [5] numElements = 1	Node 3 Elements = [9 10 11 12] numElements = 4
---	---	---

Here , $n = 1 + 1 + 4 = 6$ and $B(MAX_ELEMENTS) = 4$

$$\therefore N > \left\lceil \frac{n}{B} \right\rceil = \left\lceil \frac{6}{4} \right\rceil = \left\lceil \frac{3}{2} \right\rceil = \lceil 1.5 \rceil = 2$$

Hence , $N \geq \left\lceil \frac{n}{B} \right\rceil$ relation is perfect.

But because every nonempty node contains at least one element:

$$N \leq n$$

That is:

<i>Node 1</i>	<i>Node 2</i>	<i>Node 3</i>
<i>Elements = [1]</i>	<i>Elements = [5]</i>	<i>Elements = [9]</i>
<i>numElements = 1</i>	<i>numElements = 1</i>	<i>numElements = 1</i>

$$\textbf{Here , } N = n = 1 + 1 + 1 = 3$$

Also,

<i>Node 1</i>	<i>Node 2</i>	<i>Node 3</i>
<i>Elements = [1 2 3 4]</i>	<i>Elements = [5 6 7 8]</i>	<i>Elements = [9 10 11 12]</i>
<i>numElements = 4</i>	<i>numElements = 4</i>	<i>numElements = 4</i>

Here , $N < n = 4 + 4 + 4 = 12$

Hence , $N \leq n$,relation is perfect.

Therefore:

$$\left\lceil \frac{n}{B} \right\rceil \leq N \leq n$$

Therefore, $O(N) \subseteq O(n)$, as $N \geq \left\lceil \frac{n}{B} \right\rceil$ and $N \leq n$, and B is fixed constant.

Hence, $O(N) = O(n)$ for fixed B [This is
proved
from
above]

$\therefore$ With fixed B :

$$N = \Theta(n)$$

*$\therefore$ Scaling **N** nodes blocks with **n** number of elements is Linear.*

Unrolled List

–Memory Reference and Allocation

64 – bit architecture:

```
struct Node {  
    int numElements;  
    int elements[MAX_ELEMENTS];  
    struct Node* next;  
};
```

When a compiler processes the assignment `node->numElements = 4;`, it executes the following logical sequence:

- It retrieves the base memory address stored in the `node` pointer, representing where the struct begins in memory (hypothetically `0x1000`).
- It references the struct's blueprint to determine the offset of `numElements`. Because `numElements` is the first declared member, it physically begins at the exact start of the struct block, making its offset `0`.
- It computes the target destination by adding the member's offset to the base address (`0x1000 + 0 = 0x1000`).
- It writes the value `4` starting at that computed address (`0x1000`). Because an integer occupies 4 bytes on a 64-bit architecture, the value is distributed across the memory addresses `0x1000` through `0x1003` as a 32-bit binary sequence.

→ $baseaddress + offset = 0x1000 + 0[bytes] = \text{Where } numElement \text{ is stored.}$

Now, note: **numElements** is integer data type and 4 bytes size for 64 – bit architecture .

$$0x1000_{16} = 1 \times 16^3 + 0 \times 16^2 + 0 \times 16^1 + 0 \times 16^0 = 1 \times 4096$$

$$= 4096_{10} = \frac{4096_{10}}{4 \text{ bytes}} = \frac{4096_{10}}{4_{10}} = 1024_{10} \text{ [Hence, divisible by 4 bytes]}$$

$\therefore$ Valid address to store Integer data.

Hence allocated addresses are :

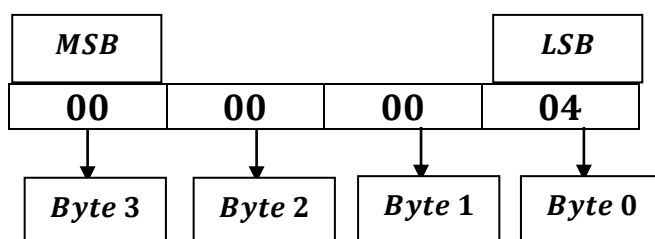
0x1000

0x1001

0x1002

0x1003

Hence , offset: 0 – 3.



0x1000 = 0x04 → Least Significant Byte

0x1001 = 0x00

0x1002 = 0x00

0x1003 = 0x00 → Most Significant Byte

As we know everything is stored in Memory address at binary format.

0x1000 = 0000 0100[8 bits]

0x1001 = 0000 0000[8 bits]

0x1002 = 0000 0000[8 bits]

0x1003 = 0000 0000[8 bits]

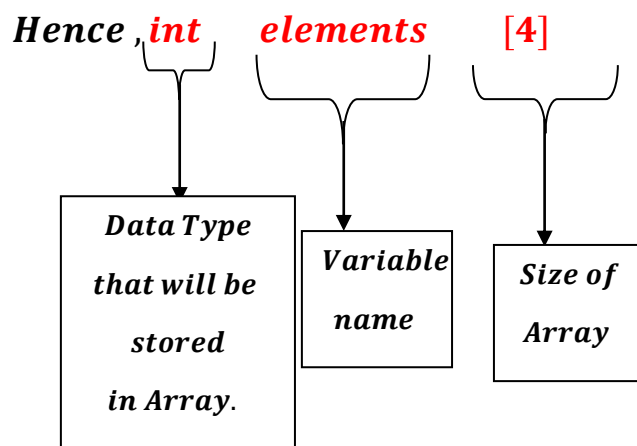
[8 × 4 = 32 bits]

Next , compiler sees:

```
int elements[MAX_ELEMENTS];
```

And ,

```
#define MAX_ELEMENTS 4
```



Therefore datatype is integer i. e. 4 bytes :

According to the provided document, when the compiler sees `int elements [4];`, it thinks:

- "Get the base memory address stored in the pointer node." (e.g., `0x1000`)
- "Look up the definition of the `Node` struct. The `numElements` integer occupies the first 4 bytes, meaning the offset for the `elements` array is 4."
- "Calculate the starting address: `base address + offset => 0x1000 + 4 => 0x1004`."
- "Recognize that `elements[4]` requires space for 4 integers. Since each integer is 4 bytes, reference or allocate a 16-byte memory block starting at `0x1004` (spanning `0x1004` through `0x1013`)."

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1000	<i>E.g.: numElements = 1, 2 ... etc.</i>	<i>numElements[0x1000 – 0x1003] 4 bytes for int</i>
0x1004	<i>E.g.: elements = {15, 42, ... etc.}</i>	<i>elements[0x1004 – 0x1013] 16 bytes for int array</i>

<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1004	<i>elements[0]</i>	<i>elements[0] → [0x1004 – 0x1007] 4 bytes for int</i>
0x1008	<i>elements[1]</i>	<i>elements[1] → [0x1008 – 0x100b] 4 bytes for int</i>

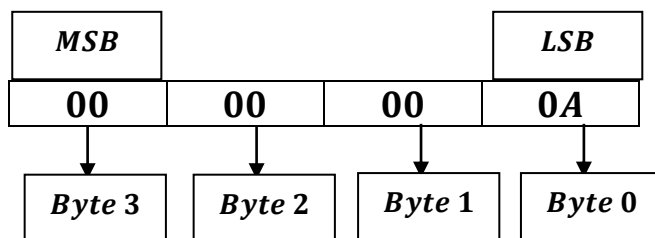
0x100c	<i>elements[2]</i>	<i>elements[2] →</i> <i>[0x100c – 0x100f]</i> <i>4 bytes for int</i>
0x1010	<i>elements[3]</i>	<i>elements[3] →</i> <i>[0x1010 – 0x1013]</i> <i>4 bytes for int</i>

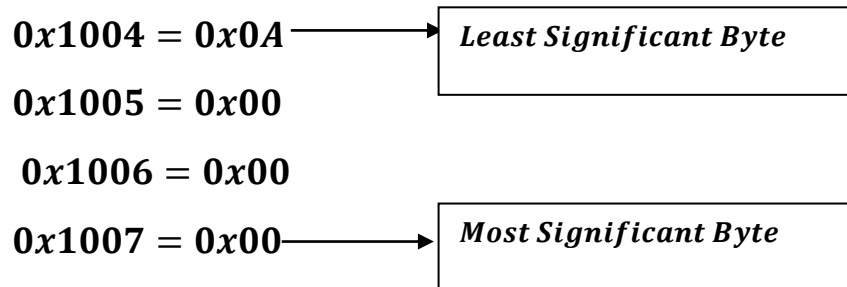
	<i>Address</i>	<i>Values</i>
<i>Elements</i> [0]	0x1004	10
<i>Elements</i> [1]	0x1008	20
<i>Elements</i> [2]	0x100c	30
<i>Elements</i> [3]	0x1010	40

Elements[0] = 10

0x1004 is divisible by 4 bytes , hence valid address to start with.

offset: 4 – 7.





As we know everything is stored in Memory address at binary format.

$0x1004 = 0000\ 1010$ [8 bits]

$0x1005 = 0000\ 0000$ [8 bits]

$0x1006 = 0000\ 0000$ [8 bits]

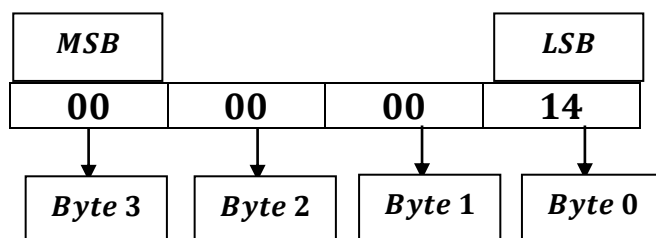
$0x1007 = 0000\ 0000$ [8 bits]

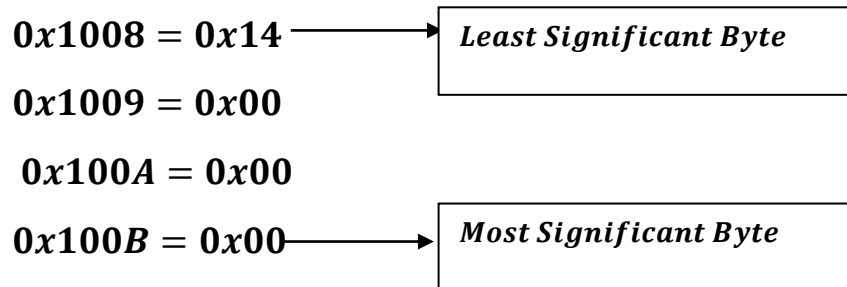
[8 × 4 = 32 bits]

Elements[1] = 20

0x1008 is divisible by 4 bytes , hence valid address to start with.

offset: 8 – 11.





As we know everything is stored in Memory address at binary format.

$0x1008 = 0001\ 0100[8\ bits]$

$0x1009 = 0000\ 0000[8\ bits]$

$0x100A = 0000\ 0000[8\ bits]$

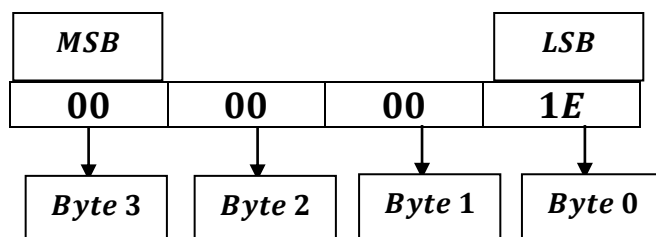
$0x100B = 0000\ 0000[8\ bits]$

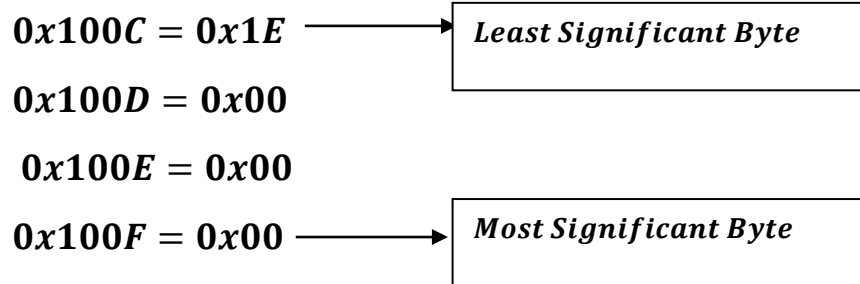
$[8 \times 4 = 32\ bits]$

Elements[2] = 30

0x100C is divisible by 4 bytes , hence valid address to start with.

offset: 12 – 15.





As we know everything is stored in Memory address at binary format.

$0x100C = 0001\ 1110[8\ bits]$

$0x100D = 0000\ 0000[8\ bits]$

$0x100E = 0000\ 0000[8\ bits]$

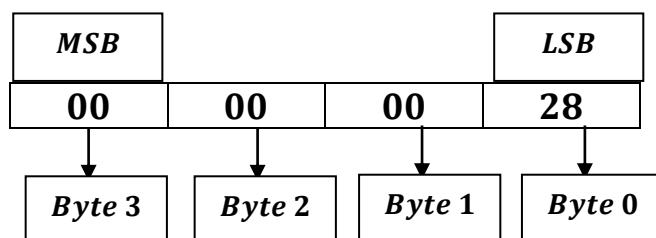
$0x100F = 0000\ 0000[8\ bits]$

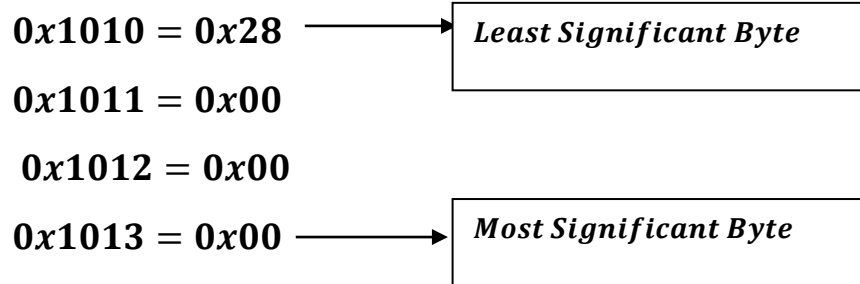
$[8 \times 4 = 32\ bits]$

Elements[3] = 40

0x1010 is divisible by 4 bytes , hence valid address to start with.

offset: 15 – 18.





As we know everything is stored in Memory address at binary format.

0x1010 = 0010 1000[8 bits]

0x1011 = 0000 0000[8 bits]

0x1012 = 0000 0000[8 bits]

0x1013 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Padding

Compiler Logic for Struct Padding

- "Evaluate the current memory offset after the elements array. Because elements[3] concludes at 0x1013, the next available consecutive memory address is 0x1014."
- "Look up the alignment rules for the 64-bit architecture specified in the struct's blueprint. The upcoming member, struct Node* next, is a memory pointer. On a 64-bit system, pointers require 8 bytes and must be aligned to a memory address divisible by 8."

- "Calculate the alignment: 0x1014 (decimal 4116) is not evenly divisible by 8. Determine that 4 bytes of padding are necessary to reach the next divisible boundary (0x1018)."
- "Insert a 4-byte padding block starting at 0x1014 (spanning 0x1014 through 0x1017) to satisfy structural alignment, establishing the correct new offset at 24 bytes."

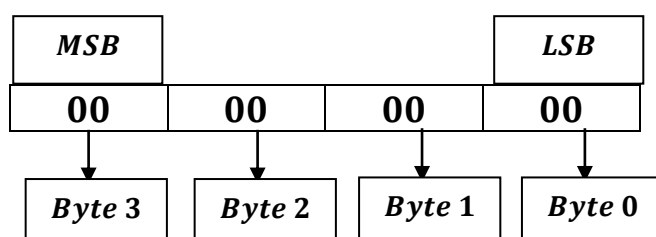
<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
		<i>padding → [0x1014 – 0x1017] 4 bytes for padding</i>

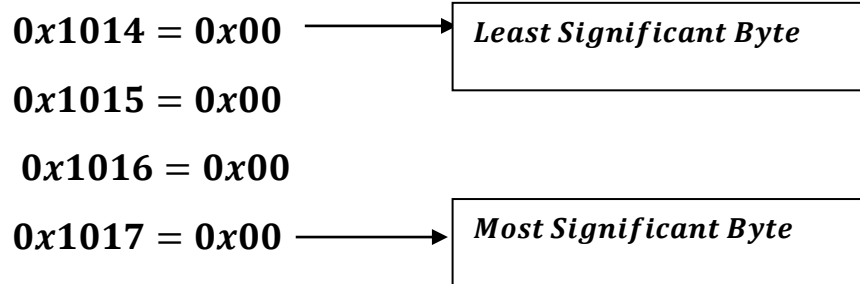
offset: 19 – 22.

As, next is 8 byte pointer and 0x1014 divided by 8 byte*

$$0x1014 = \frac{4116_{10}}{8 \text{ byte}} = 514 \text{ remainder } 4, \text{ hence not divisible,}$$

we need address of 8 byte allignment, therefore we need 4 byte padding.





As we know everything is stored in Memory address at binary format.

0x1014 = 0000 0000[8 bits]

0x1015 = 0000 0000[8 bits]

0x1016 = 0000 0000[8 bits]

0x1017 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

** next pointer*

Compiler Logic for struct Node* next

- "Get the base memory address stored in the pointer node."
(e.g., 0x1000)
- "Look up the definition of the Node struct. After accounting for numElements (4 bytes), elements[4] (16 bytes), and the inserted architectural padding (4 bytes), the resulting offset for the next pointer is 24 bytes (0x18 in hexadecimal)."

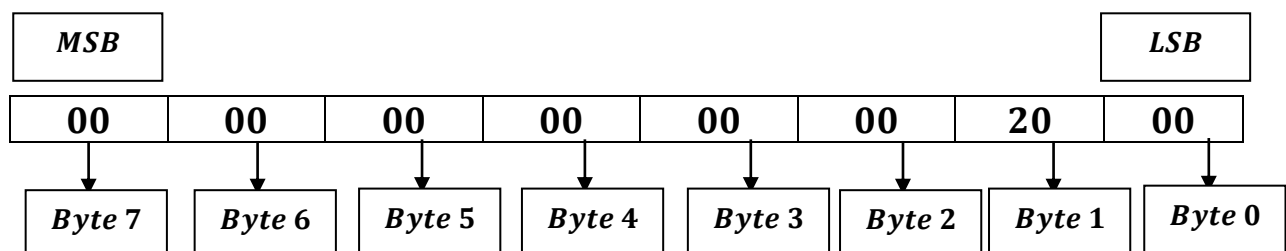
- "Calculate the starting address: base address + offset => 0x1000 + 0x18 => 0x1018."
- "Recognize that struct Node* next is a pointer operating on a 64-bit architecture. Reference or allocate an 8-byte memory block starting at 0x1018 (spanning 0x1018 through 0x101F)."

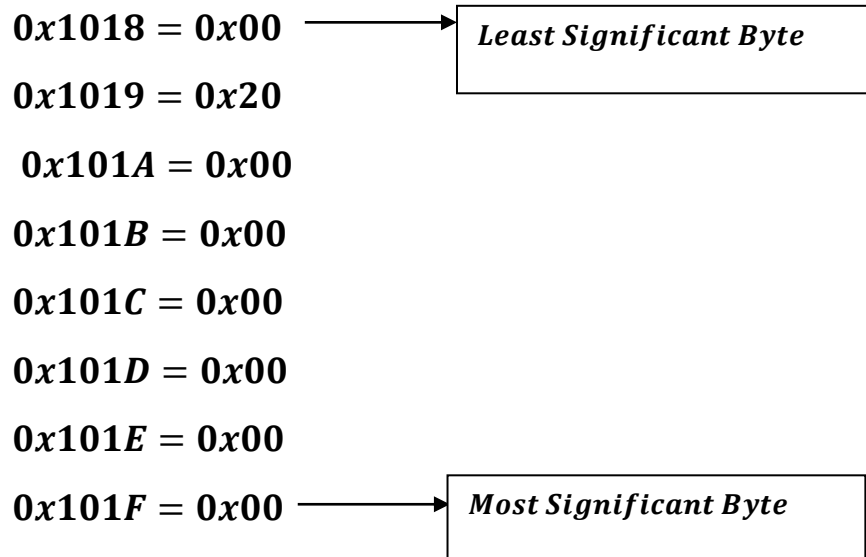
<i>Address at which value is stored</i>	<i>Value</i>	<i>offset used</i>
0x1018	<i>E.g.: Address: 0x2000</i>	* next → [0x1018 – 0x101F] 8 bytes for pointer variable

offset: 23 – 30.

$$0x1018_{16} = 4120_{10} \text{ divided by 8 bytes} = \frac{4120_{10}}{8_{10}} = 515 \text{ and}$$

Remainder = 0, hence valid memory address to start with.





As we know everything is stored in Memory address at binary format.

$0x1018 = 0000\ 0000[8\ bits]$
 $0x1019 = 0010\ 0000[8\ bits]$
 $0x101A = 0000\ 0000[8\ bits]$
 $0x101B = 0000\ 0000[8\ bits]$
 $0x101C = 0000\ 0000[8\ bits]$
 $0x101D = 0000\ 0000[8\ bits]$
 $0x101E = 0000\ 0000[8\ bits]$
 $0x101F = 0000\ 0000[8\ bits]$

$[8 \times 8\ bits = 64\ bits]$

For 32 – bit architecture ,

Pointer Variable → 4 bytes [32 bit]

Integer Variable → 4 bytes [32 bit]

numElements = 4

0x1000 = 0x04

0x1001 = 0x00

0x1002 = 0x00

0x1003 = 0x00

in Bits :

0x1000 = 0000 0100[8 bits]

0x1001 = 0000 0000[8 bits]

0x1002 = 0000 0000[8 bits]

0x1003 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Elements[0] = 10

0x1004 = 0x0A

0x1005 = 0x00

0x1006 = 0x00

0x1007 = 0x00

in Bits :

0x1004 = 0000 1010[8 bits]

0x1005 = 0000 0000[8 bits]

0x1006 = 0000 0000[8 bits]

0x1007 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Elements[1] = 20

0x1008 = 0x14

0x1009 = 0x00

0x100A = 0x00

0x100B = 0x00

in Bits :

0x1008 = 0001 0100[8 bits]

0x1009 = 0000 0000[8 bits]

0x100A = 0000 0000[8 bits]

0x100B = 0000 0000[8 bits]

[8 × 4 = 32 bits]

Elements[2] = 30

$0x100C = 0x1E$

$0x100D = 0x00$

$0x100E = 0x00$

$0x100F = 0x00$

in Bits:

$0x100C = 0001\ 1110[8\ bits]$

$0x100D = 0000\ 0000[8\ bits]$

$0x100E = 0000\ 0000[8\ bits]$

$0x100F = 0000\ 0000[8\ bits]$

$[8 \times 4 = 32\ bits]$

Elements[3] = 40

$0x1010 = 0x28$

$0x1011 = 0x00$

$0x1012 = 0x00$

$0x1013 = 0x00$

in Bits:

0x1010 = 0010 1000[8 bits]

0x1011 = 0000 0000[8 bits]

0x1012 = 0000 0000[8 bits]

0x1013 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

** next pointer*

0x1014 = 0x00

0x1015 = 0x20

0x1016 = 0x00

0x1017 = 0x00

in Bits:

0x1014 = 0000 0000[8 bits]

0x1015 = 0010 0000[8 bits]

0x1016 = 0000 0000[8 bits]

0x1017 = 0000 0000[8 bits]

[8 × 4 = 32 bits]

*On a typical 32 – bit architecture, a 4 – byte pointer occupies 32 bits.
A 4 – byte integer also occupies 32 bits. If the integer is followed
by a 4 – byte – aligned pointer, no padding is normally required
because the 4 – byte integer already ends at an offset that is aligned
to 4 bytes. Thus, the pointer can immediately follow the integer while
maintaining 4 – byte alignment.*

Unroll List

–Class Tweaks

```
void displayList() const {...}
```

```
void searchElement(int val) const {...}
```

*Read – Only Guarantee: Placing **const** at the end of a method signature tells the compiler,*

"This function is only allowed to look at data, not alter it."

```
// Function to destroy the entire list and free all memory
void destroyList(bool printMessage = true) {
    if (head == nullptr) {
        if (printMessage) cout << "The list is already
empty.\n";
        return;
    }

    Node* temp = head;
    while (temp != nullptr) {
        Node* nextNode = temp->next;
        delete temp; // Use delete instead of free()
        temp = nextNode;
    }

    head = nullptr;
    if (printMessage) cout << "The entire list has been
destroyed and memory freed.\n";
}
```

We know delete keyword has two functions:

```
delete temp;
|
|-- 1. Call Destructor (~Node())    -> Cleans up internal resources (runs the default compiler destructor since our struct only holds basic data types)
|
|-- 2. Free Memory (operator delete) -> Releases the Node's own memory back to the Heap .
```

Note: `~Node()` this destructor manually not provided in the Node struct, C++ compiler automatically generates an implicit default destructor behind the scenes.

When the program executes `delete temp;`, it still calls `~Node()`, but it calls this invisible, compiler – generated version.

Why it is not in the code

We only need to manually write a destructor if an object manages its own complex resources (like opening a file, or using the new keyword to create internal variables). Our Node struct only contains primitive data types: an integer, an integer array, and a raw pointer. The compiler already knows exactly how to clean up primitive data types, so it handles it automatically.

```
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    Node* next;

    // Constructor
    Node(int val) {
        numElements = 1;
```

```

        elements[0] = val;
        next = nullptr;
    }

    // Explicit Destructor
    ~Node() {
        // Left empty because primitive types (int, int[], pointer)
        // are automatically destroyed by the compiler.
    }
};

```

Even though it is invisible in our current file, C++ guarantees that a destructor always exists and is always executed when we use the delete keyword.

And

```

~UnrolledLinkedList() {
    if (head != nullptr) {
        destroyList(false); // Suppress output during automatic
destruction
    }
}

```

***~UnrolledLinkedList()** is the destructor for the entire class manager. It only runs when the overarching list object (created in **main()**) finishes its lifecycle and is destroyed.*

*When we define **UnrolledLinkedList list;** in our **main()** function, the class object itself lives on the computer's temporary Stack memory, while all the individual nodes it creates are dynamically allocated on the Heap.*

When the program ends, as the object simply goes out of scope, the compiler automatically triggers `~UnrolledLinkedList()`. The destructor acts as the bridge: it uses our custom `destroyList(false)` loop to actively sweep the Heap and `delete` every single node, ensuring no memory leaks remain before the class object itself is naturally erased from the Stack.

```
~UnrolledLinkedList() {  
    if (head != nullptr) {  
        destroyList(false); // Suppress output during automatic  
destruction  
    }  
}
```

And,

```
// Function to destroy the entire list and free all memory  
void destroyList(bool printMessage = true) {  
    if (head == nullptr) {  
        if (printMessage) cout << "The list is already  
empty.\n";  
        return;  
    }  
  
    Node* temp = head;  
    while (temp != nullptr) {  
        Node* nextNode = temp->next;  
        delete temp; // Use delete instead of free()  
        temp = nextNode;  
    }  
  
    head = nullptr;  
    if (printMessage) cout << "The entire list has been  
destroyed and memory freed.\n";  
}
```

As it pass ***destroyList(bool printMessage = true)***, as it passes ***false*** ,
in the argument, then:

The single if statements :

```
if (printMessage) cout << "The list is already  
empty.\n";
```

And,

```
if (printMessage) cout << "The entire list has been  
destroyed and memory freed.\n";  
}
```

becomes false and body of the if statements never executes.

These two if bodies only executes then:

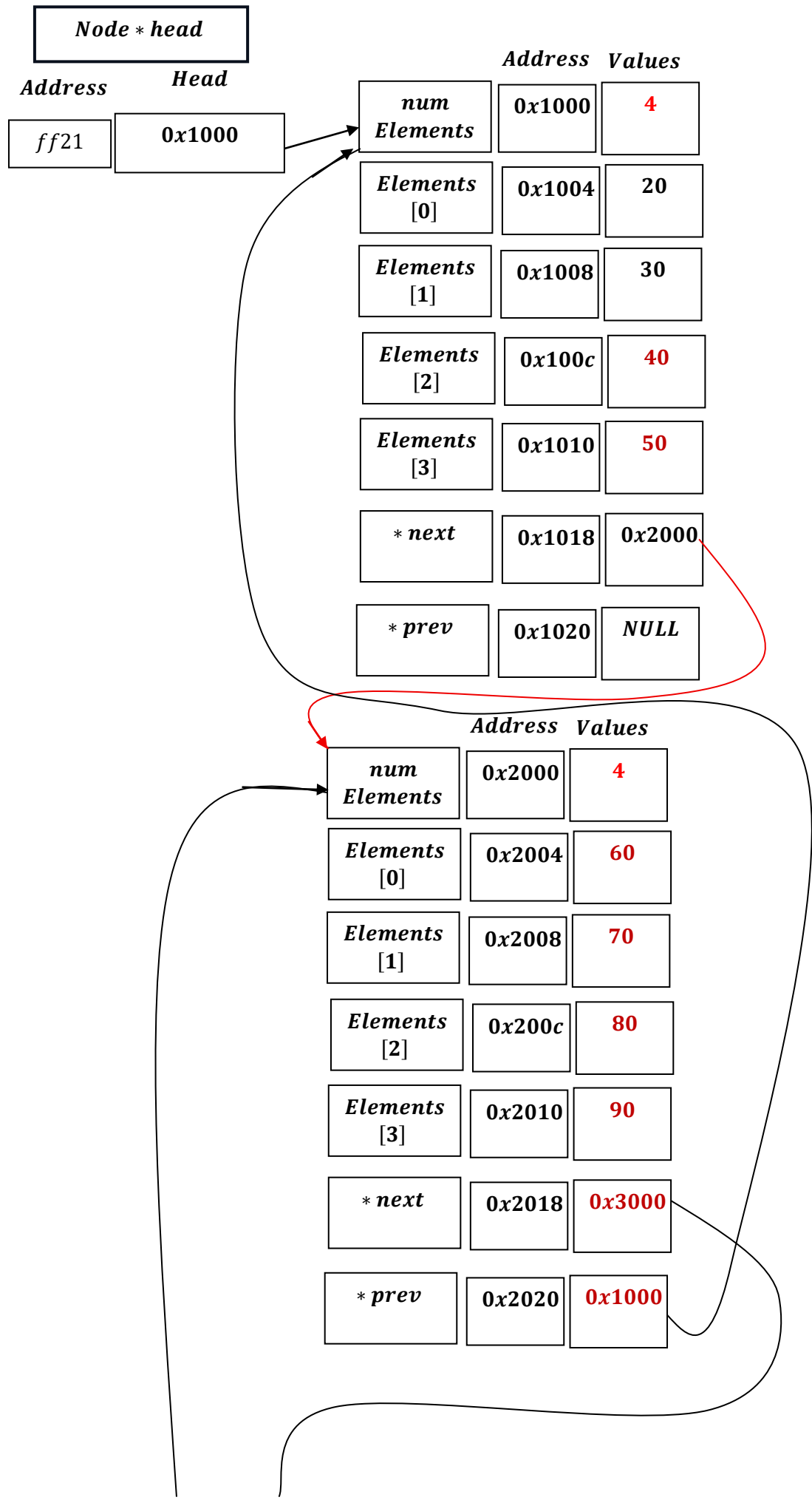
```
destroyList(true);
```

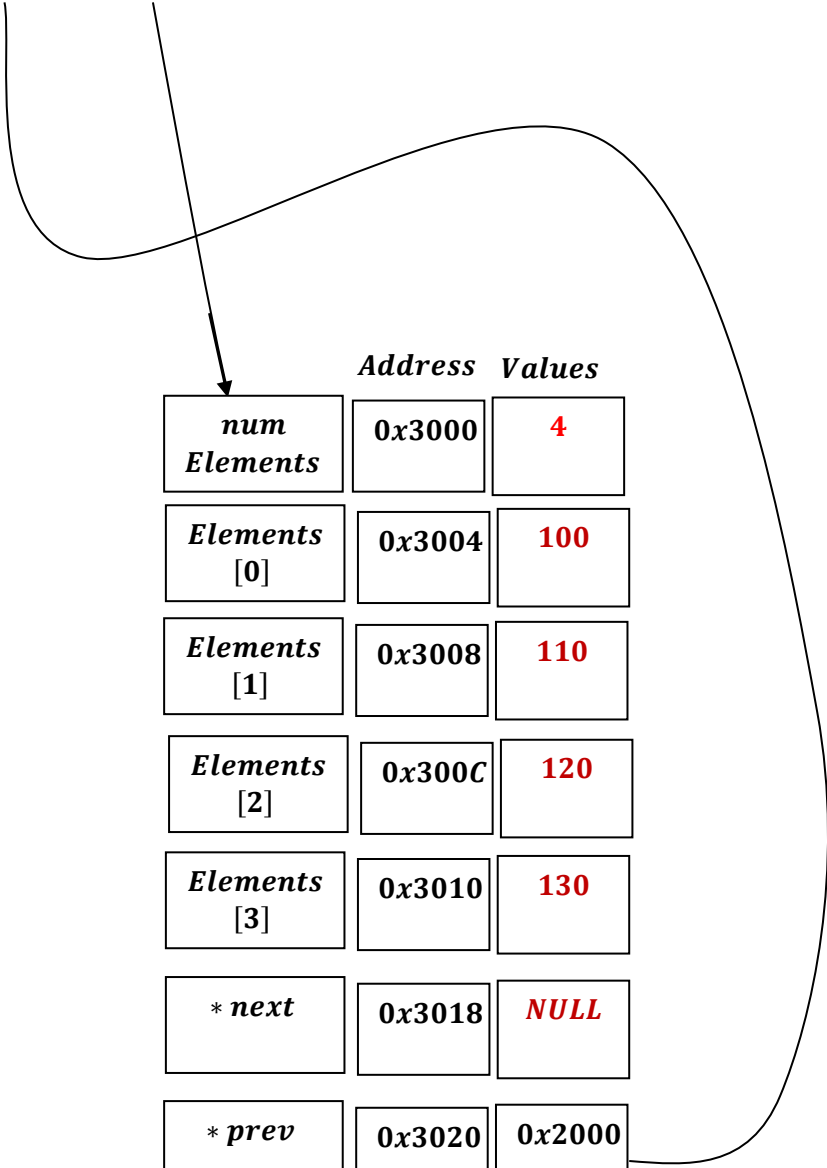
Unroll List – Doubly, Circular, Doubly – Circular

We have learnt about Singly Linked Unroll list ,now how would Doubly Linked Unroll List look like: –

1. Doubly Unrolled Linked List

```
#define MAX_ELEMENTS 4
struct Node {
    int numElements;
    int elements[MAX_ELEMENTS];
    struct Node* next;
    struct Node* prev; // New pointer for backward traversal
};
```

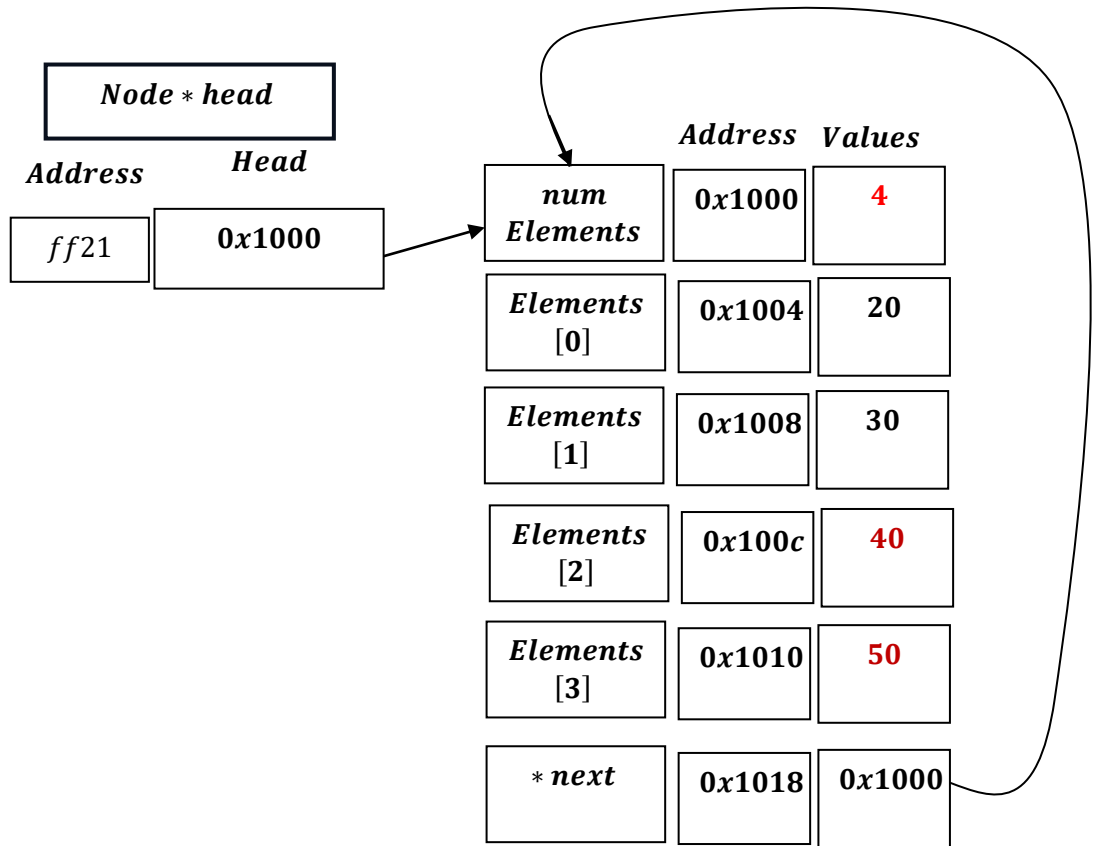




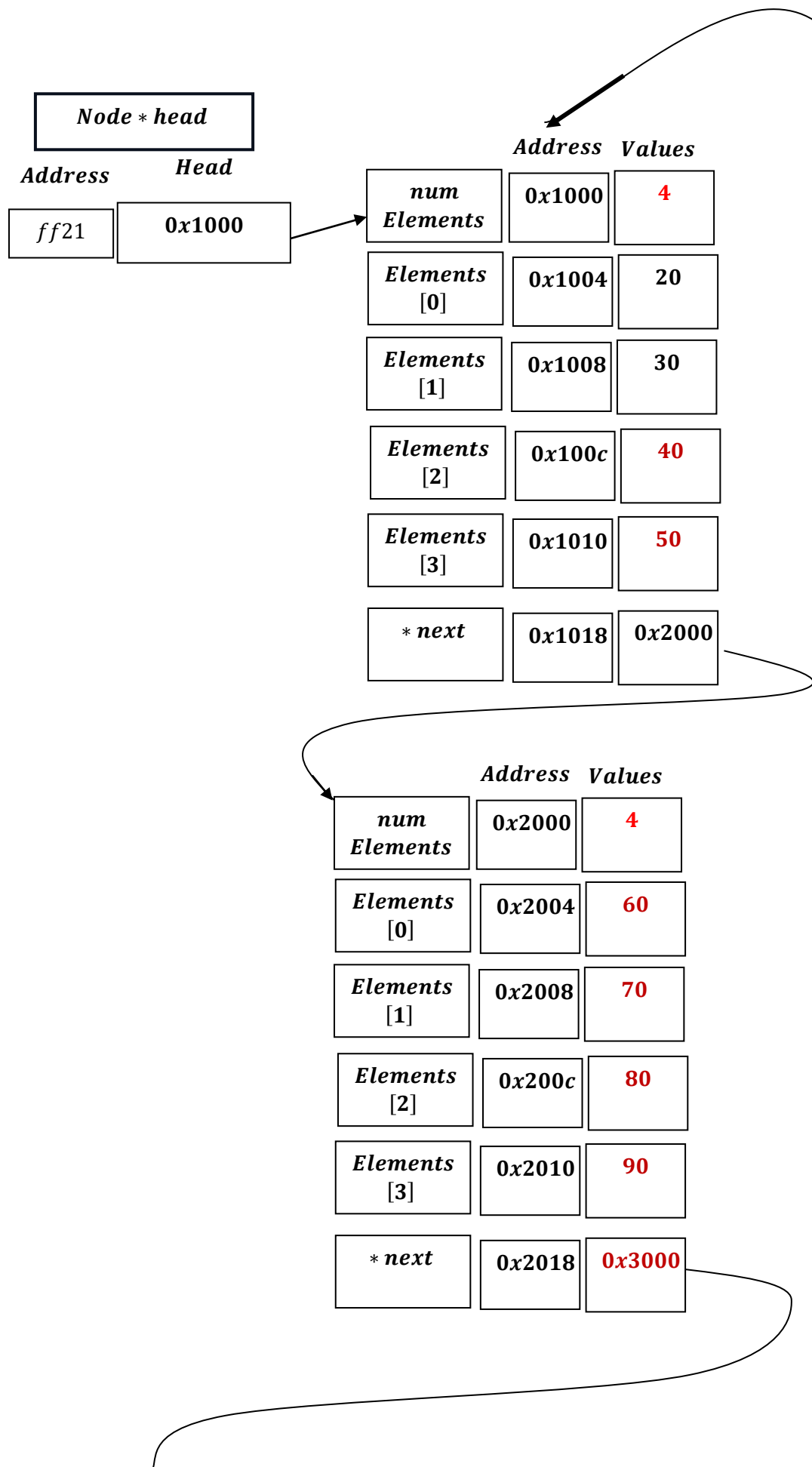
	<i>Address</i>	<i>Values</i>
<i>num Elements</i>	0x3000	4
<i>Elements [0]</i>	0x3004	100
<i>Elements [1]</i>	0x3008	110
<i>Elements [2]</i>	0x300C	120
<i>Elements [3]</i>	0x3010	130
<i>* next</i>	0x3018	NULL
<i>* prev</i>	0x3020	0x2000

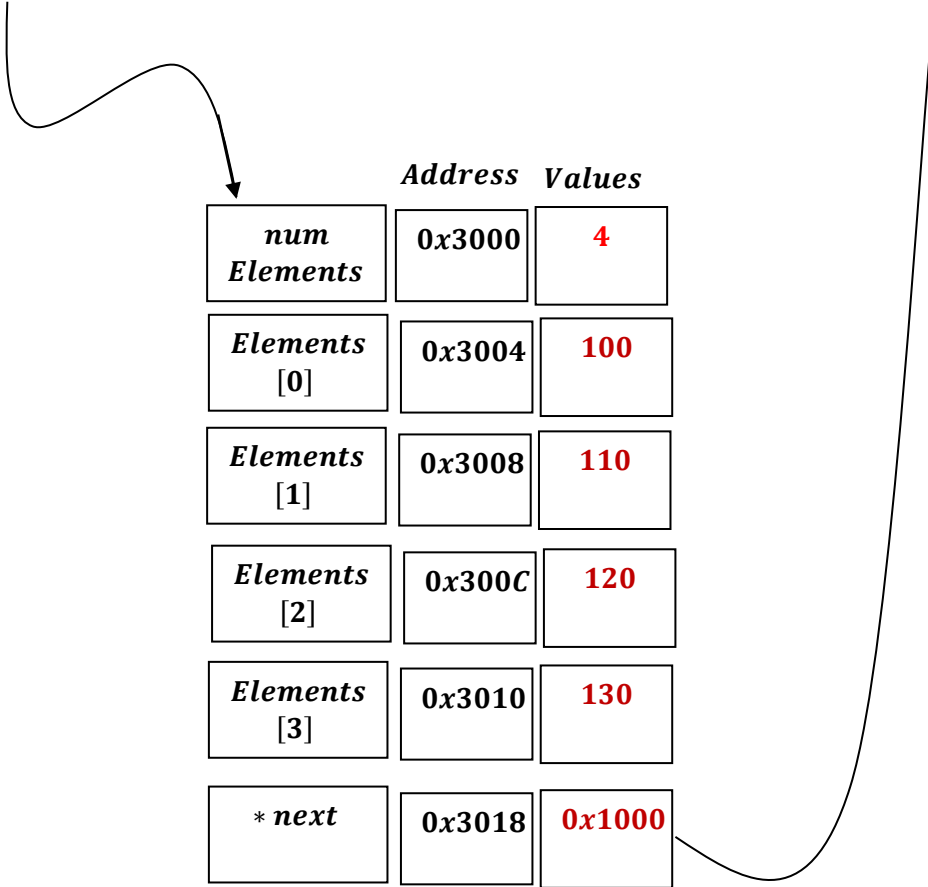
2. Circular Unrolled Linked List

2.A. With Single node:



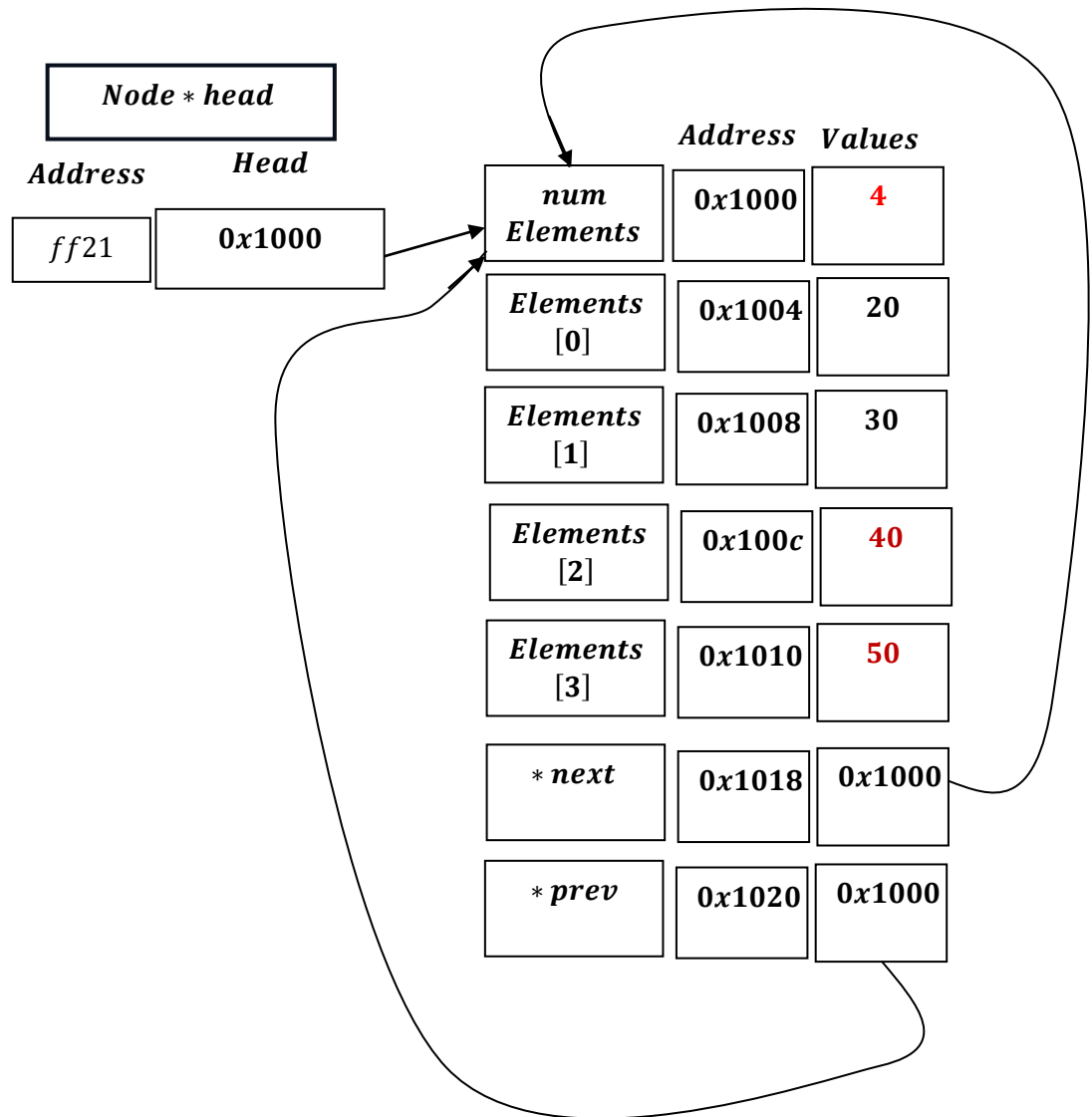
2. B. With Multiple nodes:



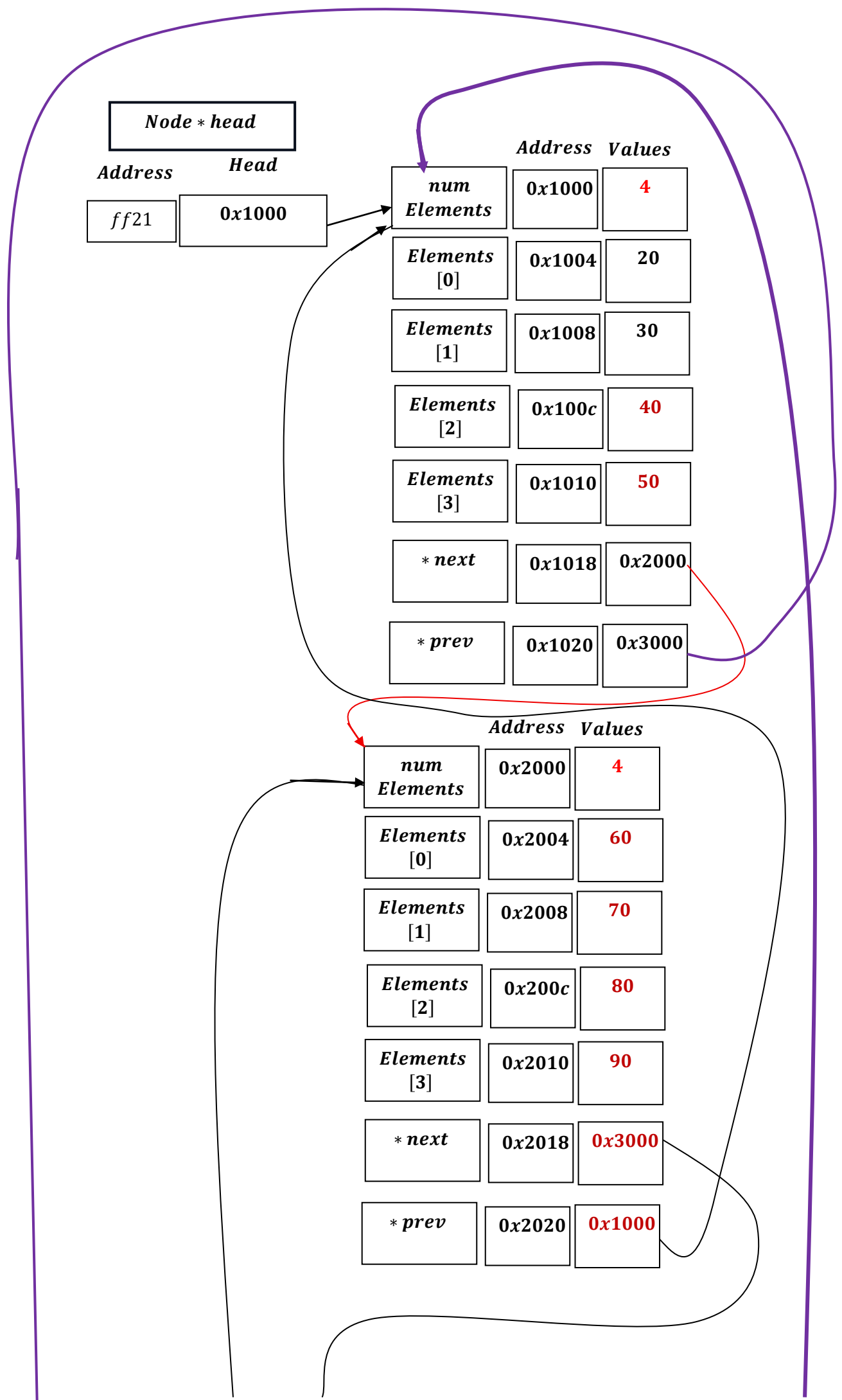


3. Circular – Doubly Unrolled Linked List

3.A. With Single node:



3.B. With Multiple nodes:



The diagram illustrates a linked list structure. A vertical table contains fields for the number of elements, individual elements, the next pointer, and the previous pointer. A black arrow points to the 'num Elements' field. A purple arrow points to the 'num Elements' field. Another purple arrow points from the '* next' field to the top right. A black arrow points from the '* prev' field to the top left. The table is as follows:

	<i>Address</i>	<i>Values</i>
<i>num</i> <i>Elements</i>	0x3000	4
<i>Elements</i> [0]	0x3004	100
<i>Elements</i> [1]	0x3008	110
<i>Elements</i> [2]	0x300C	120
<i>Elements</i> [3]	0x3010	130
* <i>next</i>	0x3018	0x1000
* <i>prev</i>	0x3020	0x2000